

NLP-Based Phishing and Social Engineering Email Detection

COIT20265 Networks and Information Security Project

Group Members: PG6 - SYD - T2 2026

Student Name	Student ID
Mohammad Riadur Rashid	12296115
Prizma Shestha	12269269
Sujina Mahato	12290331
Md Sadman Sakib	12282515

Project Mentor

Fariza Sabrina

Unit Coordinator, Lecturer, Tutor

Date: 10-10-2026

Sydney, CQUniversity Australia

EXECUTIVE SUMMARY

Phishing emails remain a significant cybersecurity threat because they use deceptive language and social engineering techniques to persuade recipients to disclose sensitive information or take unsafe actions. This project developed an explainable Natural Language Processing (NLP) system to classify email text as phishing or legitimate. The main objective was to combine strong detection performance with explanations that help users understand the reasons behind individual predictions.

The project used the [zefang-liu/phishing-email-dataset](#). After preprocessing and duplicate removal, 17,519 emails were retained, comprising 10,978 legitimate emails and 6,541 phishing emails. Classical machine learning models, including TF-IDF with Logistic Regression and Support Vector Machine, were evaluated alongside several transformer-based architectures. The experiments also investigated hybrid models that combine contextual language representations with 12 handcrafted features associated with phishing behaviour.

The final classifier was based on DeBERTa-v3-large with a gated fusion mechanism. This architecture combines contextual information learned by the transformer with handcrafted phishing-related features to support classification. On the held-out test dataset, the proposed model achieved 99.49% accuracy and an F1-score of 99.49%. For the phishing class, it achieved 99.69% precision, 98.93% recall and a 99.31% F1-score. These results demonstrate strong performance on the evaluated dataset, although they do not establish the same level of performance against all real-world or previously unseen phishing attacks.

To improve interpretability and demonstrate practical use, the project developed two local prototype configurations using SHAP and LIME. The SHAP configuration integrates a FastAPI backend, Streamlit interface and Manifest V3 browser extension. The LIME configuration uses a Flask backend and Chrome extension. Both configurations provide email classification, prediction confidence and post-hoc explanations to help users examine the features influencing a prediction.

Overall, the project achieved its primary objective by developing a high-performing hybrid phishing email classifier and integrating explainability into user-facing prototypes. The system is intended as a supplementary detection tool rather than a replacement for existing email security controls. Its current scope is limited to email text and does not comprehensively assess sender authentication, attachments or live domain reputation. Future work should focus on external dataset validation, testing against evolving and obfuscated phishing emails, and further security, usability and deployment testing to assess its suitability for practical use.

Table of Contents

Executive Summary	2
1. Introduction.....	6
2. System Overview	7
3. GitHub Link	9
4. Delivered Technical Artefacts	10
5. Contributions.....	13
6. Project Overview	14
6.1. Project Background.....	14
6.2. Problem Statement	14
6.3. Project Aim and Objectives	15
6.3.1 Project Aim	15
6.3.2 Project Objectives	15
6.4. Project Scope	15
6.5. Stakeholders and Requirements	16
6.6. Success Criteria and Acceptance Test.....	16
7. Background and Related Work	17
7.1. Phishing and Social Engineering	17
7.2. NLP-Based Phishing Detection	17
7.3. Classical NLP Approaches.....	18
7.4. Transformer-Based Approaches.....	19
7.5. Explainable Phishing Detection.....	20
8. Project Methodology.....	22
8.1. Overall Methodology.....	22
8.2. Process of Experiments: NLP-based Classification.....	22
8.2.1 Dataset Description.....	24
8.2.2 Data Preprocessing.....	24
8.2.3 Dataset Split	25
8.2.4 Baseline Models.....	26
8.2.5 Hybrid Architectures	28
8.2.6 Model Optimization.....	31
8.2.7 Proposed Gated Hybrid DeBERTa Model	32
8.2.8 Model Training and Evaluation	32

9.	Results, Analysis and Comparison.....	35
9.1.	Overview of Experimental Evaluation.....	35
9.2.	Overall Performance Comparison.....	35
9.3.	Text Representation and Model Capacity	37
9.4.	Lightweight Transformer	37
9.5.	Handcrafted Phishing-Related Features.....	38
9.6.	The Feature-Fusion Strategy	39
9.7.	Effect of Architectural Complexity.....	39
9.8.	Effect of Hybridisation Across Transformer Backbones	41
9.9.	Training and Validation Loss of Gated Hybrid DeBERTa Model	41
9.10.	Phishing-Class Performance of Transformer Models	42
9.11.	Confusion Matrix and Error Analysis	43
9.12.	Comparison with Existing Work Using the Same Dataset	45
9.13.	Discussion of Findings.....	47
9.14.	Experimental Findings	48
9.15.	Final Model Selection	49
9.16.	Limitations of the Results	50
10.	System Design and Architecture.....	51
10.1.	SHAP-Based System Design and Architecture.....	53
10.1.1	Architectural Design Principles	54
10.1.2	FastAPI Backend Design	54
10.1.3	Streamlit Interface Design	55
10.1.4	Explainability Architecture	56
10.1.5	SHAP Computational Refinement.....	57
10.1.6	Browser Extension Design.....	58
10.1.7	Browser Extension Components.....	61
10.2.	LIME-Based System Design and Architecture	66
10.2.1	Explanation Component.....	67
10.2.2	API and Interface	67
10.2.3	Deployment Environment.....	68
10.3.	SHAP and LIME within the System	68
10.4.	SHAP and LIME Comparison	69
10.5.	Security and Privacy Considerations	70

11. Deployment and End-to-End Demonstration.....	72
11.1. SHAP-Based Deployment and End-to-End Demonstration	72
11.1.1 Demonstration Scenario.....	75
11.1.2 Phishing / Legitimate Verdicts	80
11.1.3 Demonstration Scenario.....	83
11.1.4 Confidence and LIME Explanations.....	83
12. Security, Privacy and Risk Management	86
12.1. Security Considerations	86
12.2. Privacy Risks and Recommended Controls.....	90
12.3. Project Risk Register.....	91
12.4. Risk Mitigation	93
12.5. Ethical and Professional Considerations.....	95
12.5.1 Ethical Issues in AI-Based Phishing Detection.....	95
12.6. Societal and Organizational Implications	96
12.6.1 Professional Responsibility and Accountability	97
13. Teamwork Framework.....	98
13.1. Communication and Coordination.....	98
13.2. Integration and Shared Responsibility	98
13.3. Teamwork Outcomes	98
13.4. Project Tools, Technologies and AI Usage	98
14. Development Environments and Programming Tools.....	100
14.1. Diagramming, Documentation and Visual Design Tools.....	101
15. Artificial Intelligence Tools and Their Application	102
16. Quality Assurance and Change Management	103
17. Limitations and Future Work	104
17.1. Current Limitations.....	104
17.2. Dataset and Generalisation Limitations.....	104
17.3. Future Work	104
18. Conclusion	106
References.....	107

1. INTRODUCTION

Email remains a critical communication medium for organisations, but its widespread use also makes it an attractive channel for social-engineering attacks. Phishing emails attempt to deceive, pressure, intimidate or persuade recipients into disclosing sensitive information, transferring money, clicking malicious links or performing other unsafe actions. In Australia, phishing and business email compromise remain significant cybersecurity concerns and can result in substantial financial losses (Australian Signals Directorate 2025). These attacks are becoming increasingly difficult to identify because malicious messages can resemble legitimate invoices, password notifications, shared-document alerts and internal departmental requests.

Traditional email security controls remain essential, but they may not detect every type of phishing attack. Sender reputation checks, digital signatures, blocklists and malicious URL detection are valuable when identifiable technical indicators are available. Their effectiveness may be reduced when attackers use newly registered domains, compromised legitimate accounts or carefully crafted text-only messages without attachments or recognisable malicious links. Keyword-based filtering also has limitations because terms such as “invoice”, “password” and “urgent” can appear in both legitimate and fraudulent messages. Natural Language Processing (NLP) offers a complementary approach by analysing the linguistic and contextual patterns of email content rather than relying exclusively on predefined indicators. Recent research highlights the potential of machine learning and deep learning for phishing detection, while identifying challenges associated with evolving attack strategies, changes in datasets and variations in organisational communication patterns (Kyaw, Gutierrez & Ghobakhlou 2024).

This project develops and evaluates an NLP-based phishing email detection system designed to identify potentially malicious messages through contextual language analysis and engineered phishing indicators. The proposed classifier combines a fine-tuned DeBERTa-v3-large transformer with twelve handcrafted features representing social-engineering signals, contact and URL patterns, punctuation, capitalisation and message length. A gated fusion mechanism integrates the contextual representations learned by the transformer with the explicitly engineered features to support binary classification of emails as phishing or legitimate. The project also investigates explainability to make the system's predictions more interpretable and useful for human decisionmaking.

Two explainability and deployment configurations were implemented around the proposed hybrid classifier. The first integrates SHAP with a FastAPI backend, a Streamlit interface and a Manifest V3 browser extension. The second uses LIME with a Flask REST API and a Chrome extension. Both configurations provide a classification verdict, prediction confidence and post-hoc explanations intended to help users understand the factors influencing a prediction. The overall system is designed to complement existing email security controls by

providing additional linguistic analysis and interpretable predictions, supporting users in assessing suspicious emails rather than replacing established security measures or human judgement.

2. SYSTEM OVERVIEW

The developed system is an explainable Natural Language Processing (NLP)-based phishing email detection prototype designed to classify email messages as either legitimate or phishing and provide supporting explanations for its predictions. It combines a fine-tuned DeBERTa-v3-large transformer with twelve handcrafted features through a gated hybrid classification architecture. The system also incorporates two explainability and deployment configurations, using SHAP and LIME, to make the classification results more understandable to users.

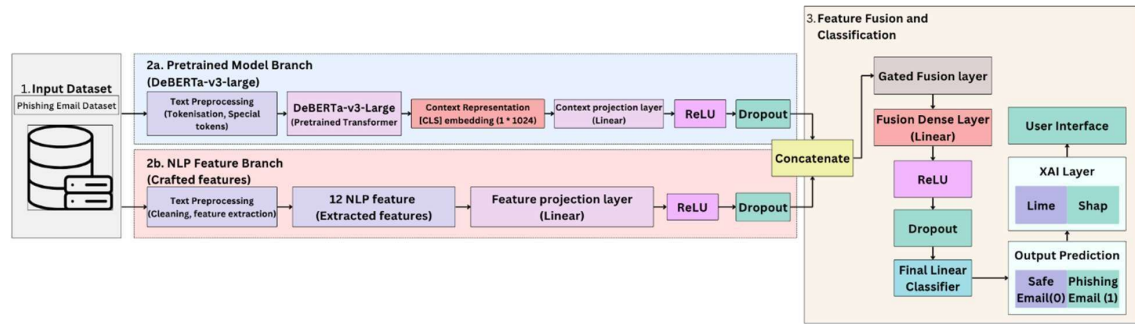


Figure 6.1.1: Overall System overview

The system follows a common processing and classification pipeline. An email is submitted through a user interface, a browser extension or an API request. The input text is then validated and preprocessed before being passed to two parallel processing branches. The transformer branch uses DeBERTa-v3-large to capture contextual and semantic information from email. In parallel, the engineered-feature branch extracts twelve indicators associated with social-engineering language and message structure, including urgency, credential-related terms, threats, financial requests, calls to action, URL and contact patterns, punctuation, capitalisation and message length.

The outputs of these branches are projected into a shared representation space and combined through a learned gating mechanism. This mechanism controls the relative contribution of the contextual representation and the handcrafted features to the final prediction. The fused representation is then passed to the classification layer, which produces probabilities for the legitimate and phishing classes. The predicted class and its associated confidence are returned to the user interface. The confidence represents the model's estimated probability for the selected class and should not be interpreted as a guarantee that an email is safe or phishing.

Following classification, the system can generate a post-hoc explanation using either of the two implemented configurations. The SHAP configuration uses a FastAPI backend, a Streamlit interface and a Manifest V3 browser extension. It identifies feature contributions

that support or oppose the selected prediction and highlights influential token-level information. The LIME configuration uses a Flask REST API and a Chrome extension. It explains an individual prediction by examining how changes to the email text affect the classifier's output and identifying influential terms. Both configurations use the trained hybrid classifier for prediction; their primary difference lies in the method used to explain the results and the associated interface and backend implementation.

The system is organised into four main functional stages: data and model development, email processing and hybrid classification, post-hoc explainability, and user-facing presentation. The model-development stage covers dataset preparation, baseline development, transformer experiments, hybrid architecture selection and performance evaluation. The operational stages allow users to submit email content, obtain a classification result and request an explanation of the prediction. The two deployment configurations demonstrate how the same classification approach can be integrated into different interfaces and application workflows.

The scope of the prototype is limited to the textual content of English-language emails. It does not perform comprehensive email-security inspection, including sender authentication verification, attachment analysis, redirected-link inspection or live domain-reputation checking. The interfaces and APIs were deployed locally for development, testing and demonstration rather than as hardened production services. Accordingly, the system is intended to complement existing email security controls and support human decision-making through language-based detection and interpretable predictions, not to replace established security mechanisms or independent verification.

3. GITHUB LINK

The complete project source code, implementation files, configuration details and supporting technical documentation are available in the project's GitHub repository. The repository provides access to the code and resources required to review the system architecture, model implementation, explainability components and deployment configurations. The repository can be accessed at: [GitHub repository Or Drive Link](#)

4. DELIVERED TECHNICAL ARTEFACTS

Name	File	Description	PDF ?	GitHub Link
System Architecture and Design	System Design and Architecture.pdf	Documented the overall system architecture, processing pipeline and major system components.	Yes	GitHub link
Dataset and Preprocessing	Data_preprocessing.ipynb	Documented dataset preparation, data cleaning, label processing and text preprocessing.	Yes	GitHub link
TF-IDF Baseline Models	Logistic_Regrassion.ipynb	Implements and evaluates TF-IDF with Logistic Regression and Support Vector Machine (SVM) as baseline models.	No	GitHub link
Transformer Model Experiments	Folder: NLP Models	Implements and compares RoBERTa-large, ModernBERTlarge, DeBERTa-v3large, DistilBERT and TinyBERT.	No	GitHub link
Hybrid Model Architectures	Folder: NLP Models	Evaluates feature concatenation, gated fusion and attentionpooling approaches.	No	GitHub link
Proposed Hybrid Model	Gated_DeBERTa.ipynb	Implements the final DeBERTav3-large, gated hybrid classifier combining contextual representations with handcrafted features.	No	GitHub link
Model Evaluation and Comparison	Result Analysis Final Draft.docx	Presents model comparisons using accuracy, precision, recall, F1-score and confusion matrices.	Yes	GitHub link
SHAP Backend and API	api.py	Implements the FastAPI backend for classification and SHAP-based explanations.	No	GitHub link
SHAP Streamlit Application	app.py	Provides the user interface for email analysis, classification results, confidence and SHAP explanations.	No	GitHub link

SHAP Testing Module	test_shap.py	Tests and verifies SHAPbased explanations for individual predictions.	No	GitHub link
SHAP Browser Extension	content.js, manifest.json, popup.html, popup.js, popup.css	Implements the browser extension for email extraction, phishing analysis and display of results.	No	GitHub link
Streamlit Testing Evidence	Streamlit_Email_Testing.pdf	Documents testing through the Streamlit interface, including verdicts, confidence and explanations.	Yes	GitHub link
Browser Extension Testing Evidence	Browser_Extension_Email_Testing.pdf	Documents browser extension email scanning, backend communication and result presentation.	Yes	GitHub link
LIME Backend Documentation	Backend Technical Documentation.pdf	Describes the Flask backend, prediction pipeline, feature extraction, LIME explanations, testing and limitations.	Yes	GitHub link
LIME Flask REST API	App.py	Implements the Flask API, including emailanalysis	No	GitHub link
LIME Prediction Module	Predictor.py	Loads the hybrid model, processes email text, extracts handcrafted features and generates predictions and LIME explanations	No	GitHub link
Model Inspection Utilities	check_model.py/inspect_checkpoint.py/checkpoint_keys.txt	Supports checkpoint inspection, model compatibility checks and implementation verification.	No	GitHub link
Lime Chrome Extension	browser_extension/chrome-extensionlime	Connects the Chrome extension to the Flask backend and displays verdicts, confidence and influential terms.	No	GitHub link]
LIME API Testing Evidence	api-testing-evidence.pdf	Documents API startup, health checks, classification responses, LIME output and inputvalidation tests.	Yes	GitHub link
Browser Extension Demonstration	browser-extension-demonstration.pdf	Demonstrates testing with legitimate, conventional phishing and	Yes	GitHub link

		AI-rewritten phishing emails.		
Dependency Specification	requirements.txt	Lists Python dependencies required to run the relevant backend, prediction and explanation components.	No	GitHub link
Reproduction and Deployment Guide	README.md	Explains project setup, installation, backend execution, API testing, extension loading and reproduction procedures.	No	GitHub link

5. CONTRIBUTIONS

Student Name	Percent	Summary of Contributions	Technical Lead on Artefacts
Mohammad Riadur Rashid	32%	Led the core NLP model development, including dataset preprocessing, TF-IDF baseline experiments, transformer model comparisons, handcrafted feature extraction, hybrid architecture experiments and development of the final gated hybrid DeBERTav3-large classifier. Contributed to model evaluation, performance comparison, system architecture and technical documentation	Dataset and Preprocessing; TFIDF Baseline Models; Transformer Model Experiments; Handcrafted Feature Extraction; Hybrid Model Architectures; Proposed Hybrid Model; Model Evaluation and Comparison; System Architecture and Design
Prizma Shestha	32%	Led the implementation and testing of the SHAP-based explainability configuration. Developed the FastAPI backend, Streamlit application and Manifest V3 browser extension to provide phishing classification results, confidence scores and SHAPbased explanations. Prepared testing evidence for email analysis through the interface and browser extension.	TF-IDF Baseline, SHAP Backend and API; SHAP Streamlit Application; SHAP Testing Module; SHAP Browser Extension; Streamlit Email Testing Evidence; SHAP Browser Extension Testing Evidence
Sujina Mahato	31%	Led the implementation and testing of the LIME-based explainability configuration. Developed the Flask REST API, prediction and explanation module, and Chrome extension. Implemented input validation, email preprocessing, prediction responses and local LIME explanations. Prepared backend documentation and API and browser-extension testing evidence.	TF-IDF Baseline, LIME Backend Technical Documentation; LIME Flask REST API; LIME Prediction and Explanation Module; Model Inspection and Verification; LIME Chrome Extension; LIME API Testing Evidence; Browser Extension Demonstration
Md Sadman Sakib	05%	Conducted ModernBERT model experimentation for phishing email classification and investigated the implementation of Integrated Gradients as an explainability technique. Contributed to evaluating alternative transformer architectures and exploring tokenlevel attribution methods for interpreting model predictions.	ModernBERT Model Experimentation; Integrated Gradients Explainability Investigation

6. PROJECT OVERVIEW

6.1. Project Background

Email remains an essential communication tool for organisations, but it is also widely used to carry out phishing attacks. Attackers often disguise malicious messages as legitimate business requests, invoices, account notifications or internal communications. These emails can trick recipients into revealing sensitive information or taking actions that compromise security. Traditional email security controls help identify known threats, but carefully written phishing messages can be difficult to detect through technical indicators or simple keyword matching alone.

Natural Language Processing (NLP) offers another way to analyse email content by examining its language, context and patterns. Transformer models can capture contextual meaning, while handcrafted features can identify specific indicators associated with phishing behaviour. Combining these approaches may provide a more comprehensive basis for classification. Explainability methods can also help users understand why a message has been classified as suspicious. This project applies these techniques to develop a hybrid phishing email detection system with SHAP- and LIME-based explanations and user-facing prototypes.

6.2. Problem Statement

Phishing is not limited to a fixed set of words or patterns. Attackers continually change their wording, tone and message structure to make fraudulent emails appear legitimate. AI-generated phishing messages can be grammatically correct and professionally written, making them harder to distinguish from genuine communications. Legitimate emails can also contain urgent language, login links or requests for sensitive information. A reliable detection system must consider the context of a message rather than relying only on individual keywords or predefined indicators.

Another challenge is balancing false negatives and false positives. A false negative occurs when a phishing email is incorrectly classified as legitimate, potentially exposing users to account compromise, malware or financial loss. A false positive occurs when a legitimate email is incorrectly flagged as phishing, which can disrupt normal communication and reduce users' confidence in the system. For this reason, model performance must be assessed using multiple evaluation metrics, including precision, recall, F1-score and confusion matrix analysis, rather than accuracy alone.

Model interpretability presents a further challenge. Transformer-based models can produce accurate predictions without clearly showing users why an email has been flagged. An unexplained classification provides limited support for users who need to assess suspicious messages. This project addresses this issue through two post-hoc explainability configurations using SHAP and LIME. SHAP estimates the contribution of individual features or tokens to a prediction, whereas LIME identifies locally influential terms and their associated weights. Both methods help explain individual predictions, but their explanations do not establish causality or independently prove that an email is malicious

6.3. Project Aim and Objectives

6.3.1 Project Aim

The aim of this project is to develop and evaluate an explainable NLP-based phishing email detection system that identifies phishing and social-engineering intent from email text. The system combines a fine-tuned DeBERTa-v3-large transformer with twelve handcrafted phishing-related features through a gated fusion mechanism. It also integrates SHAP- and LIME-based explanations into two local prototype configurations to help users understand why an email is flagged as phishing or classified as legitimate. The system is intended to complement existing email security controls, particularly when dealing with well-crafted, text-only and AI-rewritten phishing messages.

6.3.2 Project Objectives

The project objectives are to:

1. Prepare and preprocess labelled phishing and legitimate email data.
2. Establish TF-IDF-based Logistic Regression and SVM baselines.
3. Compare transformer models for phishing email classification.
4. Develop a gated hybrid classifier combining DeBERTa-v3-large with twelve handcrafted features.
5. Evaluate model performance using accuracy, precision, recall, F1-score and confusion matrices.
6. Implement SHAP and LIME to explain individual classification predictions.
7. Develop two local prototypes integrating the classifier, explainability methods and user interfaces.
8. Test the system on legitimate, conventional phishing and AI-rewritten phishing emails where available.
9. Document system limitations, privacy considerations and future improvements.

6.4. Project Scope

This project focuses on classifying English-language email text as legitimate or phishing. It covers dataset preparation, baseline modelling, transformer fine-tuning, handcrafted feature extraction, hybrid classification, model evaluation and explainability. Two local prototype configurations were developed: one using FastAPI and Streamlit with SHAP, and another using Flask with LIME. Both configurations demonstrate email classification and provide explanations to help users interpret predictions.

The system does not inspect attachments, verify sender authentication, check live domain reputation or provide comprehensive email security. Multilingual detection and systematic defence against heavily obfuscated messages are also outside the implemented scope. The prototypes are intended for demonstration and evaluation, not production deployment. The system complements existing security controls and should not be used as the sole basis for deciding whether an email is safe.

6.5. Stakeholders and Requirements

The primary stakeholder is a university help desk that receives a high volume of emails and has experienced phishing attacks. The system is intended to support IT and security staff by providing an additional language-aware detection layer and clear explanations for suspicious messages. The sponsor may provide de-identified phishing examples and review whether the system's predictions and explanations are useful for security awareness.

The main requirements are to classify emails as phishing or legitimate, identify suspicious linguistic patterns, provide understandable explanations, and offer a practical interface for testing email content. The system should also distinguish legitimate transactional messages from phishing attempts as accurately as possible and complement existing email security controls.

6.6. Success Criteria and Acceptance Test

Success criterion	Acceptance test
Email classification	Submit legitimate and phishing emails and verify that the system returns a classification.
Detection performance	Evaluate the final model using accuracy, precision, recall, F1-score and a confusion matrix.
Explainability	Verify that SHAP and LIME provide explanations for individual predictions.
Prototype functionality	Test email submission and result presentation through both implemented prototype configurations.
Robustness assessment	Test selected conventional and AI-rewritten phishing examples where available and document the results.
Responsible use	Document system limitations, privacy considerations and the need for existing security controls.

7. BACKGROUND AND RELATED WORK

7.1. Phishing and Social Engineering

Phishing is a form of social engineering in which attackers manipulate people into taking unsafe actions. These attacks can be delivered through email, messaging platforms, telephone calls or fraudulent websites. Attackers often disguise their messages as legitimate communications and use authority, urgency or fear of influencing recipients. Common objectives include stealing login credentials, obtaining financial payments, distributing malware and collecting sensitive information. Targeted attacks may exploit trust in familiar business practices, such as requesting changes to supplier bank details or impersonating an executive to authorise a payment.

Phishing remains a significant cybersecurity concern in Australia. The Australian Signals Directorate (2025) identifies phishing, compromised accounts and the collection of victims' identity information among common cyber threats. Business email compromise is also associated with reports of cybercrime involving financial losses. Similarly, ENISA (2024) highlights phishing and email pretexting as important techniques in the contemporary threat landscape. These findings demonstrate the need for layered security controls that consider both technical indicators and the persuasive language used in suspicious messages. Text-based classification can contribute to this process, but it cannot provide complete protection on its own.

Generative artificial intelligence (AI) introduces additional challenges for phishing detection. AI tools can produce grammatically correct messages, adapt their tone to a target and remove spelling or grammatical errors traditionally associated with suspicious emails. As a result, users and detection systems cannot rely solely on poor writing as an indication of phishing. The underlying intention may instead be reflected in the request for credentials, an unexpected payment, secrecy or immediate account verification. This creates a need for contextual Natural Language Processing (NLP) techniques that examine the meaning of messages rather than relying only on surface-level patterns. Changes in attacker behaviour also make evaluation on unfamiliar and rewritten messages important.

7.2. NLP-Based Phishing Detection

NLP-based phishing detection analyses the language and structure of email messages to identify patterns associated with legitimate or malicious communication. Traditional approaches represent messages using word frequencies, n-grams and manually selected linguistic features. More recent approaches use neural language models to learn contextual representations from text. These techniques can identify suspicious linguistic patterns even when the system has no prior knowledge of a particular phishing message. However, text classification alone cannot verify whether a sender is authentic or establish whether a URL or domain is malicious.

Research has demonstrated the potential of deep learning and transformer-based methods for phishing email classification. (Atawneh and Aljehani 2023) reported strong performance from BERT-based email classification. The systematic review by (Kyaw, Gutierrez and Ghobakhlou

2024) also discusses the capabilities of deep learning approaches and the continuing challenge of adapting detection systems to changing phishing behaviours. In addition, (Alhuzali et al. 2025) reported strong results from BERT and RoBERTa in a multi-dataset comparison, while highlighting that model performance can vary across datasets. These findings suggest that high performance on one test dataset does not necessarily guarantee equivalent performance on messages from different sources or with different writing styles.

This project adopts a hybrid approach that combines contextual representations from DeBERTav3-large with twelve handcrafted phishing-related features. The transformer captures contextual information from the email text, while the handcrafted features represent explicit indicators associated with social engineering and writing style. The approach is based on the possibility that learned contextual representations and domain-specific indicators provide complementary information. Sender authentication and domain reputation could provide additional evidence in a more comprehensive system, but these signals are outside the scope of the current implementation.

7.3. Classical NLP Approaches

Classical NLP methods provide a useful foundation for text classification. One widely used technique is term frequency–inverse document frequency (TF-IDF), which represents each document as a numerical vector. It assigns weights according to how frequently a term occurs in a document and how informative that term is across the corpus. These vectors can be used by classifiers such as Logistic Regression and Support Vector Machines (SVMs) to distinguish between legitimate and phishing emails.

Classical approaches offer several practical advantages. TF-IDF combined with traditional classifiers such as Logistic Regression and Support Vector Machines can provide strong phishing detection performance with relatively low computational and inference costs compared with large transformer models (Meléndez, Ptaszynski and Masui 2024; Alajaji 2026). Their efficiency and simplicity make them useful benchmarks, particularly in resource-constrained environments. In this project, TF-IDF combined with Logistic Regression and SVM was used to establish baseline performance. These baselines provide a reference point for evaluating whether the additional computational cost of transformer-based models and hybrid architectures produces a meaningful improvement.

A key limitation of TF-IDF is its reliance on the statistical importance of individual words and ngrams, rather than a direct representation of contextual meaning. Although these features can be effective for text classification, they do not inherently capture the full context or intention of a message (Meléndez, Ptaszynski and Masui 2024; Kyaw, Gutierrez and Ghobakhlu 2022). For example, the phrases “verify your password here” and “we will never ask you to disclose your password” share vocabulary but communicate different intentions. A classifier relying heavily on word occurrence may struggle to distinguish such examples unless its learned features capture enough of the relevant differences. Transformer-based models offer an alternative by learning contextual representations of words and their relationships within a message.

Classical models can also be sensitive to differences between training and test data. Changes in vocabulary, message structure or phishing tactics may reduce their effectiveness when unfamiliar examples are encountered. N-gram features can capture some local word sequences, but they do not provide the same contextual representations as transformer models. Despite these limitations, classical approaches remain important because they offer efficient baselines and help establish whether a more complex model is justified.

7.4. Transformer-Based Approaches

Transformer models use self-attention to learn relationships between tokens and represent words according to their surrounding context. Bidirectional encoder models, such as BERT and its variants, can consider information from both directions when representing a token. This allows the same word to contribute differently to a prediction depending on the surrounding language. For phishing detection, contextual information can help distinguish between legitimate account notifications, security advice and deceptive requests for credentials or payments. Fine-tuning adapts a pretrained language model to a specific classification task by training it on labelled examples and a task-specific classification layer.

DeBERTa extends the transformer architecture through disentangled attention, which represents token content and relative positional information separately. DeBERTa-v3 introduces further changes to the pretraining approach, including replaced-token detection and gradient-disentangled embedding sharing, with the aim of improving the quality of learned representations (He, Gao and Chen 2023). The DeBERTa-v3-large variant provides a high-capacity contextual representation, although its size also increases computational and memory requirements.

Several transformer families were evaluated in this project, including RoBERTa-large, ModernBERT-large, DeBERTa-v3-large, DistilBERT and TinyBERT. The final hybrid architecture was based on DeBERTa-v3-large, which builds on disentangled attention and uses replaced-token detection during pretraining (He, Gao and Chen 2021). In the proposed design, the contextual representation and the twelve-dimensional handcrafted feature vector are projected into a shared 256-dimensional representation space. A sigmoid gate is calculated from the combined representations and controls their relative contributions to the fused representation. This follows the general principle of gated feature fusion, in which a learned gate regulates the combination of different information sources. A two-class classification layer then produces the final prediction. The gate allows the model to learn how much contextual and handcrafted information to use for each email (Kyaw and Gutierrez 2026).

Despite their capabilities, transformer models have limitations that must be considered. Maximum input lengths can restrict the amount of email text processed, and inference can be computationally expensive, particularly on a CPU. Prediction confidence may also be poorly calibrated on data that differs from the training distribution. Another concern is shortcut learning, where a model relies on dataset-specific patterns, duplicated templates or formatting rather than meaningful indicators of phishing intent. These issues make evaluation beyond an internal test split important. The lower performance observed on unfamiliar emails in this

project is consistent with the broader generalisation challenge discussed by (Kyaw, Gutierrez and Ghobakhlou 2024).

7.5. Explainable Phishing Detection

Explainability is important in phishing detection because security decisions can influence user behaviour and incident-response actions. A warning that labels an email as suspicious may not explain whether the prediction is associated with a credential request, urgency, financial language or another textual pattern. Explanations can help users interpret a result and support analysts when investigating false positives and false negatives. They can also assist error analysis by revealing which parts of a message appear influential to a particular prediction.

This project uses Shapley Additive explanations (SHAP) and Local Interpretable Model-Agnostic Explanations (LIME) as post-hoc explanation methods. SHAP assigns feature-attribution values to individual predictions, indicating how features contribute relative to a chosen baseline (Lundberg and Lee 2017). In the SHAP configuration, the classifier first produces the prediction and class probabilities. A separate explanation request then estimates token-level contributions, showing factors that support or oppose the explained class. The /predict and /explain endpoints are separated so that generating an explanation is not a prerequisite for obtaining a normal prediction. SHAP contribution values should not be interpreted as probabilities, universal indicators of token risk or proof that a particular word caused an email to be malicious. LIME provides a complementary approach by approximating a classifier's behaviour locally around an individual input (Ribeiro, Singh and Guestrin 2016).

LIME explains an individual prediction by generating perturbed versions of the input, observing the classifier's responses and fitting a simpler local surrogate model. In the project's LIME configuration, the resulting term weights indicate which terms support or oppose the selected class in the neighbourhood of the original email. A fixed random seed can improve repeatability, while limiting the number of perturbations can help control computational cost. This creates a trade-off: reducing the number of perturbations may affect the stability and fidelity of the local explanation. LIME does not reproduce the transformer's complete internal reasoning, and its term weights are not probability or causal effects. They should be interpreted as local approximations of model behaviour (Salih et al. 2025).

SHAP and LIME provide complementary perspectives on individual predictions, but neither guarantees that a model has learned valid security-related patterns. Both methods depend on the underlying classifier and can produce plausible explanations for predictions influenced by dataset artefacts or other unreliable correlations. (Al-Subaiey et al. 2024) discuss the use of interpretable approaches in phishing detection to improve the usability of classification results. In this project, classification outputs and explanation outputs are presented as separate types of information. The predicted class and its confidence describe the classifier's output, whereas the explanations indicate which textual features influenced the individual prediction. Highlighted words should therefore be treated as supporting information for review, not as definitive rules for identifying phishing emails.

7.6 Research Gap

Previous research has demonstrated strong performance in phishing email detection using traditional machine learning and transformer-based models. However, several gaps remain relevant to the development of reliable and explainable detection systems.

First, evaluation on randomly divided datasets from the same source may not adequately represent changes in phishing campaigns, organisational communication styles and AI-generated messages. High performance on a benchmark dataset does not necessarily demonstrate that a model can generalise to unfamiliar emails. This highlights the need for evaluation beyond conventional train-test splits.

Second, traditional NLP approaches rely heavily on word frequencies and statistical patterns, which may not adequately capture the contextual meaning of an email. Transformer models can represent contextual relationships more effectively, but the potential benefit of combining these representations with explicit, security-relevant linguistic features requires further empirical evaluation. The effectiveness of such hybrid approaches may also depend on the selected transformer architecture and fusion mechanism.

Third, high classification performance does not necessarily make a model's decisions understandable to users. Although explainability methods such as SHAP and LIME can provide insights into individual predictions, explanations must be interpreted carefully because they do not establish malicious intent or guarantee that the model relies on reliable security indicators. Integrating these explanations into a usable detection workflow is also important for demonstrating their practical value.

Finally, model development alone does not demonstrate how phishing detection and explainability can work together in a user-facing application. A complete prototype should connect email input, classification, prediction confidence and explanations in a coherent workflow. The practical value of such a system depends not only on model performance but also on how clearly its output can be interpreted.

This project responds to these gaps by comparing classical NLP baselines and transformer models, developing a gated hybrid DeBERTa-v3-large classifier with twelve handcrafted phishing-related features, and integrating SHAP and LIME into two local prototype configurations. Evaluation on an independent set of unseen emails also provides evidence of the challenges associated with generalisation beyond the main test dataset. The project therefore contributes a comparative evaluation and an implemented prototype that brings together hybrid classification, post-hoc explainability and user-facing demonstrations.

8. PROJECT METHODOLOGY

8.1. Overall Methodology

The project follows a structured methodology to develop a phishing email detection system that can identify suspicious emails and explain its predictions. The workflow consists of dataset preparation, preprocessing, baseline modelling, transformer-based modelling, handcrafted feature extraction, hybrid model development, evaluation, explainability, and system demonstration.

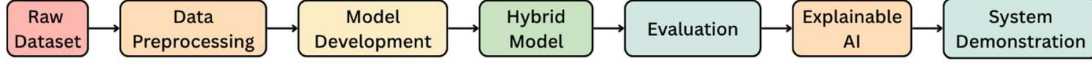


Figure 8.1.1: The Overall Methodology

The methodology is divided into two main parts:

- NLP-based Classification
- Explainable AI (XAI)

The NLP-based classification part focuses on identification of an email is phishing or legitimate. The classical NLP models establish an initial performance level using traditional text classification. Transformer models are then evaluated to capture deeper contextual patterns in email content. Handcrafted phishing-related features are added to represent specific characteristics commonly associated with phishing emails. These features are combined with transformer representations to develop and evaluate hybrid architectures. This process allows different modelling approaches to be compared before selecting the final classification model.

The XAI part focuses on making the model's predictions understandable. A high classification score alone does not show why an email was classified as phishing. Explanations can identify the words or phrases that influenced the prediction and help users understand the model's decision. This is important for a security application because users need meaningful information to decide whether the email is phishing or legitimate.

The two parts serve different purposes. NLP provides classification capability, and XAI provides interpretability. Combining them produces a system that can provide both a classification result and an explanation. The final stage demonstrates this complete process using email inputs and the resulting prediction and explanation.

8.2. Process of Experiments: NLP-based Classification

The experimental process was designed as a sequence of stages, beginning with the preparation of the phishing email dataset and ending with the comparison of model performance.

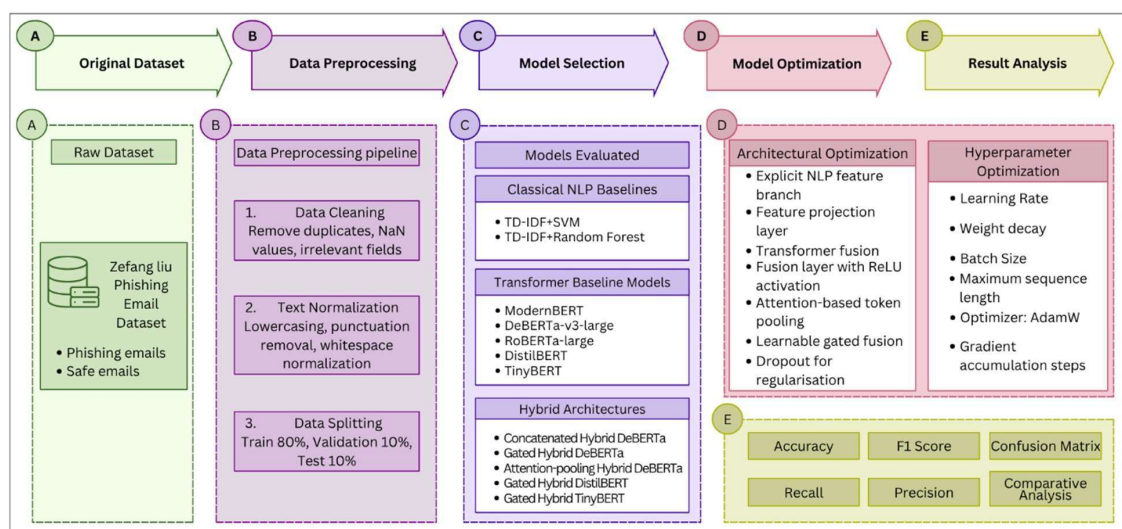


Figure 8.2.1: An approach to email classification is presented as follows: Frame (A) shows that the dataset is collected. (B) Data preprocessing steps are applied to generate the pre-processed dataset. (C) Several baseline models are evaluated. (D) Hybrid architectures are developed through architectural and hyperparameter experimentation. (E) The model results are analysed using performance metrics.

The model development followed a progressive experimental process. The initial stage established classical NLP baselines using TF-IDF with Logistic Regression and Support Vector Machine. These models provided reference performance for the subsequent experiments.

The next stage involved training several pretrained transformer models as standalone classifiers. These included RoBERTa-large, ModernBERT-large, DeBERTa-v3-large, DistilBERT and TinyBERT. The standalone transformer experiments established reference performance before introducing additional handcrafted features.

The following stage focused on hybrid model development. The hybrid approach combined contextual information from a pretrained transformer with explicitly added phishing-related NLP features. Different feature-fusion strategies were investigated to examine how the two sources of information could be integrated. These experiments included feature concatenation, gated fusion and attention pooling.

Hybrid experiments were conducted using different transformer backbones, including DeBERTav3-large, DistilBERT and TinyBERT. This allowed the study to examine whether hybridisation produced consistent performance across transformer architectures and model sizes.

The final stage involved comparative evaluation of the experimental models. The results were analysed using accuracy, precision, recall, F1-score and confusion matrices. The analysis

considered the effects of transformer architecture, handcrafted features and fusion strategy on email classification performance. The selected model was then used as the basis for the explainability component and system demonstration.

8.2.1 Dataset Description

The dataset used in this project is the zefang-liu/phishing-email-dataset (2026). It contains email messages labelled as either Safe Email or Phishing Email. The dataset provides labelled textual data suitable for supervised phishing email classification.

The original dataset contains 18,650 records. After the cleaning and preprocessing procedure, 17,519 valid records were retained. The final dataset consists of 10,978 safe emails and 6,541 phishing emails.

Name	Number of records
Columns	3
Safe Email	11322
Phishing Email	7328
Missing Values	Email text (16)
Empty email texts	19
Duplicate email text	1112
Different label	1

The processed dataset was divided into three fixed subsets using a stratified split with random seed 42. The training set contains 14,015 records, while the validation and test sets each contain 1,752 records. The test set was kept separate from model development and was used only for the final performance comparison.

8.2.2 Data Preprocessing

Data preprocessing was performed to improve the consistency of the email text while preserving information that could be useful for phishing detection. The same preprocessing procedure was applied before the data was used for model training and evaluation.

First, removed the non-required column named ‘unnamed’. Then missing or empty email texts were removed. Records containing different labels were also removed, followed by the removal of exact duplicate emails. The remaining text was processed using HTML entity decoding and Unicode NFKC normalisation.

Control characters were removed, while HTML/XML tags were removed without discarding the visible text content. Line breaks and excessive whitespace were also normalised. The original textual information was otherwise retained.

No stop-word removal, stemming, lemmatisation, or aggressive removal of punctuation and URLs were performed. These elements can contain useful signals for phishing detection. Finally, the class labels were converted into binary values, with Safe Email = 0 and Phishing Email = 1.

Table 8.2.2.1: Dataset description after preprocessing

Name	Number of records
Columns	4
Safe Email	10978
Phishing Email	6541
Missing Values	Email text (0)
Empty email texts	0
Duplicate email text	0
Different label	0
Label encoding (Safe email)	'0'
Label encoding (Phishing email)	'1'

8.2.3 Dataset Split

After preprocessing, the dataset was divided into training, validation, and test subsets. A stratified split with a fixed random seed of 42 was used to maintain a consistent class distribution across the three subsets.

Set name	Percentage	Number of data
Train	80	14015
Validation	10	1752
Test	10	1752

The training set contains 14,015 emails and was used for model training. The validation set contains 1,752 emails and was used to monitor model performance during training and support model selection. The remaining 1,752 emails were reserved as the test set.

The test data was not used for training or model selection. Keeping this set separate allowed the final models to be evaluated under the same unseen test conditions.

8.2.4 Baseline Models

Baseline models were established before developing the hybrid architectures to provide reference points for evaluating the effectiveness of the proposed approaches. Both classical NLP and pretrained transformer models were considered to provide a broader comparison across different modelling approaches.

A. Classical NLP Baseline Models

Classical NLP baselines were included to establish a simple and interpretable benchmark for the phishing email classification task. This provides a reference for comparing traditional text-based machine learning approaches with the more advanced transformer and hybrid models.

Two traditional machine learning approaches were evaluated using TF-IDF (Term FrequencyInverse Document Frequency) for text representation:

TF-IDF + Support Vector Machine and Random Forest

TF-IDF + SVM and Random Forest were included as classical NLP baseline for the phishing email classification task. In both approaches, the preprocessed email text is converted into numerical feature vectors using Term Frequency-Inverse Document Frequency (TF-IDF), which represents the importance of terms within individual emails and across the overall collection. The resulting

TF-IDF representations are then provided to either a Support Vector Machine (SVM) or a Random Forest (RF) classifier to classify between safe and phishing emails. SVM provides a traditional classification approach, while Random Forest combines multiple decision trees to produce the final classification. These models provide a traditional NLP benchmark for comparison with the transformer-based models (Dadgar, Araghi and Farahani 2016; Sjarif et al. 2019).



Figure 8.2.2 Process of Classical NLP Baseline Models

B. Transformer Baseline Models

Each baseline model received the processed email text as input and performed binary classification between Safe Email and Phishing Email. The models represent different transformer architectures and model sizes, allowing the study to compare both larger and more lightweight pretrained approaches.

RoBERTa-large

RoBERTa-large is a transformer-based language model designed to improve the pretraining approach used by BERT (Liu et al. 2019). It consists of 24 transformer encoder layers, with a hidden size of 1024 and 16 attention heads. The model uses self-attention to capture contextual relationships between words and produces contextual representations that can be adapted for downstream text classification tasks.

ModernBERT-large

ModernBERT-large is a modernised bidirectional, encoder-only Transformer based on the BERT architecture. It was pretrained on approximately 2 trillion tokens of English text and code and supports a native context length of up to 8,192 tokens. The architecture introduces improvements such as Rotary Positional Embeddings (RoPE), local-global alternating attention, unpadding, and Flash Attention, which improve long-context processing and computational efficiency. The large version contains 28 Transformer layers and approximately 395 million parameters. In this research, ModernBERT-large was evaluated as a pretrained transformer baseline for phishing email classification (Answer.AI 2024).

DistilBERT

DistilBERT is a smaller transformer model developed through knowledge distillation from BERT. It retains the main contextual language-understanding capabilities of BERT while using a reduced architecture. The commonly used DistilBERT-base model contains 6 transformer layers, 768 hidden dimensions, and 12 attention heads, making it considerably lighter than larger transformer models (Sanh et al. 2019).

TinyBERT

TinyBERT is a compact transformer model created using knowledge distillation to transfer knowledge from BERT into a smaller architecture. The TinyBERT model used in this research contains 4 transformer layers, a 312-dimensional hidden representation, and 12 attention heads. Its smaller architecture provides reduced computational requirements while retaining the ability to perform natural language understanding tasks (Jiao et al. 2020).

DeBERTa-v3-large

DeBERTa-v3-large is a pretrained Transformer encoder designed to improve contextual language understanding. The DeBERTa architecture introduces disentangled attention, where word content and position information are represented separately when calculating attention between tokens. This allows the model to capture both the meaning of words and their relative positions more effectively.

The V3 version further improves the original DeBERTa through ELECTRA-style Replaced Token Detection (RTD) pretraining and Gradient-Disentangled Embedding Sharing. These changes improve the efficiency of pretraining and the quality of the resulting language representations (He, Gao and Chen 2021).

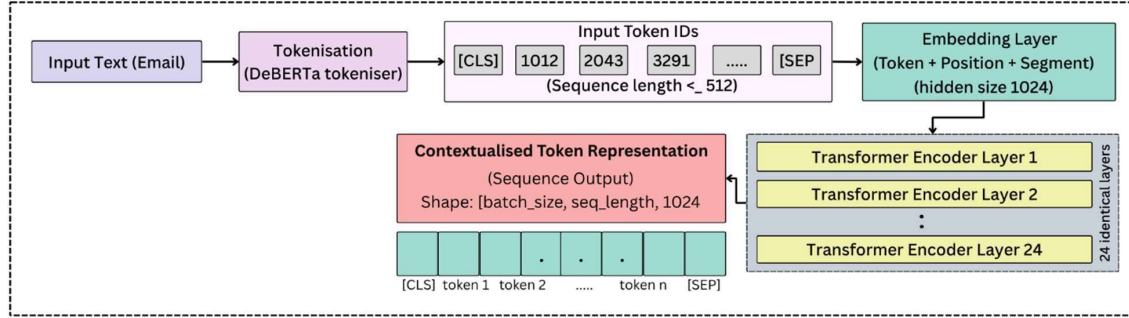


Figure 8.2.3: Architecture of DeBERTa-v3-large

The DeBERTa-v3-large model used in this research contains 24 Transformer layers, a 1024dimensional hidden representation, 16 attention heads, and a 128K-token vocabulary. The official model documentation reports 304M backbone parameters, with an additional 131M parameters in the embedding layer.

DeBERTa-v3-large processes the tokenised email through its Transformer encoder layers and produces a contextual representation for each token. For an input sequence of n tokens, the final encoder produces:

$$H = [h, h, \dots, h], \quad h \in R$$

In this project, DeBERTa-v3-large was used as the main pretrained backbone for three hybrid architectures. It receives the processed email text and produces contextual representations, which are then combined with the 12 handcrafted phishing-related NLP features in the proposed hybrid models.

8.2.5 Hybrid Architectures

The hybrid architecture extends pretrained transformer models by introducing a second branch containing handcrafted phishing-related NLP features. The two branches provide different types of information. The transformer branch captures contextual relationships within the email text, while the feature branch provides explicit indicators associated with phishing behavior.

Three hybrid architectures were developed using DeBERTa-v3-large.

Hybrid Architecture 1: Feature Concatenation

The first hybrid architecture combines the contextual representation generated by the pretrained DeBERTa-v3-large model with 12 handcrafted NLP features extracted from the email text. The two information sources are processed through separate branches before being combined.

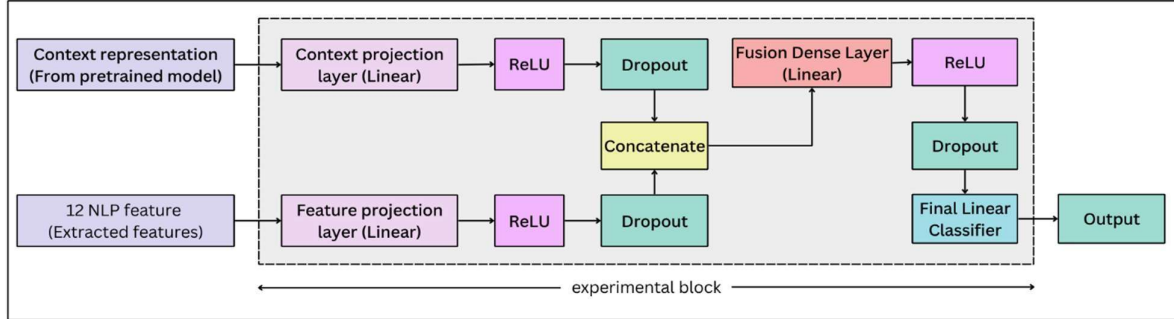


Figure 8.2.4: Architecture of Feature Concatenation Model

In the first architecture, the first-token representation from DeBERTa (equation i) is used as the overall contextual representation:

$$h = H[:, 0, :]$$

This 1024-dimensional representation is then passed through a linear projection and activation to obtain a compact representation for the hybrid model. In parallel, the 12 handcrafted NLP features are projected into the same feature space. Dropout is applied to both branches to reduce overfitting.

The two resulting representations are then concatenated to preserve information from both the contextual and handcrafted feature branches:

$$h = h ; h$$

The concatenated representation is passed through a fusion dense layer with ReLU activation and dropout. Finally, a linear classification layer produces two output values corresponding to the Safe Email and Phishing Email classes.

Hybrid Architecture 2: Gated Fusion

The second hybrid architecture extends the feature concatenation approach by introducing a gated fusion layer between the two feature branches and the fusion dense layer. The architecture combines the contextual representation from the pretrained DeBERTa-v3-large model with the 12 handcrafted NLP features.

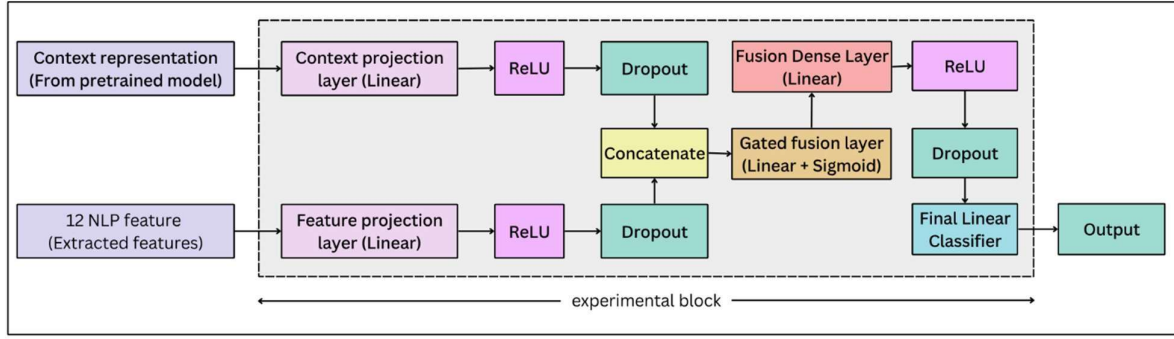


Figure 8.2.5: Architecture of Gated Fusion Model

The contextual representation and the 12 NLP features are first passed through separate linear projection layers, followed by ReLU activation and dropout. Both branches are projected into a 256-dimensional representation before fusion.

The two projected representations are then concatenated and provided to the gated fusion layer, which uses a linear transformation and sigmoid activation to learn a gate for combining the two information sources:

$$g = \sigma W h ; h + b$$

The gated representation is then passed to the fusion dense layer, followed by ReLU activation and dropout. Finally, the linear classifier produces the two classification outputs: Safe Email and Phishing Email.

Hybrid Architecture 3: Attention Pooling

The third hybrid architecture extends Draft 2 by introducing an attention pooling layer to obtain the contextual representation from the pretrained DeBERTa-v3-large model. The architecture combines the contextual information with the 12 handcrafted NLP features through separate processing branches.

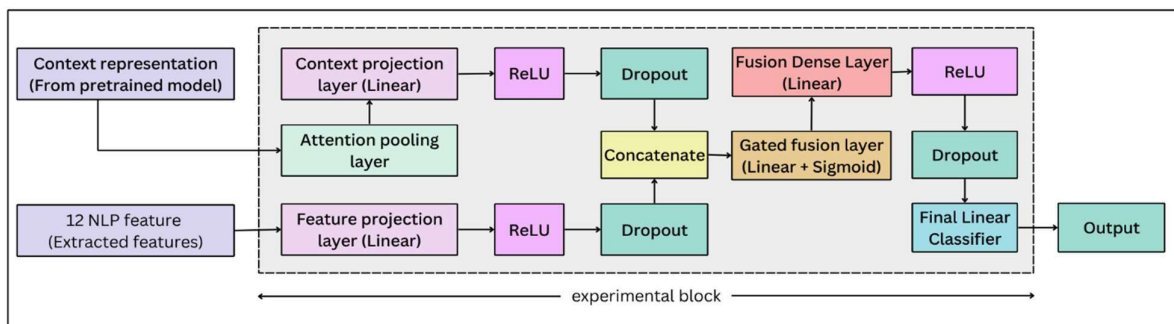


Figure 8.2.6: Architecture of Attention Pooling model

Instead of directly using the first-token representation, the token-level representations generated by DeBERTa are passed through the attention pooling layer to obtain a weighted contextual representation. This representation is then passed through a linear projection layer, followed by ReLU activation and dropout. In parallel, the 12 handcrafted NLP features are passed through a feature projection layer, ReLU activation, and dropout.

In this architecture, for the token representation h_i , attention score is first calculated, and the scores are converted into normalised attention weights using a SoftMax function.

$$s = w h + b$$

$$a = \text{softmax}(s)$$

The contextual representation is then obtained as the weighted sum of the token representations:

$$h = \sum a_i h_i$$

This allows the model to assign greater importance to tokens that contribute more strongly to the overall email representation. The resulting contextual representation is then passed through the context projection layer, ReLU activation, and dropout. In parallel, the 12 handcrafted NLP features are processed through the feature projection layer, ReLU activation, and dropout. The two processed representations are concatenated and passed through the gated fusion layer. The resulting fused representation is then processed by the fusion dense layer, ReLU activation, and dropout before the final linear classifier produces the Safe Email and Phishing Email predictions.

8.2.6 Model Optimization

Model optimization was conducted to investigate how different architectural components and training configurations affected email classification performance. The optimization process consisted of architectural experimentation and hyperparameter experimentation.

A. Architectural Optimization

The hybrid architectures were developed progressively by introducing additional components to the pretrained transformer models. The experiments investigated the use of the 12 handcrafted NLP feature branch, feature projection, feature concatenation, gated fusion, attention-based token pooling, fusion layers, and dropout.

B. Hyperparameter optimization

For the hybrid architectures, the main training parameters considered were learning rate, batch size, gradient accumulation, sequence length, and numerical precision. The models used a

maximum sequence length of 512 tokens and a learning rate of 1×10^{-5} . FP32 precision was used for stable training.

For the larger DeBERTa-based architectures, batch size and gradient accumulation were adjusted according to GPU memory requirements. Architecture 2 used batch size 2 with gradient accumulation of 8 steps, together with gradient checkpointing to address memory limitations. The same training configuration was maintained for the subsequent DeBERTa hybrid experiments to support a consistent comparison.

8.2.7 Proposed Gated Hybrid DeBERTa Model

The proposed architecture combines the contextual representation from DeBERTa-v3-large with the 12 handcrafted phishing-related NLP features. The two information sources are processed through separate branches before they are fused.

The DeBERTa-v3-large branch receives the preprocessed email text and produces a contextual representation from the Transformer encoder. This representation is passed through a linear projection layer, ReLU activation and dropout. In parallel, the 12 handcrafted features are passed through a separate feature projection layer, followed by ReLU activation and dropout. Both branches are projected into a 256-dimensional feature space before fusion.

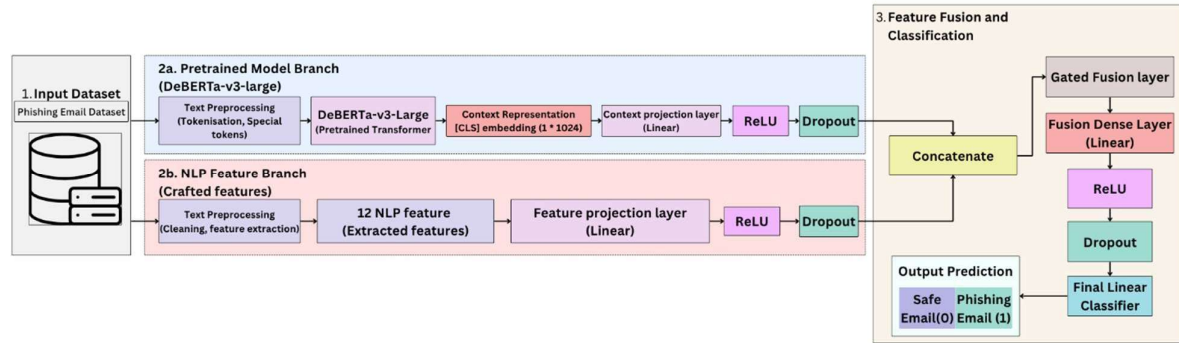


Figure 8.2.7: Architecture of Gated Hybrid DeBERTa Model

The two projected representations are then concatenated and provided to the gated fusion layer. The gate uses a linear transformation with sigmoid activation to learn the contribution of the contextual and handcrafted feature representations. The resulting fused representation is passed through a fusion dense layer with ReLU activation and dropout. A final linear classifier produces the two output classes: Safe Email and Phishing Email.

This architecture allows the model to use both contextual information learned from the email text and explicit phishing-related indicators within a single classification framework.

8.2.8 Model Training and Evaluation

The models were trained using the training dataset, while the validation set was used to monitor performance during model development. Training configurations were selected according to the requirements of the individual transformer architecture. The experiments used different sequence lengths, batch sizes, learning rates, and training configurations depending on the model. FP32 precision was used across the experiments to maintain stable model training.

For model evaluation, the same held-out test set was used for all baseline and hybrid architectural models. This provided a consistent basis for comparison between the different approaches. Evaluation Metrics

To evaluate the performance of the proposed model, many metrics have been assessed. A true positive (TP) is when the model accurately detects a positive instance. A true negative (TN) occurs when the model accurately detects and classifies a situation as negative. False positives (FP) and false negatives (FN) occur when the model makes erroneous predictions of positive and negative instances, respectively. Accuracy refers to the proportion of accurate predictions produced by the model, expressed as a percentage.

Accuracy

Measures the overall correctness of the model. It is the ratio of correctly predicted observations (both true positives and true negatives) to the total observations (Sokolova and Lapalme 2009).

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Measures the accuracy of positive predictions. It is the ratio of correctly predicted positive observations to the total predicted positive observations (Sokolova and Lapalme 2009).

$$precision = \frac{TP}{TP + FP}$$

Recall

Measures the fraction of actual positives that are correctly identified. It is the ratio of correctly predicted positive observations to all observations in the actual class (Sokolova and Lapalme 2009).

$$recall = \frac{TP}{TP + FN}$$

F1 Score

The harmonic mean of precision and recall. It provides a single metric that balances both precision and recall (Sokolova and Lapalme 2009).

$$precision \times recall$$

$$f1\ score = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

9. RESULTS, ANALYSIS AND COMPARISON

9.1. Overview of Experimental Evaluation

The experimental evaluation was conducted to investigate how different text representations, transformer architectures, model capacities and feature-fusion strategies affect phishing email classification performance. Rather than evaluating the models solely according to their final accuracy, the experiments were designed to examine the contribution of contextual language representations, explicitly engineered phishing-related features and different hybridisation strategies.

The experiments progressed from classical TF-IDF-based classifiers to pretrained transformer models and subsequently to hybrid architectures. The hybrid models combined contextual representations generated by a pretrained transformer with twelve handcrafted phishing-related NLP features. Additional experiments using smaller transformer backbones were conducted to determine whether the effectiveness of the hybrid approach was consistent across different model capacities.

All models were evaluated using the same held-out test set of 1,752 emails, consisting of 1,098 Safe Emails and 654 Phishing Emails. Overall Accuracy, Precision, Recall and F1-score were used for the principal model comparison. Phishing-class metrics and confusion matrices were additionally examined to provide a more detailed assessment of phishing detection and classification errors.

The analysis therefore focuses not only on which model obtained the highest score, but also on how and why performance changed when different representations and architectural configurations were introduced.

9.2. Overall Performance Comparison

Table 9.2.1 presents the overall performance of the evaluated models. Overall Precision, Recall and F1-score are reported using weighted averaging across the two classes.

Table 9.2.1: Overall performance of the evaluated models

Model	Accuracy	Precision	Recall	F1-score
TF-IDF + Logistic Regression	97.98%	98.44%	96.13%	97.27%
TF-IDF + Support Vector Machine	98.72%	98.62%	97.94%	98.28%
RoBERTa-large	99.26%	99.26%	99.26%	99.26%
DeBERTa-v3-large	99.37%	99.37%	99.37%	99.37%

ModernBERT-large	99.37%	99.37%	99.37%	99.37%
DistilBERT	99.20%	99.20%	99.20%	99.20%
TinyBERT	97.09%	97.09%	97.09%	97.08%
Concatenated Hybrid DeBERTa Model	99.37%	99.53%	98.77%	99.15%
Gated Hybrid DeBERTa Model	99.49%	99.49%	99.49%	99.49%
Attention Pooling Hybrid DeBERTa Model	99.32%	99.31%	99.32%	99.31%
Gated Hybrid DistilBERT Model	98.97%	98.98%	98.97%	98.97%
Gated Hybrid TinyBERT Model	96.00%	96.14%	96.00%	96.03%

The results demonstrate that the transformer-based approaches achieved very high classification performance on the test dataset. Among the standalone transformer models, DeBERTa-v3-large and ModernBERT-large achieved the highest overall accuracy at 99.37%, followed by RoBERTa-large at 99.26% and DistilBERT at 99.20%. TinyBERT produced a lower overall accuracy of 97.09%.

The highest overall performance was obtained by the gated hybrid DeBERTa model, which achieved 99.49% accuracy, precision, recall and F1-score. Compared with the standalone DeBERTa-v3-large model, this represents an improvement of:

$$99.49 - 99.37 = 0.12$$

or 0.12 percentage points in overall accuracy.

Although the numerical improvement is relatively small, this should be interpreted in the context of an already high-performing baseline. At an accuracy above 99%, only a small number of classification errors remain, meaning that further improvements are necessarily constrained by the limited number of incorrectly classified samples.

9.3. Text Representation and Model Capacity

The first experimental scenario examined the effect of using different types of text representations and transformer architectures.

The classical TF-IDF approach represents an email according to the statistical importance of its terms. While this representation is effective for identifying characteristic words, it does not explicitly model the contextual relationship between words.

Transformer architectures use self-attention to generate contextual representations, allowing information from different parts of an email to influence the representation of individual tokens. This is particularly relevant to phishing detection because words such as account, verify, payment or security may occur in both legitimate and phishing messages. Their significance can depend on the surrounding context and the overall structure of the message.

The strong performance of the transformer baselines is consistent with the ability of contextual representations to capture such relationships. DeBERTa-v3-large and ModernBERT-large achieved 99.37%, while RoBERTa-large and DistilBERT also achieved above 99%.

The lower performance of TinyBERT, at 97.09%, provides a contrasting result. TinyBERT has substantially lower model capacity than the larger transformer architecture evaluated in this project. Its lower performance suggests that the reduced representation capacity may limit its ability to capture some of the contextual patterns present in the dataset.

The model size should not be considered the sole explanation for this difference. The evaluated transformer models also differ in architecture, pretraining strategy and representation mechanisms.

Therefore, the results demonstrate a performance difference between the evaluated architectures rather than establishing a direct causal relationship between parameter count and accuracy.

An important observation is that the differences among the larger transformer models were relatively small. This indicates that once a sufficiently strong contextual representation was available, further differences in transformer architecture produced only marginal changes in overall classification performance.

9.4. Lightweight Transformer

The lightweight transformer experiments provide further insight into the relationship between model capacity and hybridisation. The results show that adding handcrafted phishing-related features did not consistently improve the performance of smaller transformer models.

DistilBERT achieved an overall accuracy of 99.20% when used as a standalone model, compared with 98.97% for its corresponding hybrid model. Similarly, TinyBERT achieved 97.09% accuracy as a standalone model, while its hybrid version achieved 96.00%. The performance reduction was therefore more noticeable for TinyBERT.

Table:9.4.1: Lightweight transformer comparison

Base Model	Standalone Accuracy	Gated Hybrid Accuracy
DistilBERT	99.20%	98.97%
TinyBERT	97.09%	96.00%

These results suggest that the additional feature branch did not automatically compensate for the lower representational capacity of the lightweight transformers. Instead, the effectiveness of hybridisation appears to depend on how well the handcrafted features complement the contextual representations produced by the underlying transformer. In the experiments, the hybrid strategy produced its strongest performance when combined with the larger DeBERTa-v3-large backbone.

This finding is important because it shows that hybridisation should not be assumed to be superior to a standalone transformer in every situation. The results indicate that the benefit of combining contextual and handcrafted features depends on the interaction between the quality of the underlying language representation, model capacity and additional feature information.

9.5. Handcrafted Phishing-Related Features

The next experimental scenario examined whether explicitly engineered phishing-related features could complement the contextual representations learned by the transformer. Twelve handcrafted features were designed to capture common characteristics found in phishing emails. These included urgency, credential requests, threats and authority, financial and payment-related language, calls to action, URLs, email addresses, telephone numbers, exclamation and question marks, uppercase-word usage, and text length.

The proposed hybrid architecture combines two sources of information. The transformer branch learns contextual patterns and relationships from the email text. The handcrafted feature branch provides explicit signals linked to common phishing characteristics. This combination allows the model to consider both the broader linguistic context and specific phishing-related indicators.

The comparison between DeBERTa-v3-large and the proposed Experiment 2 model provides clear evidence of the contribution of the additional feature branch. DeBERTa-v3-large achieved 99.37% overall accuracy, while the proposed hybrid model achieved 99.49%. The 0.12 percentage-point improvement shows that the handcrafted features provided additional useful information under the tested configuration.

The improvement is relatively small, which suggests that the transformer has already captured much of the information contained in the email text. The handcrafted features still contributed measurable information when integrated with the larger transformer representation. This supports the use of feature fusion as a complementary approach rather than treating handcrafted features as a replacement for contextual language modelling.

9.6. The Feature-Fusion Strategy

The hybrid experiments examined whether the method used to combine contextual and handcrafted features affected model performance. The experiments compared different fusion strategies while keeping the underlying DeBERTa-v3-large architecture consistent.

The initial hybrid architecture combined the two representations through feature concatenation. Experiment 2 extended this approach by introducing a learnable gated fusion mechanism. Experiment 3 added an attention-pooling step to refine the contextual representation before applying the hybrid fusion process.

In experiment 2, the contextual representation and handcrafted feature representation were first projected into a common 256-dimensional space. A sigmoid gate was then used to learn the relative contribution of each representation:

$$g = \sigma(w^T h; h + b)$$

The final fused representation was calculated as:

$$h = g \odot h + (1 - g) \odot h$$

This mechanism allows the model to learn how much contextual and handcrafted information should contribute to the final representation. It provides a more flexible combination than directly concatenating the two feature sets.

The results show a difference between the two fusion configurations. Experiment 2 achieved 99.49% overall accuracy, while Experiment 3 achieved 99.32%. This represents a difference of 0.17 percentage points in favour of experiment 2.

The result indicates that the choice of fusion strategies affected model performance under the evaluated conditions. The attention-pooling mechanism used in Experiment 3 did not improve performance compared with the gated fusion configuration.

The lower performance of experiment 3 may be associated with the additional attention-pooling step. The model first processed the transformer representation through attention pooling and then combined it with the handcrafted features. This extra processing did not produce a performance improvement in the evaluated experiment. Experiment 2 achieved higher accuracy with the gated fusion approach, suggesting that the simpler fusion process was more effective for this model configuration.

The experimentally supported finding is that the gated fusion configuration achieved stronger test performance than the attention-pooling configuration evaluated in this study.

9.7. Effect of Architectural Complexity

The effect of architectural changes within the hybrid modelling framework can be assessed by comparing the three hybrid experiments. Experiment 1 used direct feature concatenation and achieved an accuracy of 99.37% with an F1-score of 99.15%. Experiment 2 introduced gated

fusion and increased both accuracy and F1-score to 99.49%. Experiment 3 added attention pooling to the transformer representation and achieved 99.32% accuracy and 99.31% F1-score.

The progression from Experiment 1 to Experiment 2 shows a measurable improvement following the introduction of gated fusion. Feature concatenation directly combines the transformer representation with the 12 handcrafted NLP features. Gated fusion extends this approach by introducing a learnable mechanism that regulates the contribution of the two feature sources before classification. The increase from 99.37% to 99.49% accuracy and from 99.15% to 99.49% F1score indicates improved performance under the evaluated configuration.

Experiment 3 introduced an additional attention-pooling stage to refine the contextual representation before feature fusion. Its accuracy and F1-score were lower than those of Experiment 2, despite the additional architectural component. The results show that the increased processing of the transformer representation did not produce a corresponding improvement in classification performance.

Table 9.7.1: Comparison of Architectural Complexity and Performance

Experiment	Architecture	Architectural Complexity	Accuracy	F1-score	Description
Experiment 1	Feature Concatenation	Moderate	99.37%	99.15%	Directly combining transformer and handcrafted features
Experiment 2	Gated Fusion	Moderate	99.49	99.49	Combines transformer and handcrafted features through a learnable gating mechanism
Experiment 3	Attentionpooling	High	99.32	99.31	Applies attention pooling to transformer representations before feature fusion

The comparison across the three experiments demonstrates that architectural complexity alone did not lead to higher performance. The results increased from feature concatenation to gated fusion but decreased when attention pooling was added. This highlights the importance of how transformers derived and handcrafted features are integrated rather than simply increasing the number of processing components.

9.8. Effect of Hybridisation Across Transformer Backbones

A further experiment evaluated whether the hybrid approach produced consistent benefits across different transformer backbones. Table 9.8.1 compares the standalone and hybrid accuracy of DeBERTa-v3-large, DistilBERT, and TinyBERT.

Table 9.8.1: Compares Standalone accuracy and Hybrid accuracy

Backbone	Hybrid Architecture	Standalone Accuracy	Hybrid Accuracy	Difference
DeBERTa-v3-large	Gated Fusion	99.37%	99.49%	+0.12%
DistilBERT	Gated Fusion	99.20%	98.97%	-0.23%
TinyBERT	Gated Fusion	97.09%	96.00%	-1.06%

The results show that hybridisation did not provide a uniform improvement across the three backbones. DeBERTa-v3-large increased from 99.37% to 99.49%, giving a 0.12 percent improvement. On the other hand, the hybrid versions of DistilBERT and TinyBERT dropped by 0.23 and 1.09 percent, respectively.

The findings indicate that the effectiveness of feature fusion is dependent on the underlying transformer backbone. DeBERTa-v3-large benefited from the additional handcrafted features, but the smaller backbones did not show the same improvement. The reduction in performance for DistilBERT and TinyBERT also indicates that adding a handcrafted feature branch does not inherently improve classification when the underlying representation is changed.

The observed performance demonstrates that hybridisation is backbone-dependent, with the strongest improvement occurring when the handcrafted features were combined with DeBERTa-v3-large.

9.9. Training and Validation Loss of Gated Hybrid DeBERTa Model

Training and validation loss provide an additional perspective on how the proposed model (experiment 2) learned from the data and how well its learned representations generalised to unseen validation samples. Training loss measures the difference between the model's predictions and the target labels on the training data, whereas validation loss measures the same objective on validation samples that were not used to update the model parameters.

Across the three training epochs, the loss values show a clear reduction in training errors. At Epoch 1, the training loss was 0.069629, while the validation loss was 0.057938. At Epoch 2, the training loss decreased substantially to 0.031458, with the validation loss also decreasing to 0.044775. By Epoch 3, the training loss had decreased further to 0.004025, while the validation loss was 0.043525. The training loss showed a consistent downward trend throughout training, while the validation loss improved substantially between Epochs 1 and 2 and then remained relatively stable between Epochs 2 and 3.



Figure 9.9.1: Training loss and validation loss

The lower training loss compared with validation loss is expected because the model parameters are directly optimised using the training data. The validation performance remained very high, indicating that the model retained strong predictive capability on data that was not used for parameter updates.

The final test evaluation subsequently produced an overall F1-score of 99.49%, providing evidence that the strong validation performance was not restricted to the training data.

The test set was kept separate from model development and was used only for final evaluation, ensuring that the final reported performance was measured on previously unseen data.

9.10. Phishing-Class Performance of Transformer Models

Although overall performance provides the primary basis for model comparison, phishing-class performance is particularly important because the main objective of the system is to identify phishing emails.

Table 9.10.1: Phishing-class performance

Model	Precision	Recall	F1-score
RoBERTa-large	99.23%	98.78%	99.00%
DeBERTa-v3-large	98.93%	99.39%	99.16%
ModernBERT-large	98.93%	99.39%	99.16%
DistilBERT	98.93%	98.93%	98.93%

TinyBERT	97.04%	95.11%	96.06%
Gated Hybrid DeBERTa Model	99.69%	98.93%	99.31%
Attention-pooling Hybrid DeBERTa Model	99.23%	98.93%	99.08%
Gated Hybrid DistilBERT Model	98.18%	99.08%	98.63%
Gated Hybrid TinyBERT Model	92.20%	97.55%	94.80%

The results show that the larger transformer models generally maintained strong phishing-class performance. DeBERTa-v3-large and ModernBERT-large achieved the highest phishing recall at 99.39%, indicating that both models identified a very high proportion of the phishing emails in the test set. Their F1-scores were also high at 99.16%.

The hybrid models produced different precision and recall patterns across the transformer backbones. The DeBERTa-based hybrids achieved F1-scores of 99.31% and 99.08%, with precision values of 99.69% and 99.23%, respectively. The DistilBERT Gated Hybrid achieved the highest recall among the hybrid models at 99.08%, but its lower precision of 98.18% resulted in an F1-score of 98.63%.

A larger difference was observed for TinyBERT. The standalone model achieved 96.06% F1-score, while its gated hybrid achieved 94.80%. Its precision also decreased from 97.04% to 92.20%, despite an increase in recall from 95.11% to 97.55%. This result shows that adding handcrafted features did not produce a consistent improvement across all transformer backbones.

Overall, the class-specific results demonstrate that the models achieved different balances between precision and recall. DeBERTa-v3-large and ModernBERT-large provided the highest phishing recall, while the DeBERTa-based hybrid configurations produced strong precision and F1-score results. These differences provide a more detailed view of model behaviour than overall accuracy alone.

9.11. Confusion Matrix and Error Analysis

The confusion matrix provides a detailed view of the classification errors produced by the DeBERTa Gated Hybrid Model. As shown in Figure 9.11.1, the model correctly classified 1,096 safe emails and 647 phishing emails. Only two safe emails were incorrectly classified as phishing, while seven phishing emails were classified as safe.

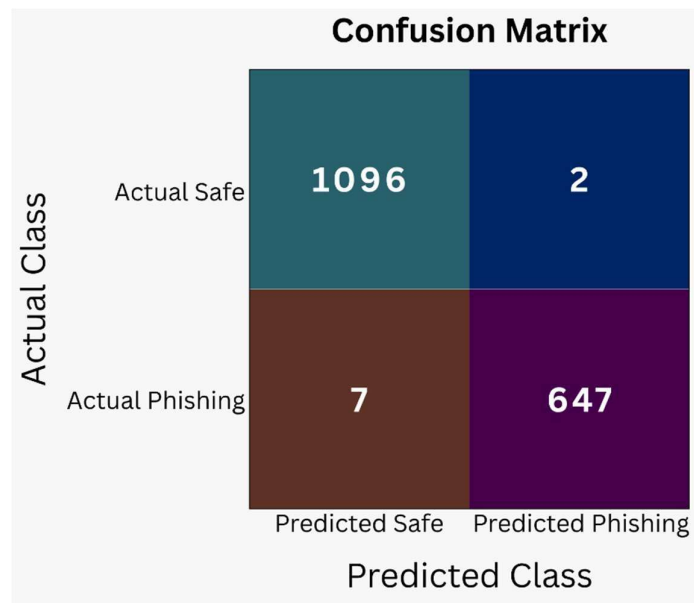


Figure 9.11.1: DeBERTa Gated Hybrid Model Confusion Matrix

The low number of misclassified emails demonstrates that the model maintained strong classification performance across both classes. The seven false negatives are particularly relevant to the phishing detection task because these emails were not identified as phishing. The two false positives represent safe emails that were incorrectly flagged as phishing. These results complement the precision, recall and F1-score analysis by showing the actual distribution of correct and incorrect predictions.

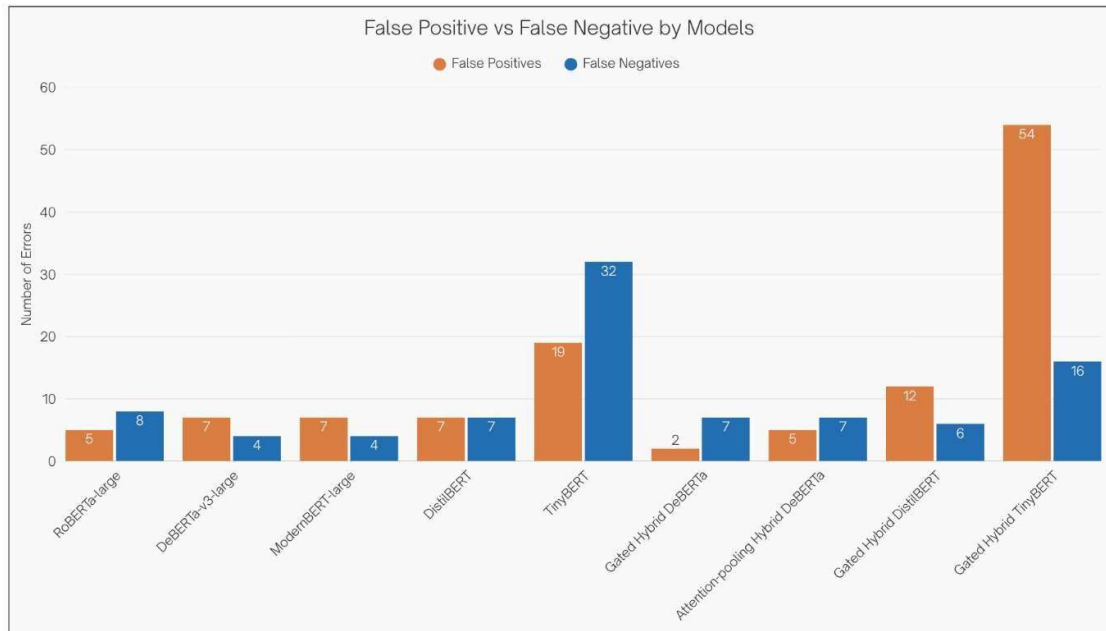


Figure 9.11.2: Comparison of false positive and false negative

It compares the number of false positives (FP) and false negatives (FN) produced by each evaluated model. False positives represent legitimate emails incorrectly classified as phishing, and false negatives represent phishing emails incorrectly classified as legitimate. Gated Hybrid DeBERTa Model recorded the lowest number of false positives, with only two cases, while DeBERTa-v3-large and ModernBERT-large achieved the lowest number of false negatives, with four cases each. TinyBERT Hybrid produced the highest number of false positives (54), indicating a greater tendency to misclassify legitimate emails as phishing. These results highlight the tradeoff between reducing false alarms and preventing phishing emails from being missed.

9.12. Comparison with Existing Work Using the Same Dataset

To provide a more meaningful evaluation of the proposed model, its performance was compared with studies that used the same “zefang-liu/phishing-email-dataset”. Using a common dataset provides a closer reference point than comparisons with studies based on different phishing corpora, although differences in preprocessing, data splits, training configurations and evaluation protocols must still be considered.

A Robust and Explainable Transformer-Based Framework for Phishing Email Detection directly used the “zefang-liu/phishing-email-dataset”, which contains approximately 18,650 email samples. The study applied a stratified 70% training, 15% validation and 15% test split and finetuned DistilBERT for binary phishing classification. Its baseline DistilBERT model achieved 98.0% accuracy on clean test data. (Sajad U P 2026)

Evaluating the Effectiveness of LLMs and ML reported in Algorithms evaluated several traditional machine-learning and transformer-based models using publicly available phishing email datasets, including the same Hugging Face dataset. Its DistilBERT model achieved 98.44% accuracy, 97.71% precision, 96.86% recall and 97.28% F1-score on the imbalanced email dataset. The study also evaluated traditional models, with Random Forest achieving 99.47% accuracy and 99.08% F1-score on the same imbalanced email evaluation. (Tarapiah et al. 2025)

A recent study Classical-First Selective Cascades for Resource-Constrained Phishing Email Detection used “zefang-liu/phishing-email-dataset as its primary dataset”. After cleaning, the study retained 18,634 emails and evaluated a range of classical and transformer-based approaches. Its fine-tuned DistilBERT reference achieved an F1-score of 96.1%, while its TF-IDF and Linear SVM configuration also achieved approximately 96.1% F1-score on the held-out dataset. (Alajaji 2026)

The proposed models produced higher measured results under the experimental configuration used in this project. DeBERTa-v3-large achieved 99.37% accuracy and F1-score, increasing to 99.49% after integration with the 12 handcrafted phishing-related features. The proposed hybrid model also achieved 99.49% precision and recall. These results are summarised in Table 9.12.1.

Table 9.12.1: Comparison with existing work on the same dataset

Study	Model	Accuracy	F1-score
Robust & Explainable Transformer Framework	DistilBERT	98.00%	–
Evaluating the Effectiveness of LLMs and ML	DistilBERT	98.44%	97.27%
Classical-First Selective Cascades	Fine-tuned DistilBERT	–	96.1%
Classical-First Selective Cascades	TF-IDF + Linear SVM	–	96.1%
Ours	Base DeBERTa-v3-large	99.37%	99.37%
Our proposed model	Gated Hybrid DeBERTa Model	99.49%	99.49%



Figure 9.12.1: Existing work comparison

Existing studies using the same dataset reported strong results with both transformer and classical approaches. These results provide a useful benchmark for the proposed approach, although differences in preprocessing and evaluation settings limit direct comparison. The findings demonstrate that the dataset supports effective phishing detection using both traditional and transformer-based methods.

9.13. Discussion of Findings

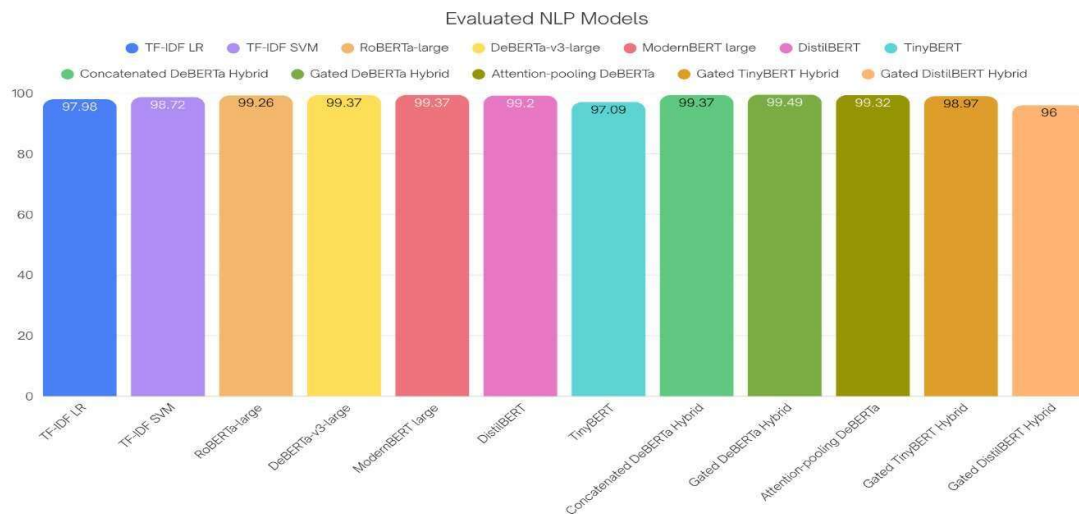


Figure 9.13.1: Discussion of finding

The results provide several insights into the behaviour of the evaluated classification approaches. The classical TF-IDF models established a strong baseline, with Logistic Regression achieving 97.98% accuracy and SVM improving this to 98.72%. The transformer models produced further improvements, with the larger architectures achieving accuracy above 99%.

Among the transformer models, DeBERTa-v3-large and ModernBERT-large both achieved 99.37%, showing only a small performance difference between the stronger contextual representations. The effect of feature integration varied across the backbones. The DeBERTa-based hybrid improved from 99.37% to 99.49%, while the DistilBERT and TinyBERT hybrid models showed lower performance than their standalone versions.

The fusion strategy also affected the results. Experiment 2 achieved 99.49%, compared with 99.32% for Experiment 3, showing that additional architectural complexity did not necessarily improve classification performance.

The final confusion matrix of Experiment 2 contained nine classification errors, including seven false negatives. This indicates that some phishing emails were still classified as legitimate and highlights an important limitation for security-focused detection.

The findings show progression from strong TF-IDF based classification to improved contextual modelling with transformers, with the results influenced by the choice of backbone, feature integration and fusion strategy.

9.14. Experimental Findings

The experimental evaluation identified several key patterns across the classical, transformer-based and hybrid models. The findings highlight differences in baseline performance, the influence of model architecture and the effect of incorporating handcrafted phishing-related features. The main findings are summarised in Table 9.14.1.

Table 9.14.1: Summary of Experimental Findings

Finding	Experimental Result	Discussion
Classical models provided a strong baseline	TF-IDF + Logistic Regression: 97.98% accuracy, TF-IDF + SVM: 98.72%	Classical TF-IDF models provided a strong reference point for evaluating the transformer approaches.
Larger transformers achieved stronger performance	Larger transformer models achieved above 99% accuracy, while TinyBERT achieved 97.09%	The results indicate a performance difference between the larger and
		lightweight transformer architectures.

Handcrafted features added complementary information	DeBERTa-v3-large: 99.37%, Experiment 2 Gated DeBERTa: 99.49%	The handcrafted phishing-related features provided additional information when combined with the DeBERTa representation.
Fusion strategy affected performance	Experiment 2 Gated DeBERTa: 99.49%, Experiment 3 Attentionpooling DeBERTa: 99.32%	The evaluated gated-fusion configuration performed better than the attentionpooling configuration.
Greater complexity did not improve performance	Experiment 3 introduced additional attention processing but achieved a lower result than Experiment 2	Additional architectural components did not produce a corresponding performance improvement.
Hybridisation depended on the transformer backbone	DeBERTa hybrid improved performance, while DistilBERT and TinyBERT hybrids decreased	The effect of handcrafted feature integration varied according to the underlying transformer architecture.
Multiple metrics were needed for evaluation	DeBERTa-v3-large and ModernBERT-large had higher phishing recall, while Experiment 2 had higher phishing precision and F1score	Accuracy alone does not fully describe the behaviour of a phishing detection model.
Errors remained despite high overall performance	Experiment 2 produced 2 false positives and 7 false negatives from 1,752 test emails	The model achieved high overall performance but still misclassified a small number of emails, including phishing emails classified as legitimate.

9.15. Final Model Selection

Based on the controlled experimental evaluation, Gated Hybrid DeBERTa Model was selected as the final model. The model achieved 99.49% accuracy, precision, recall and F1-score, with 9 incorrect classifications out of 1,752 test samples.

For the Phishing Email class, the model achieved 99.69% precision, 98.93% recall and 99.31% F1-score. The results also showed that the gated fusion architecture provided stronger overall performance than the other evaluation in this project.

Model selection was based on the combined results across overall classification performance, phishing-class performance, confusion-matrix analysis and architectural comparisons. Under the experimental conditions of this project, Experiment 2 - DeBERTa Gated Hybrid provided the strongest overall balance between the evaluated performance measures and was therefore used as the final model for the subsequent stages of the project.

9.16. Limitations of the Results

The results should be interpreted within the scope of the experimental design and evaluation conditions. The models were evaluated using a single dataset and fixed test split, so the reported performance may not reflect behaviour on new or externally collected phishing emails.

The hybrid experiments evaluated the combined handcrafted feature representation instead of the contribution of each feature individually. As a result, the effect of specific handcrafted features cannot be established from current experiments.

The performance differences between several high performing transformer models were also relatively small. These differences may be affected by training variation and experimental settings, so they should not be treated as substantial differences in generalisation performance.

The comparison with existing studies is subject to differences in datasets, preprocessing, data splits and evaluation procedures. Reported results from other studies should therefore be considered as reference points rather than direct head-to-head comparisons.

10. SYSTEM DESIGN AND ARCHITECTURE

Following the model-development phase, the project progressed from standalone NLP experiments to an integrated phishing email detection system. The resulting architecture was designed around a separation between user interaction, backend inference, machine-learning classification, supplementary security information and explainability. The project contains two genuine explainability implementations built around the same gated-hybrid DeBERTa-v3-large classification approach: a SHAP implementation and a LIME implementation.

The SHAP implementation consists of the DeBERTa-based classification pipeline, FastAPI backend, SHAP explainability component, Streamlit application and browser extension. The model and explanation logic are centralised in the backend rather than independently implemented inside each user interface. Streamlit and the browser extension therefore act as clients of the same prediction service. SHAP explanation is requested separately from ordinary classification because explanation generation requires substantially more computation than obtaining a prediction.

The LIME implementation uses the same hybrid classification concept but is exposed through a Flask application and Chrome extension/API workflow. The Flask service validates the incoming request, invokes the predictor and returns the prediction result as a JSON response. LIME is implemented as the local explanation method and produces influential terms for the selected class. The prediction logic remains in the backend, preventing duplication of the model in the JavaScript side.

Although the two implementations use different backend and explanation configurations, the central classification architecture remains the same. Email content is preprocessed and passed through two information paths: DeBERTa-v3-large provides the contextual representation, while 12 engineered security and linguistic features provide the numerical representation. The two branches are combined through gated fusion before the final Safe/Phishing classification. SHAP and LIME are positioned after classification and explain the resulting model prediction rather than independently deciding whether an email is phishing.

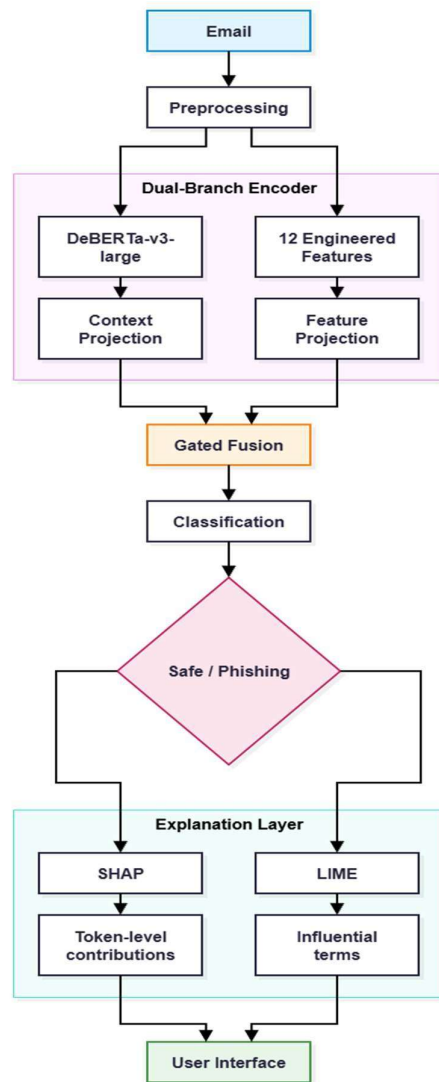


Figure 9.16.1: Overall System Architecture

Figure 10.1 presents the common system architecture used in the project. The email is first preprocessed before entering the dual-branch encoder. DeBERTa-v3-large captures contextual language information, while the 12 engineered features capture measurable linguistic and security characteristics. After context and feature projection, the representations are combined through gated fusion and passed to the classifier. The resulting Safe/Phishing prediction can then be explained through SHAP or LIME. SHAP provides token-level contribution information, whereas LIME provides locally influential terms and signed weights. The explanation layer therefore supports interpretation of the classifier output but does not replace the classifier.

10.1. SHAP-Based System Design and Architecture

The SHAP-based implementation was developed as one of the two explainability configurations of the phishing-email detection system. It integrates the trained gated-hybrid DeBERTa-v3-large classifier with a FastAPI backend, SHAP explanation pipeline, Streamlit application and browser extension. Within this implementation, the trained classifier remains responsible for producing the Safe/Phishing prediction, while SHAP operates as a post-hoc explanation mechanism for examining the contribution of individual tokens to that prediction.

A central design decision in the SHAP implementation was to separate normal classification from explanation generation. The FastAPI backend provides the common service boundary between the trained model and the user-facing applications, allowing both Streamlit and the browser extension to access the same classification pipeline. Prediction and explanation are exposed separately so that a user can receive the classification result without automatically executing the more computationally expensive SHAP procedure. This separation also keeps the machine-learning and explanation logic out of the user interfaces and avoids maintaining independent copies of the classifier across different clients.

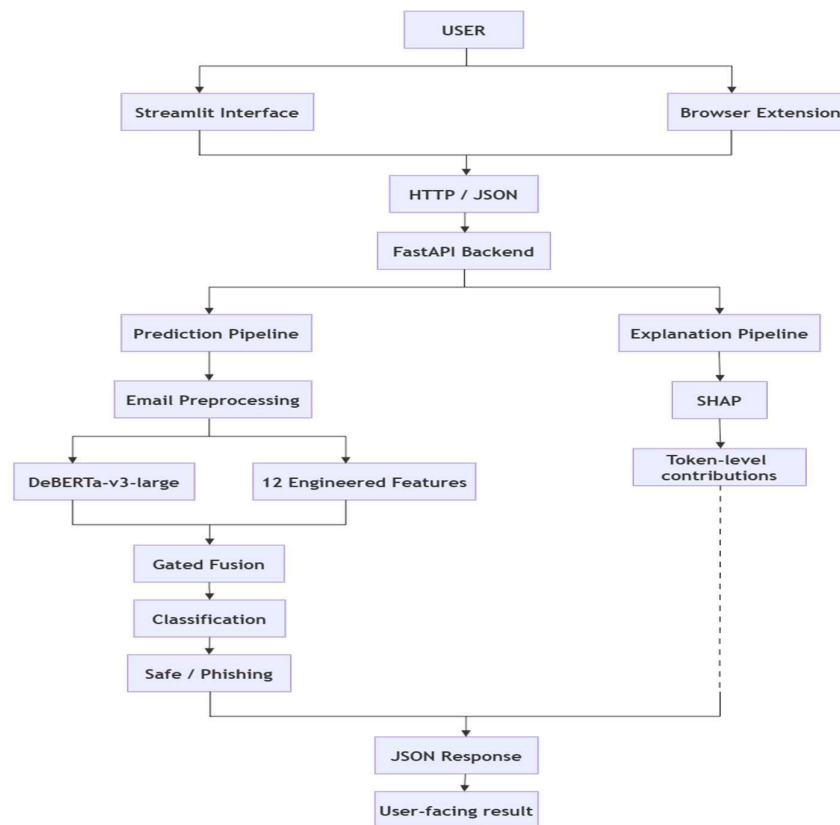


Figure 10.1.1: SHAP-Based System Architecture

The architecture shown in Figure 10.1.1 illustrates how the components of the SHAP implementation interact. Email information collected through Streamlit, or the browser

extension is transmitted to the FastAPI backend through HTTP/JSON. The prediction pipeline preprocesses the email and passes it through the DeBERTa-v3-large and engineered-feature branches before gated fusion and classification. When an explanation is requested, the SHAP pipeline operates on the trained classifier and returns token-level contribution information. Both the prediction and explanation results are then presented through the requesting user interface.

10.1.1 Architectural Design Principles

The first was separation of concerns. User-interface code is responsible primarily for collecting or obtaining email information and presenting returned results, while the backend is responsible for model inference. Explainability is additionally separated from ordinary prediction because generating SHAP values is substantially more computationally expensive than obtaining a classification.

The second was interface independence. A backend API provides a common boundary between the classifier and user-facing applications. This enables the Streamlit application and browser extension to use the same classification pipeline rather than embedding separate copies of the model.

The third was prediction/explanation separation. The trained model determines the Safe/Phishing classification. Security indicators and SHAP values provide contextual or explanatory information but do not independently override the model's decision. This distinction is particularly important for a security application because users could otherwise interpret a highlighted keyword or URL as the actual decision rule.

The fourth was progressive system integration. Components were developed and tested incrementally rather than constructing the entire system simultaneously. The project evidence records iterative integration and troubleshooting across the model, API, frontend and explainability components.

10.1.2 FastAPI Backend Design

FastAPI acts as the integration layer between the machine-learning components and the user-facing interfaces. Rather than requiring Streamlit or the browser extension to load and execute the transformer directly, the frontend submits email information to the backend and receives structured results.

This design provides an important engineering abstraction:

Frontend → HTTP request → FastAPI → ML inference → JSON response

The system exposes prediction and explanation functionality separately. The ordinary prediction workflow is provided through the `/predict` route, while explainability is handled through `/explain`.

The input payload used by the interfaces contains: sender, subject, body

The Streamlit implementation constructs this payload and sends it to the prediction endpoint as JSON. The browser extension follows the same basic communication pattern, submitting the extracted sender, subject and body to the backend.

Table 10.1.2.1: Principal API Responsibilities

Component	Responsibility
/predict	Execute normal classification workflow
/explain	Generate prediction-associated SHAP explanation
Input	Sender, subject and email body
Model layer	DeBERTa-based hybrid classification
Output	Prediction information and associated response fields
Client applications	Streamlit and browser extension

Separating /predict and /explain is preferable to automatically running SHAP for every classification. Normal prediction can be returned without waiting for the substantially heavier explanation procedure, while an explanation can be generated when required.

10.1.3 Streamlit Interface Design

Streamlit provides the primary standalone user-facing prototype. It allows a user to enter email information and submit it for analysis without directly interacting with the model or API.

The frontend sends the following JSON structure to FastAPI:

```
{
  sender: <sender>,
  subject: <subject>,
  body: <email
body> }
```

The source implementation confirms that normal prediction is sent to the prediction endpoint using an HTTP POST request. The interface also implements explicit handling for connection failures, request timeouts and other API errors.

The operational sequence is therefore:

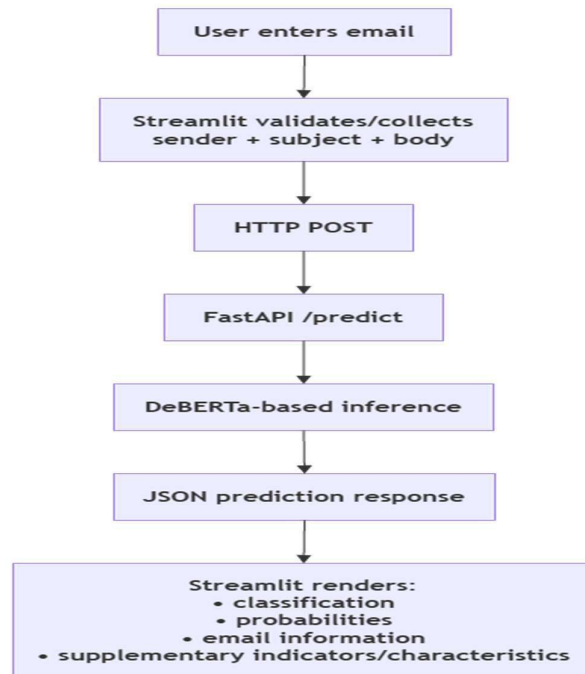


Figure 10.1.3.1: Streamlit prediction workflow

Explainability is requested separately:

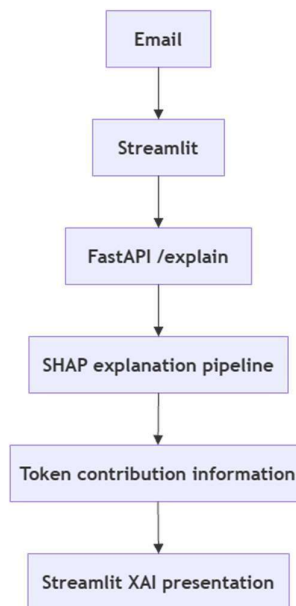


Figure 10.1.3.2: Explainability Request

10.1.4 Explainability Architecture

The explainability layer was implemented using SHAP as a post-hoc mechanism. Its architectural position is intentionally downstream from the trained classifier.

The relationship can be represented as:

$$f(x) \rightarrow y$$

where f is the trained classifier, x is the email and y is the model prediction.

SHAP then analyses the behaviour of f around that prediction to estimate token-level contributions. It does not produce an independent ground-truth judgement.

Therefore:

Classifier $\square$ Prediction



SHAP



Explanation of prediction rather than: SHAP $\square$

decides whether email is phishing

This distinction is essential for interpreting the resulting interface. A token with a strong SHAP contribution indicates that the token influenced the model output under the explanation procedure; it does not prove that the word is inherently a phishing indicator.

10.1.5 SHAP Computational Refinement

The original explanation implementation introduced an important engineering challenge. Applying SHAP to DeBERTa-v3-large required repeated model evaluations, which resulted in long execution times and session instability. The project therefore separated the computational requirements of classification from explanation generation rather than reducing the normal classifier's input configuration.

The revised SHAP implementation introduced bounded explanation parameters:

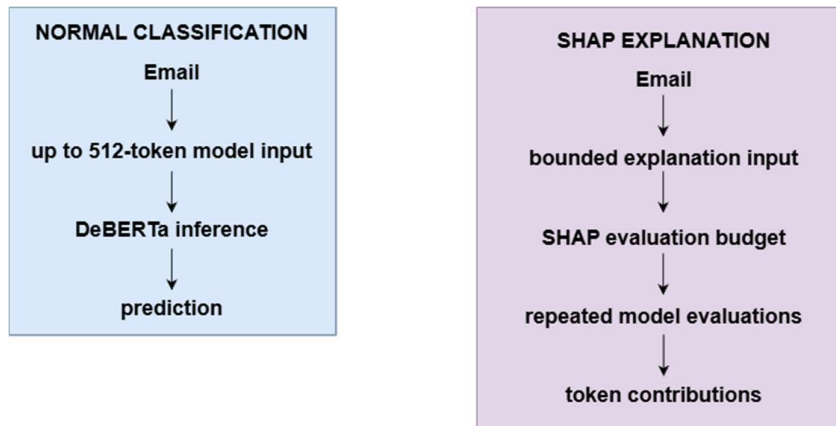
$$SHAP_MAX_TOKENS = 64$$

$$SHAP_MAX_EVALS = 150$$

$$SHAP_BATCH_SIZE = 4$$

The explanation route also performs garbage collection and clears the CUDA cache where available before generating the explanation. The shortened text is used specifically for SHAP processing, after which the explainer executes with explicit `max_evals` and `batch_size` constraints.

Consequently, two different computational paths exist:



10.1.6 Browser Extension Design

The browser extension was developed as a stretch goal to reduce the effort required to analyse suspicious emails. Instead of copying email content into a standalone application, users can initiate detection from the browser. The extension uses Manifest V3 and communicates with the local FastAPI backend at <http://127.0.0.1:8000>.

The extension supports two interaction paths: a popup that requests email content from content.js, and an injected Scan Email for Phishing button in the controlled test environment. The content script extracts the sender, subject and body, then submits them to the /predict endpoint as JSON. The backend applies the same preprocessing and trained model used by the Streamlit interface, ensuring that classification remains consistent across both interfaces.

The prediction is displayed in an overlay with the classification and confidence information. Users can optionally select Explain Decision to request a SHAP explanation through the /explain endpoint. The interface distinguishes phishing and safe predictions and presents supporting and opposing factors. Error handling covers failed extraction, empty content, unsuccessful API responses and backend errors, reducing the risk of failures being mistaken for safe results.

The extension's main limitation is email extraction. Although the manifest includes Gmail, the current extraction logic is designed around the project's controlled University Email test-page structure. Reliable Gmail extraction has not been established and would require a dedicated extraction method and further testing. The extension is therefore a demonstration prototype rather than a production-ready email security tool.

From a security and privacy perspective, the extension accesses email content on matched pages and sends it to the local backend. The restricted local API host permission limits remote API access, but content access must still be minimised. Production deployment would require a privacy notice, appropriate permission controls, secure backend communication and defined data-retention practices.

The extension functions as an orchestration and presentation layer rather than a separate classification model. Its workflow can be represented as:

[

```
\text{Output}=\text{UI}(\text{API}(\text{Extract}(\text{Page})))  
]
```

The extension extracts email content, sends it to the backend for classification and displays the returned result. The classification probabilities are generated by the backend model, not by the JavaScript extension.

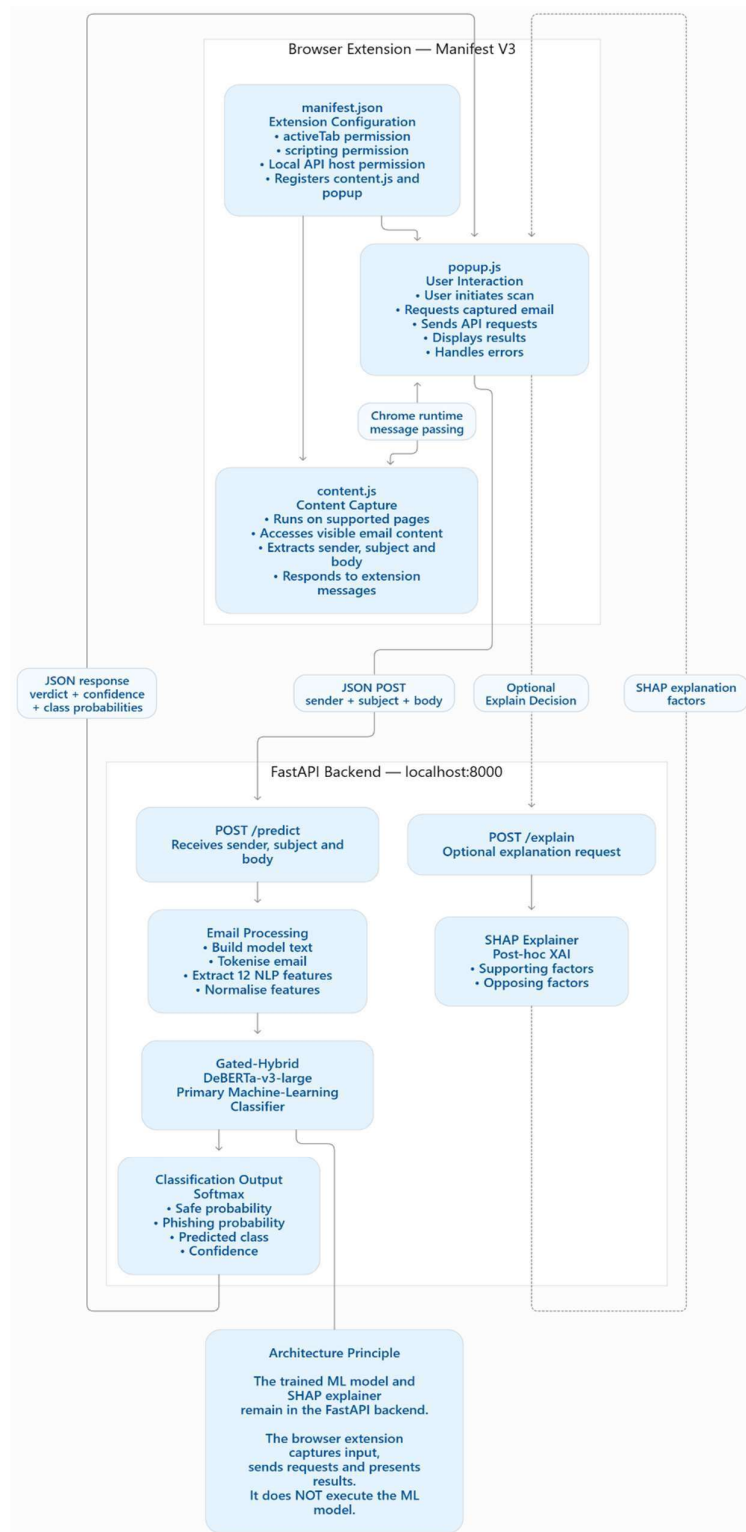


Figure 10.1.6.1: Browser Extension Component Architecture

10.1.7 Browser Extension Components

The supplied source establishes at least three important extension responsibilities: the popup interface, popup control logic and page/email extraction through a content-script communication workflow.

When the user initiates analysis, popup.js queries the currently active browser tab: `chrome.tabs.query(...)`. It then sends a message to the tab requesting the email: action: "getEmail"

The returned object is expected to contain detected email information, including the body and where available, sender and subject. The extension checks whether an email was successfully obtained before continuing. The popup displays classification information together with email details and supplementary indicators. The source also includes logic to display Safe and Phishing probabilities and normalize their presentation as percentages.

The resulting workflow is:

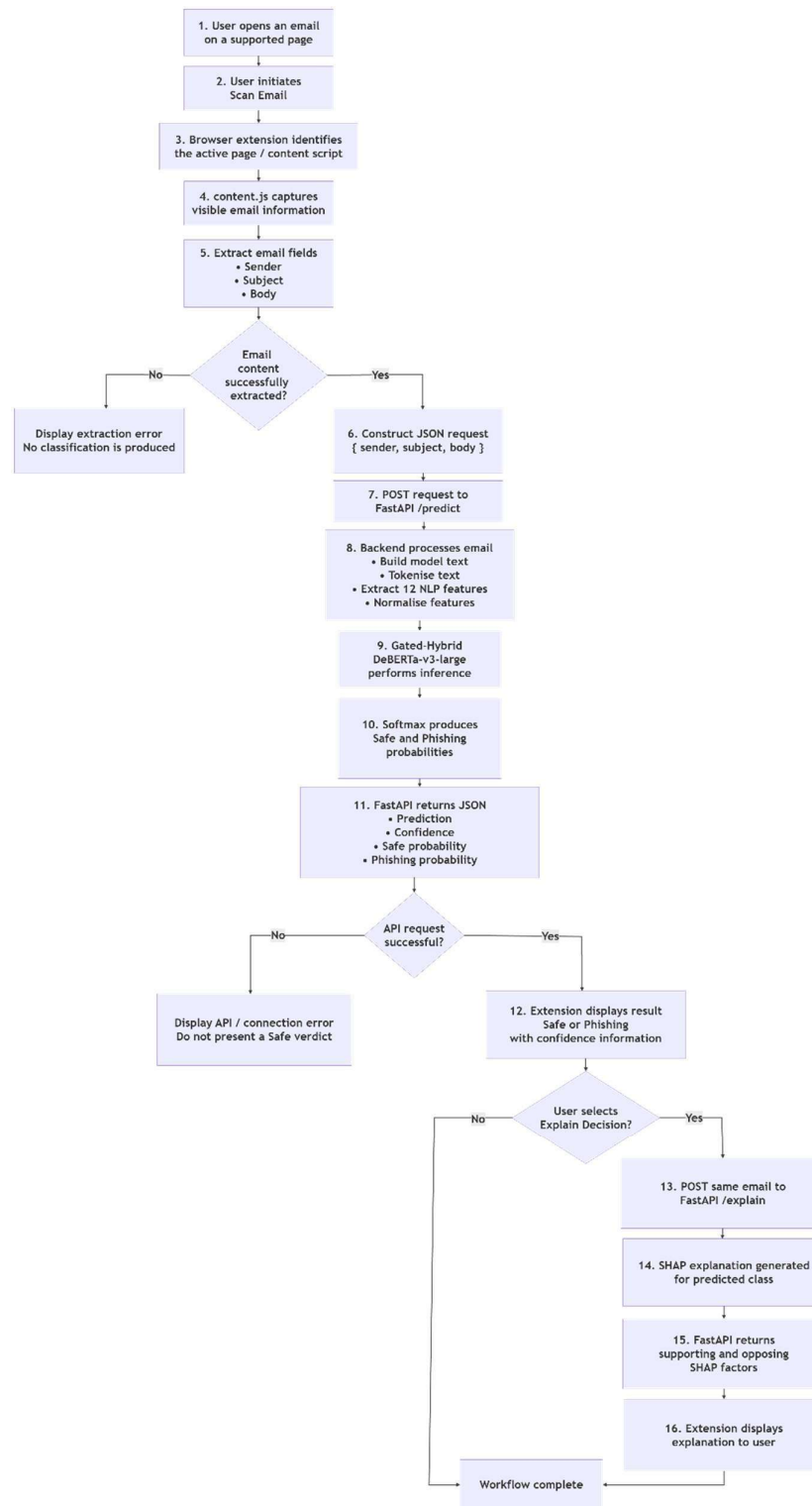


Figure 10.1.7.1: Browser Extension Email Capture Workflow

10.1.8 Browser Extension User Interface

The extension popup was designed to provide information directly in the browsing context rather than redirecting the user to another application.

The interface contains email information such as sender and subject and includes an area for phishing indicators. The supplied final popup HTML identifies the system as powered by DeBERTa-v3-large.

The extension logic further distinguishes four presentation situations:

- 1) safe prediction with no phishing indicators
- 2) safe prediction with neutral characteristics
- 3) phishing prediction with supplementary indicators
- 4) phishing predictions where no rule-based indicator is available.

This distinction is explicitly documented in the source code.

Of particular importance, the source contains a safeguard stating that a phishing prediction must not display the reassuring message: “No common phishing indicators detected” as though that contradicted the classifier.

This is a small but meaningful usability decision. A model can classify an email as phishing even when simple manually recognizable indicators are absent. Displaying a green “no indicators” message in that situation could create contradictory risk communication.

10.1.9 Error Handling and System Robustness

A usable ML prototype must address failures outside the classifier itself. The implemented interfaces therefore contain error-handling mechanisms for communication and extraction failures.

For Streamlit, the source handles:

- inability to connect to the API;
- request timeout;
- general HTTP/request errors; and
- separate timeout handling for SHAP explanation.

For the browser extension, the source checks whether an active browser tab can be accessed and whether valid email content has been returned before sending the classification request. If email extraction fails, an error is raised rather than sending an empty message to the classifier.

These controls do not make the prototype production-ready, but they improve robustness during demonstration and functional testing by distinguishing model output from infrastructure failures.

10.1.10 Deployment Considerations

The implemented system demonstrates functional integration but remains a prototype architecture.

The local FastAPI configuration is suitable for controlled development and demonstration but would require significant engineering before public deployment. A production architecture needs to consider authentication, HTTPS, concurrency, model-serving infrastructure, resource management, monitoring, rate limiting and secure handling of email data.

DeBERTa-v3-large also introduces computational deployment costs. Its strong predictive performance must therefore be considered alongside memory consumption and inference latency. SHAP is still more computationally expensive because explanations require repeated model evaluations, which is why the project introduced a bounded explanation pathway.

A production design could therefore retain the architectural separation already demonstrated in the prototype:

Fast prediction service

- ☐ Immediate classification
- ☐ Optional explanation service

10.1.11 End-to-End System Workflow

Combining the preceding components gives the complete implemented workflow.

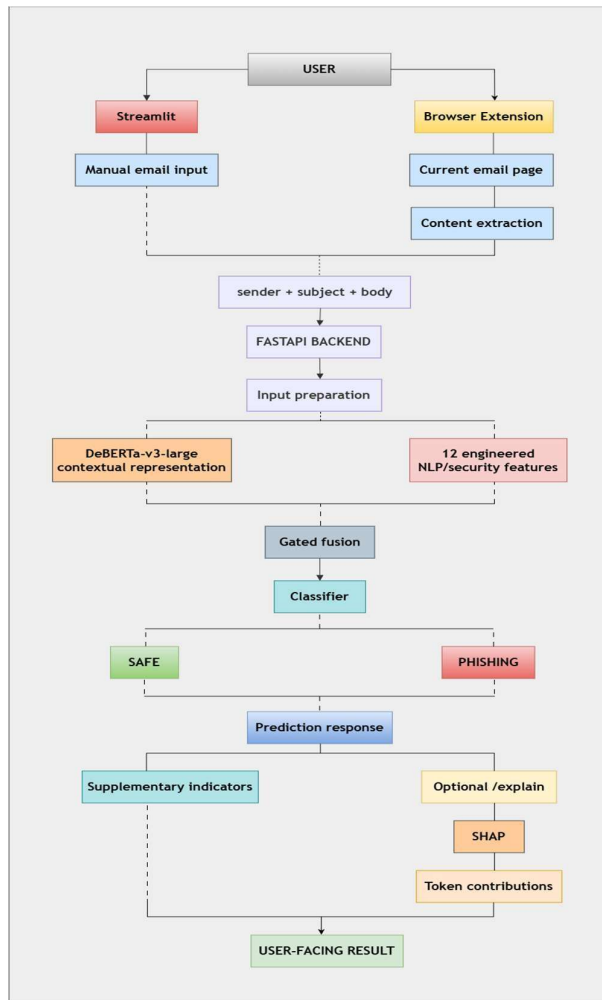


Figure 10.1.11.1: End-to-End System Workflow

10.2. LIME-Based System Design and Architecture

The project also retains the LIME-based implementation as a separate prototype configuration. The phishing-email detection workflow was implemented in a multi-layered web application separating the user experience, request routing, machine-learning inference, explainability, and model persistence layers. The prototype is a combination of transformer-based language model and twelve engineered security and linguistic features. Implementation is done using the Local Interpretable Model-agnostic Explanations method, using LimeTextExplainer.

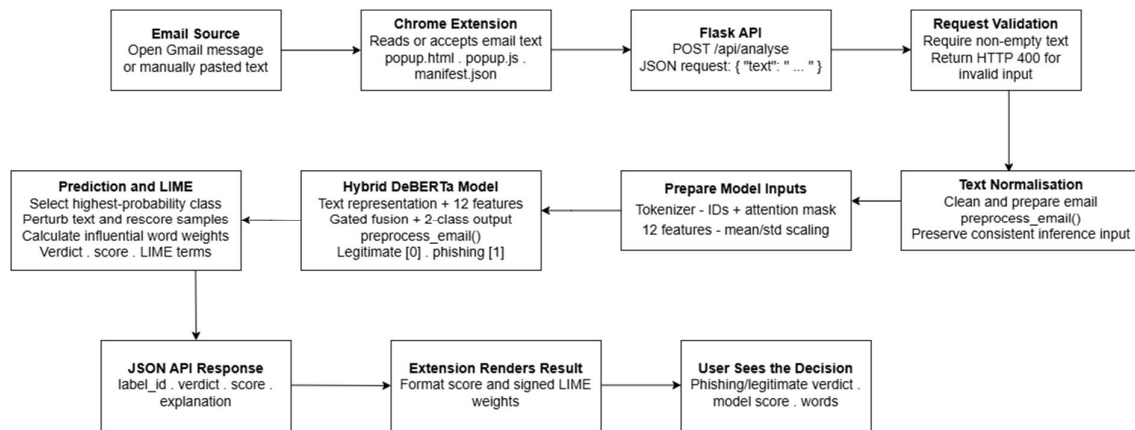


Figure 10.2.1: LIME-Based System Design and Architecture

The presentation layer includes a Chrome extension popup and a different command line API client. The extension can obtain the visible text of an open Gmail message using a contentextraction script. This is in addition to a manual text-entry option as automatic extraction may be impacted by changes to the document structure in Gmail. The manually entered/extracted information is passed to the back end as a JSON request.

The Flask application (app.py) is the service layer. It unmarks the health-check route and the emailanalysis route, validates the incoming requests, invokes the predictor and returns the prediction result as a JSON response. API has no implementation specific logic in it. This helps to isolate the machine learning responsibilities from the web-service responsibilities and enables the same predictor to be used by an evaluation script, by the browser extension or by another authorised client.

The prediction layer is implemented in predictor.py. Loads the tokenizer, transformer configuration, trained weights, engineered-feature statistics and model settings from the model directory. These resources are loaded only once, at start-up of Flask, and not for every request. This design is crucial since the backbone of DeBERTa-v3-large is computation-intensive while running inference on a CPU machine.

The trained classifier has a hybrid architecture. The contextual representation generated by the transformer is projected to compatible representations along with the 12-dimensional engineeredfeature vector before gated fusion and binary classification. The 12-dimensional

engineered-feature vector and the contextual representation generated by the transformer are transformed to compatible representations before gated fusion and binary classification. This design aligns with the findings that both contextual language modelling and the explicit identification of socialengineering behavior (e.g., phishing) are effective techniques for detecting phishing (Al-Subaiey et al. 2024; Uddin, Mahiuddin & Sarker 2026).

A request consequently travels through the architecture one way as an email message and the other way as a decision in structure:

Validation in Chrome extension or API call, preprocess (Flask), tokenisation/feature extraction, hybrid classifier, LIME explanation, JSON response, UI.

The components do not share user-interface state directly, but rather through clearly defined interfaces. This separation helps to facilitate future development and reproducibility. If later a web interface is developed instead of the local Chrome extension, the model implementation does not need to be modified. Likewise, the model could be used in the background of the existing production web server, with the same requests and responses.

10.2.1 Explanation Component

LIME was integrated to improve interpretability and support error analysis, particularly when legitimate emails are incorrectly flagged as phishing. Using LimeTextExplainer, the system generates perturbed versions of an email, observes the classifier's predictions and fits a local surrogate model to identify influential terms (Ribeiro, Singh & Guestrin 2016).

The implementation uses legitimate and phishing as class labels, a random seed of 42, 50 perturbed samples and displays the eight most influential terms. The predict() function handles normal classification, while predict_with_explanation() generates both the prediction and its explanation. This separation avoids the additional computational cost of LIME when explanations are not required.

Positive term weights support the class being explained, while negative weights oppose it in the local approximation. Their magnitudes indicate relative influence within that explanation; they are not probabilities, causal effects or definitive indicators of malicious intent. LIME explanations may also vary with sampling and the input context. Therefore, the interface presents these terms as influential terms, not as the model's definitive reasoning (Salih et al. 2025; Uddin, Mahiuddin & Sarker 2026).

10.2.2 API and Interface

The LIME prototype uses a Flask API and a Chrome extension built with Manifest V3. The API provides a GET / health-check endpoint to confirm service availability and a POST /api/analyse endpoint to classify email text and return an explanation.

The response includes a numeric label, a readable verdict, the predicted-class score and a LIME explanation containing influential terms and their signed weights. The Chrome extension acts as a thin client, sending email text to the backend and displaying the result. Keeping inference on the backend avoids duplicating the model in JavaScript and ensures that API and browser-based tests use the same classifier.

The extension uses the activeTab and scripting permissions, together with access to the local API, to limit unnecessary browser access (Google 2024). The interface also handles empty input, connection failures, invalid responses and request timeouts. LIME requires multiple model predictions, so a 120-second timeout is used when running DeBERTa-v3-large on CPU. This reflects the prototype's experimental purpose rather than suitability for high-volume, real-time mail processing.

10.2.3 Deployment Environment

The prototype is locally deployed on a Windows computer with Visual Studio Code, Python virtual environment and Flask. The virtual environment serves to separate the application's dependencies from the system Python (python). Flask, PyTorch, Transformers, NumPy, SentencePiece, scikitlearn and LIME are required packages. This will aid in reproducing the result on another compatible computer, as these dependencies are recorded in the requirements.txt file.

The application is attached to: `http://127.0.0.1:5000`

The loopback address is a service that is available on the same computer but not displayed on the local network as part of the computer's design. This helps to meet the demonstration requirement without exposing any unnecessary network. When the user uses the service as an ordinary local service, the message text will not be sent to a cloud inference service.

The model is created only when the application is first run (app.py). This generates an obvious start up time but prevents the reloading of a big transformer on each request. Once initialised, Flask stays waiting for requests and will re-use the same model instance. Inference mode (0): It is used to disable calculation of gradient which will help to save memory and computing power during prediction.

The deployment should still be considered a local prototype and not a production service. According to the Flask documentation, the built-in development server is not intended for production deployment because it's not as secure, stable or efficient (Pallets Projects 2025). A production version should deploy an appropriate WSGI server or a containerised version of the service, have controlled network exposure, have a structured log, have authentication and rate limiting and have monitored resource limits.

Containerisation would also ensure deployment uniformity, by encapsulating the Python runtime, Python dependencies and application configuration. But having the large model inside of a container would make the image size and deployment time larger. A more mature design would store model artefacts in a controlled model registry and check the integrity of the stored artefacts before loading.

10.3. SHAP and LIME within the System

The trained model determines the Safe/Phishing classification. Security indicators and SHAP values provide contextual or explanatory information but do not independently override the model's decision. The LIME implementation follows the same separation: the classification

result can be tested without the explanation layer, which is the layer responsible for the explanation of the result.

The explainability layer was implemented using SHAP as a post-hoc mechanism. SHAP analyses the behaviour around a prediction to estimate token-level contributions. LIME is model agnostic and creates perturbed versions of the selected email, observes changes in the classifier's probabilities and fits an interpretable local surrogate model. These yield weighted terms for the decision close to that input.

Both explanation methods therefore remain downstream from the trained classifier. SHAP contribution information and LIME influential terms explain model behaviour; they do not produce an independent ground-truth judgement and do not determine whether the email is phishing.

The two implementations also expose different backend configurations. The SHAP implementation uses FastAPI with separate /predict and /explain routes, while the LIME implementation uses Flask and POST /api/analyse. Both keep prediction logic in the backend and allow the browser extension or another client to consume the same underlying classifier without implementing the model in JavaScript.

The computational requirements of explanation are greater than ordinary classification in both implementations. Applying SHAP to DeBERTa-v3-large requires repeated model evaluations, while LIME generates modified versions of the email and continues to ask the classifier. The final project therefore treats explanation as supporting information associated with a prediction rather than as the classification mechanism itself.

10.4. SHAP and LIME Comparison

The trained model determines the Safe/Phishing classification in both implementations. SHAP and LIME operate as post-hoc explanation mechanisms and provide supporting information about the resulting prediction rather than independently determining whether an email is phishing. Although both implementations use the same underlying gated-hybrid classification approach, they differ in their explanation mechanisms, backend configurations and computational workflows.

Table 10.4.1: SHAP and LIME Comparison

Aspect	SHAP Implementation	LIME Implementation
Purpose	Explains the Safe/Phishing prediction produced by the trained classifier.	Explains the Safe/Phishing prediction produced by the trained classifier.
Explanation approach	Post-hoc explanation using SHAP.	Post-hoc, model-agnostic explanation using LIME.

How explanation is generated	Analyses model behaviour around the prediction to estimate token-level contributions.	Creates perturbed versions of the selected email, observes changes in classifier probabilities and fits an interpretable local surrogate model.
Explanation output	Token-level contribution information.	Influential terms with signed local weights.
Role in classification	Does not independently determine or override the Safe/Phishing classification.	Does not independently determine or override the Safe/Phishing classification.
Backend framework	FastAPI	Flask
Local backend	http://127.0.0.1:8000	http://127.0.0.1:5000
API routes	/predict for classification and /explain for SHAP explanation.	POST /api/analyse for the analysis workflow.
User interfaces	Streamlit application and browser extension.	Browser extension/API client.
Computational requirement	More computationally expensive than ordinary classification because SHAP requires repeated model evaluations.	More computationally expensive than ordinary classification because LIME generates perturbed emails and repeatedly queries the classifier.
Relationship with classifier	Operates downstream from the trained classifier.	Operates downstream from the trained classifier.

Despite their different explanation mechanisms and backend configurations, both implementations maintain the same fundamental separation between classification and explanation. The trained gated-hybrid model produces the Safe/Phishing prediction, while SHAP or LIME provides additional information to help interpret that prediction. The explanation output should therefore be treated as supporting information about model behaviour rather than as an independent groundtruth judgement.

10.5. Security and Privacy Considerations

The SHAP and LIME prototypes process email content that may contain personal, confidential or organisationally sensitive information. Both implementations use local backend services, with FastAPI at 127.0.0.1:8000 for SHAP and Flask at 127.0.0.1:5000 for LIME. This loopback configuration avoids sending email content to a remote service during local demonstrations, but it does not make the prototypes suitable for production deployment.

Input validation is essential because prediction and explanation operations consume computational resources. The LIME implementation rejects malformed JSON, missing or

blank text, and inputs exceeding 30,000 characters. However, these controls alone cannot prevent resource exhaustion from repeated requests. Remote deployment would require additional safeguards, including authentication, rate limiting, execution timeouts, access controls and HTTPS/TLS. Sensitive email content should also be excluded from logs unless retention is necessary and appropriately protected.

The browser extensions should follow the principle of least privilege and request only the permissions required to access supported email content and communicate with their respective backends (Google 2024). Email samples, screenshots and assessment evidence should be reviewed for personal or confidential information. Where possible, public benchmark data or appropriately de-identified samples should be used. This approach aligns with the Australian Privacy Principles concerning the protection and handling of personal information (Office of the Australian Information Commissioner 2025).

The system also has important detection limitations. It primarily analyses email text and does not independently verify sender authentication through SPF, DKIM or DMARC, inspect attachments, or perform live domain-reputation and threat-intelligence checks. The prototypes should therefore be treated as decision-support tools that complement, rather than replace, established emailsecurity controls.

11. DEPLOYMENT AND END-TO-END DEMONSTRATION

The deployment phase integrated the trained gated-hybrid DeBERTa-v3-large classifier into two local prototypes. The objective was to demonstrate the complete workflow from email input to Safe/Phishing classification, confidence output and post-hoc explanation.

The SHAP-based configuration uses a FastAPI backend at 127.0.0.1:8000, integrated with a Streamlit application and Chrome browser extension. It returns classification probabilities and token-level SHAP explanations.

The LIME-based configuration uses a Flask backend at 127.0.0.1:5000, together with the trained model, LIME explanation component and Chrome browser extension. Its `/api/analyse` endpoint returns the prediction, model score and influential terms.

Although the configurations use different frameworks and explanation methods, both demonstrate how the same hybrid classifier can be integrated into a user-facing workflow. SHAP estimates feature contributions, whereas LIME approximates the classifier's behaviour around an individual email. In both cases, explanations support the interpretation of predictions rather than independently determining whether an email is malicious.

Both implementations are local prototypes intended for demonstration and evaluation, not production email-security services. The following sections present their deployment environments, workflows and demonstration outputs separately.

11.1. SHAP-Based Deployment and End-to-End Demonstration

SHAP-based deployment is one of two explainability configurations developed for the phishing email detection system. It integrates the trained DeBERTa-v3-large gated-hybrid classifier with a local FastAPI backend, Streamlit interface, Chrome Manifest V3 extension and SHAP explainability component.

The FastAPI backend, running locally at 127.0.0.1:8000, provides a shared service for classification and explanation. The classifier combines contextual representations from DeBERTa-v3-large with twelve handcrafted phishing-related features through a learned gated-fusion mechanism to produce a Safe or Phishing prediction.

The deployment provides two user-facing interfaces. The Streamlit application accepts sender information, subject and email body, then displays the prediction, class probabilities, supplementary security indicators and SHAP explanation. The Chrome extension supports email analysis through its popup and an injected Scan Email for Phishing button. Both interaction methods use the same backend and classifier rather than separate detection systems.

SHAP generates a post-hoc explanation after classification, identifying factors that support or oppose the predicted class. It helps users interpret the model's decision but does not change the original prediction or independently determine whether an email is malicious.

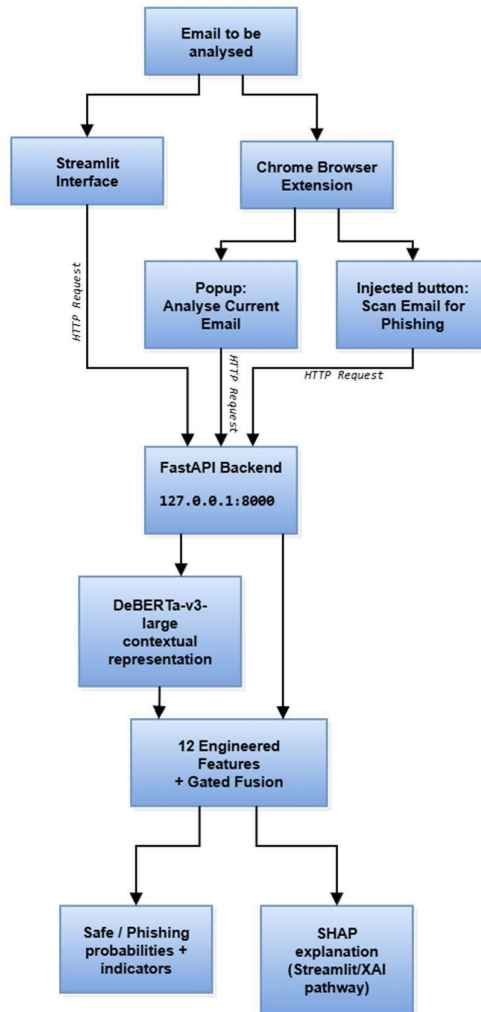


Figure 11.1.1: Deployment and end-to-end demonstration architecture

The image shows a Streamlit web application interface for email analysis. On the left, there is a 'System Status' sidebar with a green 'API Online' indicator. The main content area is titled 'University Email Security' and 'AI-Powered Phishing Email Detection System'. It includes a brief description: 'Analyse an email using the DeBERTa-v3-large phishing detection model.' Below this is the 'Email Analysis' section with input fields for 'Sender Email Address' (pre-filled with 'sender@example.com'), 'Email Subject' (placeholder: 'Enter the email subject...'), and 'Email Body' (placeholder: 'Paste the email body here...'). A red 'Analyse Email' button is at the bottom. On the right, there is a 'Deploy' sidebar with options for 'System', 'Light', and 'Dark' themes, and buttons for 'Return', 'Auto-refresh', 'Clear cache', 'Print', and 'Record screen'.

Figure 11.1.2: Representative Streamlit email-analysis input interface.

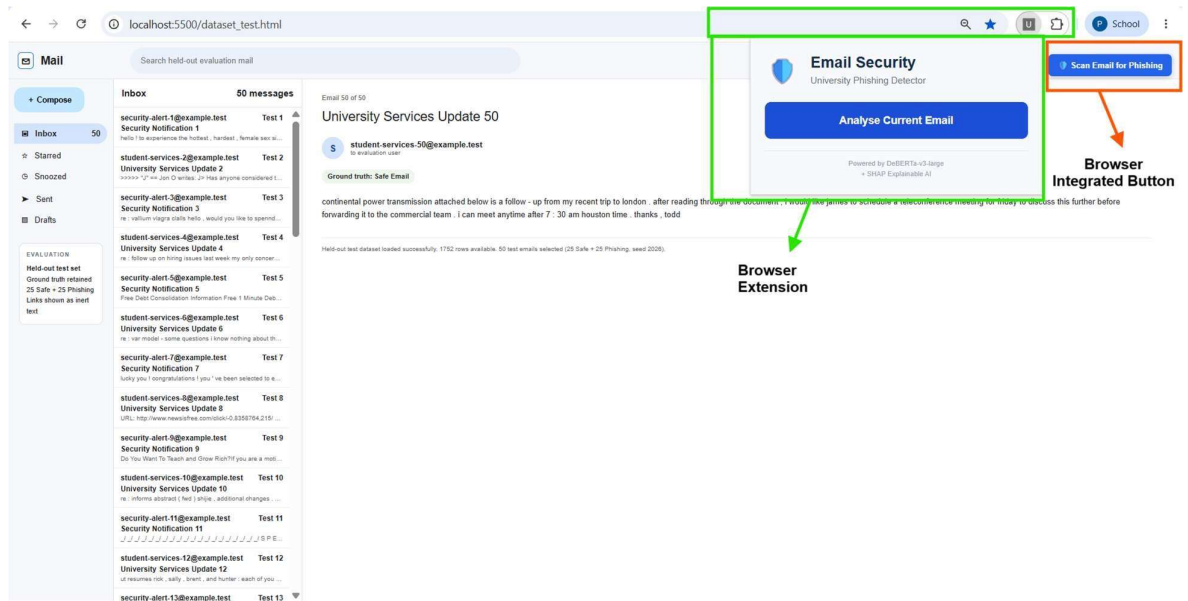


Figure 11.1.3: Mail Interface showing Browser Extension and Browser Integration button

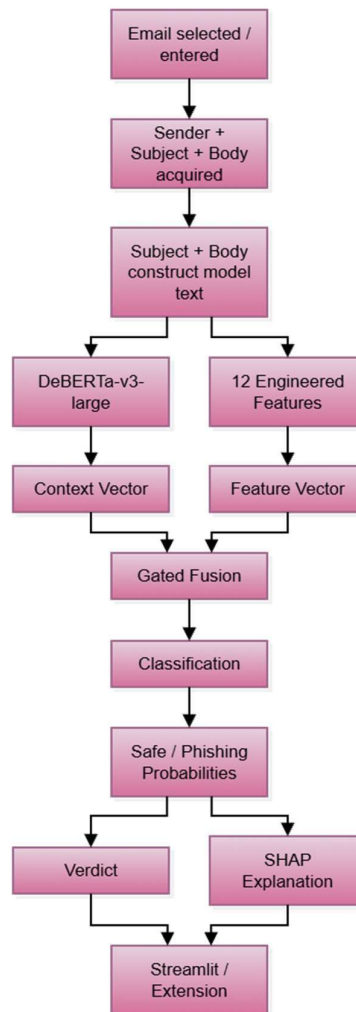


Figure 11.14: End-to-end email-processing and explanation workflow.

11.1.1 Demonstration Scenario

The demonstration strategy was designed to provide evidence at several levels rather than relying on a single user interface. It demonstrates that the trained classifier is reusable as a backend service and that different client interfaces can invoke the same analytical capability. The demonstrations were conducted in a controlled environment so that behaviour could be reproduced without interacting with live-phishing infrastructure.

Demonstration A - Streamlit interface

The Streamlit interface provided the most complete analytical demonstration. For each selected email, the sender details, subject and email body were entered, and the email was submitted for analysis. The interface then presented the predicted class, confidence, Safe and Phishing probabilities, security indicators and structural characteristics. A separate Explain AI Decision action generated the SHAP explanation. This pathway is valuable for evaluation because it exposes both the classifier output and the interpretability layer in a single interface.

The screenshot shows a web interface titled "University Email Security" with the subtitle "AI-Powered Phishing Email Detection System". Below this, it states "Analyse an email using the DeBERTa-v3-large phishing detection model." The main section is "Email Analysis" and contains three input fields: "Sender Email Address" with the value "security-alert-1@example.test", "Email Subject" with the value "Security Notification 1", and "Email Body" with a long, detailed phishing email body text. At the bottom, there is a red button labeled "Analyse Email".

University Email Security
AI-Powered Phishing Email Detection System

Analyse an email using the DeBERTa-v3-large phishing detection model.

Email Analysis

Sender Email Address
security-alert-1@example.test

Email Subject
Security Notification 1

Email Body
secure your account dear lasalle bank customer , we recently noticed one or more attempts to login into your lasalle bank online banking account for a foreign ip address and we have reasons to believe that your account was hijacked by a third party without your notification . if you recently logged into your account while traveling to a foreign country , the unusual login attempts may have been made by you . however if you are the rightful owner of the account , click on the link below and submit as we are trying to verify your account information . (in case you are not enrolled use your social security number as you user id and the first six digits of your social security number as a password) . the login attempt was made from : ip : 82 . 89 . 87 . 55 isp host : host 55 - 87 . pool 8289 . interbusiness . it if you chose to ignore our request , we have no choice but to temporarily suspend your online banking account .

Analyse Email

Detection Result

Potential Phishing Email Detected
Confidence: 99.95%

Probability Breakdown

Safe Probability 0.05% Phishing Probability 99.95%

The model produced a high confidence classification.

Email Details

Sender: security-alert-1@example.test
Subject: Security Notification 1

Security Indicators

These indicators provide additional context about patterns detected in the email. They supplement the DeBERTa-v3-large classification and are not the model prediction itself.

Possible request for credentials or account information

Email Characteristics

These are neutral structural characteristics of the message. Their presence alone does not mean that an email is phishing.

1 phone number present

Moderate-length email content detected

Explainable AI Analysis

SHAP Explainable AI identifies the words and tokens that contributed most strongly to the model's decision.

Explain AI Decision

SHAP explanation generated successfully.

SHAP Explanation

Explanation for: Phishing Email

Factors Supporting the Prediction

- 1. Notification
Contribution: 0.000513
- 2. account
Contribution: 0.002057
- 3. 1
Contribution: 0.001349
- 4. secure
Contribution: 0.000712
- 5. account
Contribution: 0.000291
- 6. customer
Contribution: 0.000188
- 7. ,
Contribution: 0.000168
- 8. while
Contribution: 0.000095
- 9. traveling
Contribution: 0.000095
- 10. if
Contribution: 0.000095

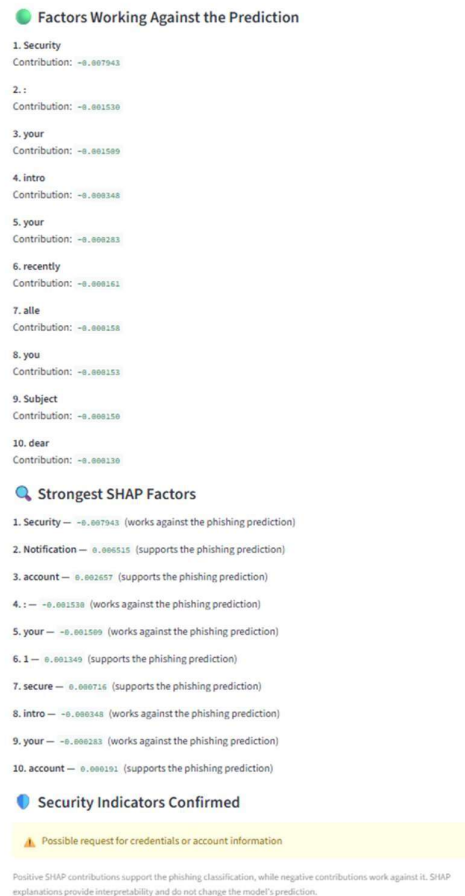


Figure 11.1.2.1: Representative Streamlit detection result showing the deployed classifier output.

Demonstration B - Browser extension deployment

The popup action and the injected scan button are two triggers within one browser-extension deployment. They share the same extraction/integration logic and the same backend classifier; they are not separate user-facing systems.

The second principal demonstration used the Chrome browser extension within the controlled mail-style held out test environment. This deployment was exercised through two user interaction mechanisms. The first was the extension popup, where the user selected “Analyse Current Email”. The second was the injected “Scan Email for Phishing” button displayed on the controlled email page. In both cases, the extension acquired the currently displayed email, communicated with the local FastAPI service and displayed the returned classification. Chrome extension architecture supports this pattern through content scripts, permissions and message passing between extension components.

This demonstration is significant because it reduces the interaction cost of analysis. The user does not need to copy the email into a separate ML interface. Instead, the security result appears in the same browser context in which the message is being reviewed.

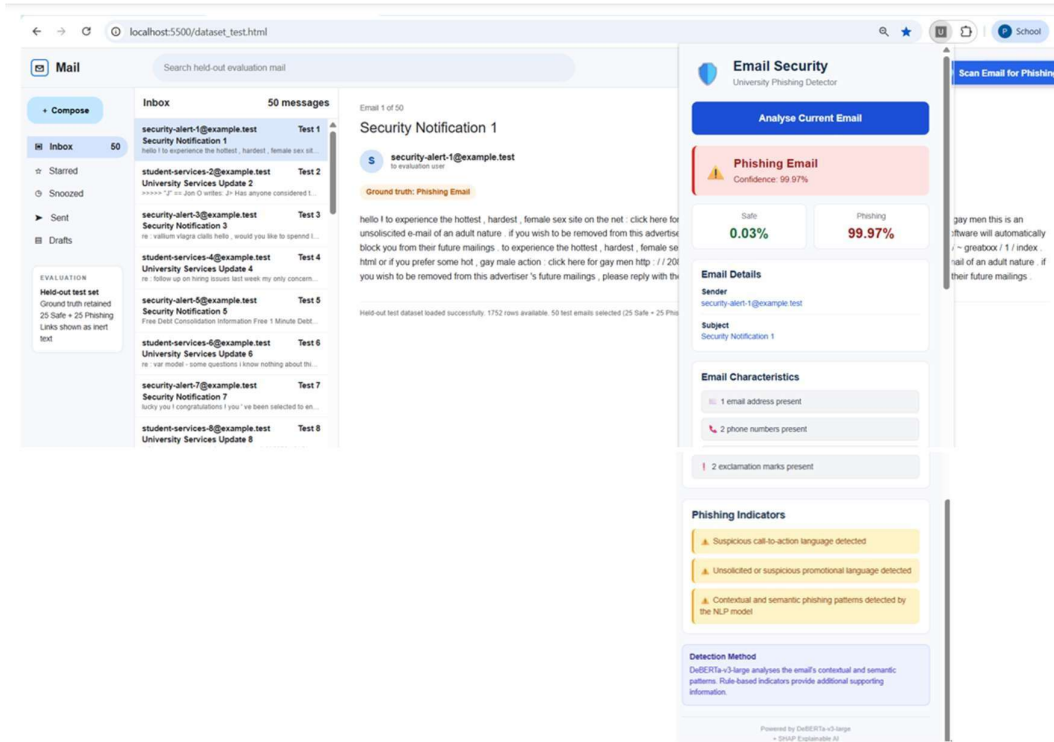


Figure 11.1.2.2: Browser-extension phishing-email demonstration in the controlled test environment.

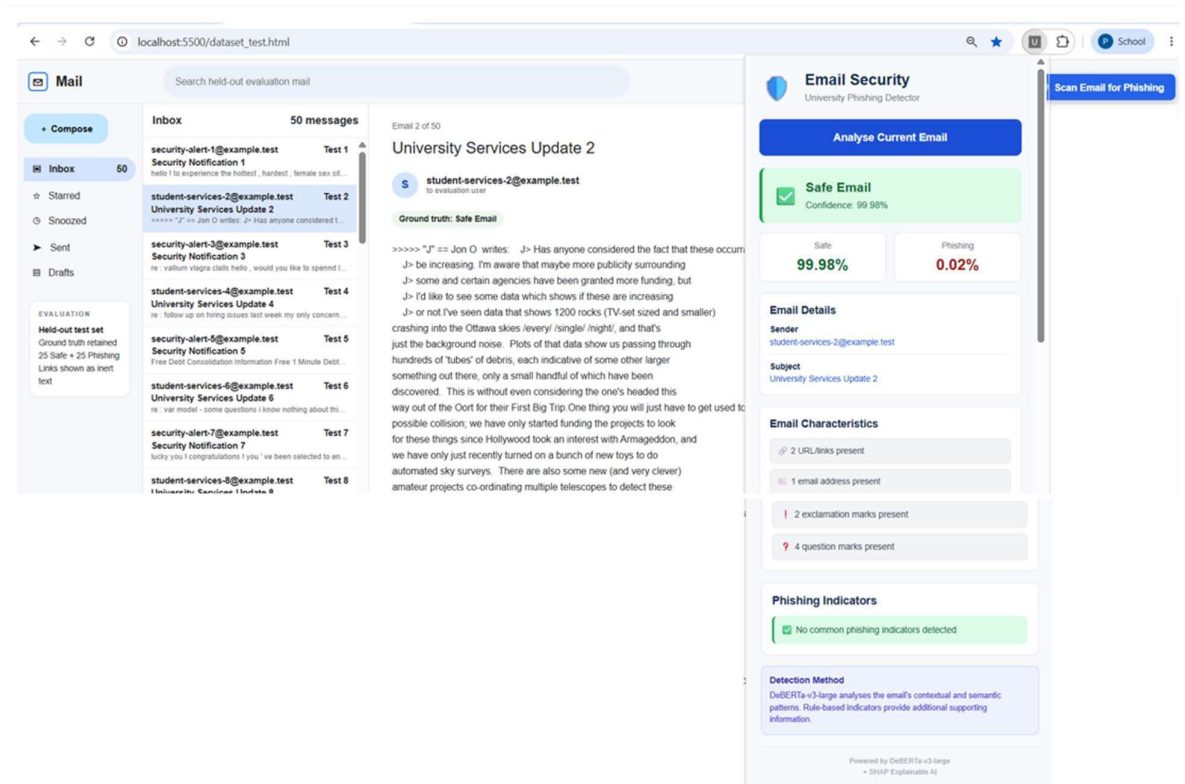


Figure 11.1.2.3: Browser-extension Safe-email demonstration using the same backend workflow.

Browser-extension interaction mechanism B - 'Scan Email for Phishing' button

The second browser-extension interaction mechanism used the injected 'Scan Email for Phishing' button on the controlled email page. This button is part of the browser-extension functionality. It provides an alternative trigger for the same browser-extension analysis pathway. The extension acquires the currently displayed email, communicates with the same FastAPI backend and receives the result from the same DeBERTa-v3-large gated-hybrid classifier. Its contribution is therefore primarily one of integration and usability, because analysis can be initiated directly while the user is viewing the message.

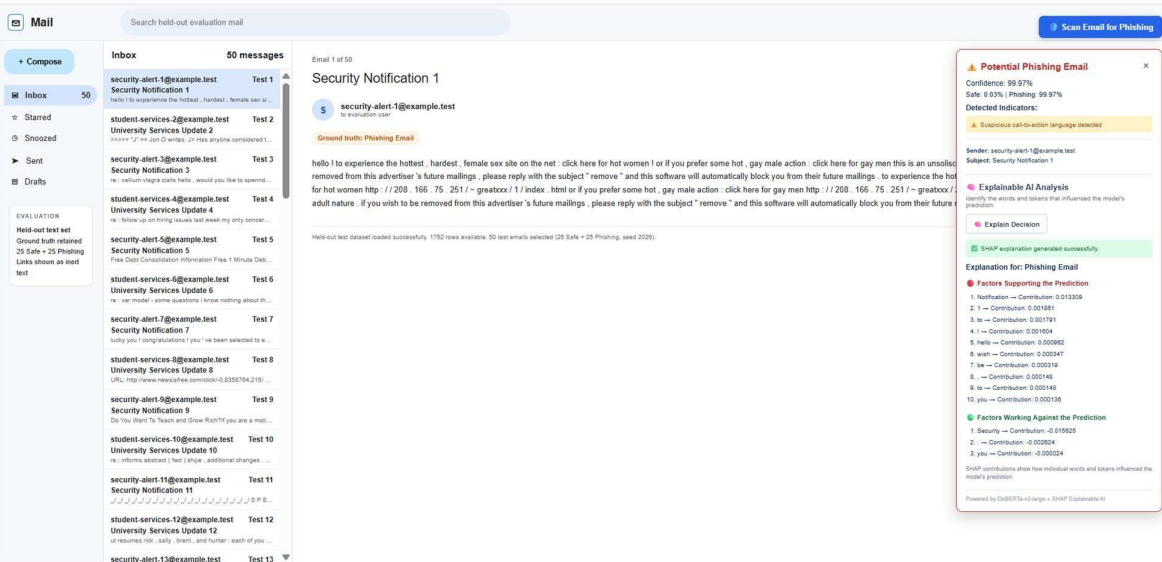


Figure 11.1.2.4: Phishing email demonstration using browser-extension integrated button

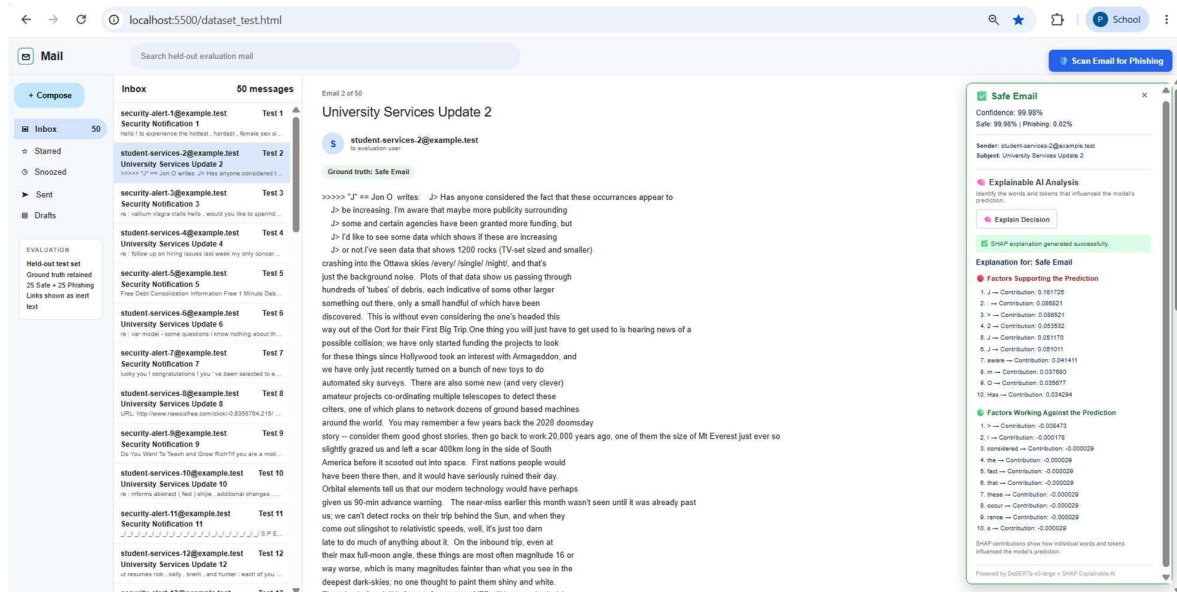


Figure 11.1.2.5: Safe email demonstration using browser-extension integrated button

11.1.2 Phishing / Legitimate Verdicts

The deployed system presents its primary decision as a binary classification between Safe Email and Potential Phishing Email. A phishing verdict is displayed when the phishing class receives the larger model probability, whereas a Safe verdict is displayed when the Safe class receives the larger probability. This binary presentation provides an understandable user-facing result while retaining the underlying probability distribution for more detailed interpretation.

The verdict should be treated as decision support rather than absolute proof of maliciousness or legitimacy. Machine learning classifiers can produce true positives, true negatives, false positives and false negatives. In a phishing context, a false positive occurs when a legitimate message is classified as phishing, potentially creating unnecessary concern or interrupting legitimate communication. A false negative occurs when a phishing email is classified as Safe and is generally security-sensitive because it may create unwarranted trust in a malicious message. Precision, recall and F1 therefore provide more informative evaluation than accuracy alone (Pedregosa et al. 2011).

A university or enterprise mail environment may value high phishing recall because missed malicious messages can be costly, while excessive false positives can also reduce trust in the tool and cause alert fatigue. NIST's risk-oriented approach similarly encourages security controls to be considered in relation to organisational context and risk rather than as isolated technical scores (Pascoe, Quinn & Scarfone 2024).

The screenshots in this sections show how these abstract model outputs become user-facing decisions. For example, several phishing demonstrations display a phishing probability around 99.97%, while safe examples display Safe probabilities around 99.98% or 99.99%. These

A particularly valuable demonstration is the borderline Streamlit case in which the model returned 54.33% Safe and 45.67% Phishing. Rather than presenting this result as equivalent to a 99% confidence classification, the interface displayed a low-confidence/manual-review warning. This behaviour demonstrates why probability information should accompany the binary label. In security-sensitive contexts, near-boundary predictions are appropriate candidates for human review, independent verification or additional security controls.

11.2 LIME-Based Deployment and End-to-End Demonstration

The LIME implementation was deployed as a controlled local prototype. The backend was implemented using Flask and was bound to 127.0.0.1 on port 5000, allowing applications running on the same computer to communicate with the prediction service without

intentionally exposing the API to other devices or the public Internet. The root endpoint provided a health response to confirm that the service was running, while email analysis was performed through POST /api/analyse. This local deployment also reduced the need to transmit potentially sensitive email content to an external third-party cloud service.

The classification component uses the same gated-hybrid approach based on DeBERTa-v3-large and twelve engineered NLP features. The trained model and associated artefacts, including the model checkpoint, DeBERTa configuration, tokenizer files, feature names and feature normalization statistics, were stored locally for deployment. When the Flask application was started, the PhishingPredictor was created, and the trained model was loaded into memory once rather than being reloaded for every email request. Predictions were subsequently performed using the loaded model in evaluation and PyTorch inference mode.

The primary user-facing interface for the LIME deployment was the Chrome Manifest V3 browser extension. The extension was implemented using components including manifest.json, popup.html and popup.js, while the machine-learning model and prediction logic remained within the Flask backend rather than being implemented in the browser. The extension requested the permissions required to access the active browser tab and communicate with the local API. This separation allowed the browser extension to operate as a lightweight client of the trained phishing-detection model.

Email content could enter the LIME workflow in two ways. When an email was opened in Gmail, the extension could attempt to retrieve the visible subject and body from the page. Alternatively, the user could manually copy and paste the email text for analysis. The manual input option was retained because changes to the Gmail interface or page structure could affect automatic extraction. Before submitting the request, the extension checked that the input was not blank and did not exceed the 30,000-character limit. The email content was then transmitted as JSON to the local Flask service for further server-side validation and analysis.

Following validation and preprocessing, the email was passed to the gated-hybrid classifier. The cleaned text was tokenized for DeBERTa-v3-large while the twelve engineered features were calculated concurrently and normalized using the training-derived statistics. The contextual and engineered-feature representations were then combined through the learned gate before the classifier produced the final class probabilities and verdict. After the prediction had been generated, LIME provided the post-hoc explanation by creating modified versions of the email and repeatedly querying the classifier. These results were used to fit an interpretable local model and identify terms with positive or negative weights relative to the class being explained.

The Flask API returned the analysis result as JSON, including the predicted label, verdict, selectedclass score and LIME explanation. The Chrome extension then presented the result to the user through the verdict, model confidence and influential terms. The API was also tested independently of the browser interface, demonstrating that the Flask backend could operate as

a standalone prediction and explanation service rather than depending entirely on the Chrome extension.

LIME therefore operates as an explanation mechanism associated with an already generated classification rather than as an independent phishing detector. Its weighted terms describe the local influence of selected words around the analyzed email and should not be interpreted as percentages, causal explanations or permanent phishing indicators. The LIME deployment consequently demonstrates how the trained classifier can be combined with a lightweight browser interface and local explanation mechanism to provide both a user-facing phishing prediction and additional information for interpreting that prediction.

11.1.3 Demonstration Scenario

Live Demonstration and End-to-End Testing

The live demonstration showcased the complete workflow using three email types: a legitimate workplace message, a conventional phishing email and an AI-rewritten phishing email. The presenter started the Flask service, verified its availability through the health endpoint, and analysed each email using the Chrome extension. The extension displayed the predicted verdict, confidence score and LIME explanation. Flask logs confirmed successful POST /api/analyse requests and responses.

On the original holdout test set, the model achieved 99.49% accuracy, 99.69% phishing precision, 98.93% phishing recall and 99.31% phishing F1-score. However, testing on unseen emails revealed weaker generalisation, with 53.33% of phishing emails in that set not detected. This highlights why high benchmark performance does not guarantee reliable detection of unfamiliar messages.

The implemented label mapping was verified across predictor.py, evaluate_unseen.py, the Flask API and the Chrome extension: 0 = legitimate and 1 = phishing. The API returned the predicted class and its associated confidence score. This score represents the model's output, not a calibrated probability that the email is genuinely malicious or safe. Formal confidence calibration was outside the project scope.

The demonstration confirmed that the prototype could process emails and present predictions with local explanations. Nevertheless, false positives may flag legitimate messages, while false negatives may allow phishing emails to appear legitimate. The system should therefore support, not replace, user judgement and existing organisational email security controls.

11.1.4 Confidence and LIME Explanations

The LIME interface presented the predicted class and confidence score alongside a local explanation of influential terms. Positive weights indicated terms that supported the explained class, while negative weights indicated terms that opposed it. These weights represented the relative influence of terms in a local surrogate model, not percentages, probabilities or direct measurements of DeBERTa's internal reasoning.

To balance repeatability and response time, the implementation used a fixed random seed and 50 perturbation samples per explanation. Although this reduced computational cost, the

limited number of samples could affect explanation stability. LIME explanations are local approximations, not causal evidence, and may vary with the sampling process and feature representation.

Providing explanations made the classification results easier to interpret than a label alone. However, an explanation does not guarantee a correct prediction. The unseen-email evaluation showed that the detector could produce confident but incorrect results. Explanations should therefore support human review rather than encourage users to accept predictions without question. 11.3 Combined Deployment and Demonstration Comparison

The SHAP and LIME based deployments provided complementary evidence that the trained phishing email classifier could operate as part of a complete user facing workflow rather than only as a standalone machine learning model.

Deployment aspect	SHAP based demonstration	LIME based demonstration
Primary demonstration focus	Demonstrated classification and explainability across multiple userfacing interactions.	Demonstrated browser-based classification, local explanation and independent API operation.
User-facing evidence	Streamlit interface and Chrome browser extension workflow.	Chrome browser extension workflow.
Browser interaction evidence	Extension popup and injected scan button were demonstrated as two interaction mechanisms within the same extension deployment.	Extension enabled email content to be submitted for analysis and the result returned directly to the user.
Explanation evidence	Displayed supporting, opposing and strongest SHAP factors for individual predictions.	Displayed influential terms and their local LIME weights for individual predictions.
Functional validation	Demonstrated the complete workflow from email input through classification, probabilities, indicators and optional explanation.	Demonstrated the complete workflow from email input through classification, confidence and LIME explanation.
Independent backend evidence	The same inference service was reused by different user-facing interfaces.	The API was additionally tested independently using PowerShell, demonstrating operation without dependence on the browser interface.
Demonstration coverage	Included controlled Safe, Phishing and lower-confidence examples across the implemented interfaces.	Included legitimate, conventional phishing and AI-rewritten phishing examples.

Main deployment outcome	Demonstrated reuse of the classifier and explanation capability across different interaction mechanisms.	Demonstrated successful browser-to-backend integration together with independent API functionality.
-------------------------	--	---

The combined demonstrations show that successful deployment involves more than obtaining a correct classification from the trained model. In both implementations, email content had to move successfully through the user interface, backend processing, trained classifier and explanation mechanism before the result could be presented to the user. The demonstrations therefore provide functional evidence that the individual components developed throughout the project can operate together as complete end-to-end workflows.

The two deployments also provided different forms of integration evidence. The SHAP implementation demonstrated reuse of the trained classifier across Streamlit and browser extension interactions, while the LIME implementation demonstrated browser-based operation together with independent API testing. These differences provide complementary evidence of system integration rather than representing competing versions of the classifier.

Importantly, successful deployment should not be interpreted as evidence of model generalization. A working interface and a correct prediction for an individual email demonstrate functional integration, whereas accuracy, precision, recall, F1-score and confusion-matrix results measure performance across labelled evaluation data. The deployment evidence in this chapter should therefore be interpreted together with the quantitative evaluation presented in Chapters 8 and 9. This distinction is particularly important because both implementations are intended as decision support prototypes rather than production email-security systems.

12. SECURITY, PRIVACY AND RISK MANAGEMENT

Security, privacy and risk management are essential considerations in developing an NLP-based phishing email detection system. Although the primary objective is to identify potentially malicious emails, the system must also protect sensitive email information, maintain the integrity of classification results and minimize vulnerabilities introduced through its interfaces and backend services. Accordingly, security considerations were examined in relation to the system architecture, data processing, application interfaces and machine-learning components.

The project implements a DeBERTa-v3-large gated-hybrid classifier supported by two explainability approaches: SHAP and LIME. The SHAP implementation integrates FastAPI, Streamlit and a Chrome browser extension, while the LIME implementation uses Flask and a browser extension workflow. Both configurations were developed as controlled local prototypes rather than production ready email security services. Security risks must therefore be distinguished from controls already demonstrated and safeguards recommended for future deployment.

12.1. Security Considerations

The security assessment of the proposed system considers confidentiality, integrity and availability, alongside risks associated with application interfaces, untrusted email content and machine learning predictions. These principles align with the NIST Secure Software Development Framework, which emphasizes integrating security practices throughout software development to minimize vulnerabilities (Souppaya, Scarfone & Dodson 2022).

Application and API security

The system separates users facing interfaces from backend inference services. In the SHAP implementation, Streamlit and the Chrome extension communicate with a locally hosted FastAPI backend at 127.0.0.1:8000, which provides prediction, explanation and health-check functionality. The LIME implementation uses a Flask backend at 127.0.0.1:5000 to process classification and explanation requests. This separation centralizes model execution and avoids exposing the trained classifier directly within browser side code.

Nevertheless, API endpoints introduce potential risks involving unauthorized requests, malformed inputs, excessive resource consumption and security misconfiguration. The SHAP prototype uses permissive Cross-Origin Resource Sharing (CORS) settings to facilitate local integration. While suitable for controlled development, such configurations should be restricted before production deployment. The OWASP API Security Top 10 identifies security misconfiguration and unrestricted resource consumption as significant.

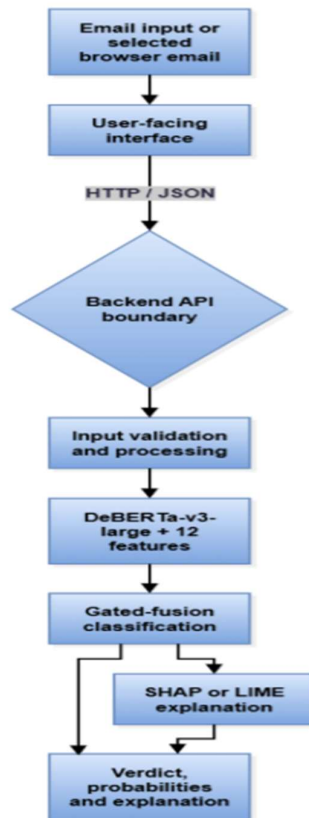


Figure 12.1: Security Boundaries within the Phishing Email Detection Architecture

Browser extension security

The Chrome browser extensions introduce additional security considerations because they interact with email content displayed within web pages. The SHAP implementation supports analysis through an extension popup and an injected scanning button, both connected to the same backend classification pipeline.

Browser extensions require appropriate permission management to prevent unnecessary access to webpage information. Chrome Manifest V3 supports the declaration of permissions and content scripts, making it important to restrict access to the websites and functionality required by the application (Google 2026). Furthermore, extracted email content must be treated as untrusted input to reduce risks associated with malicious markup or unexpected data.

The browser extension demonstrations were conducted in controlled email testing environments. Consequently, the implementation should not be presented as having undergone comprehensive security validation across production email platforms.

Email input and data integrity

Phishing emails may contain deceptive URLs, unusual characters, misleading instructions and potentially malicious content. The project's preprocessing pipeline applies HTML entity decoding, Unicode normalization, control character removal and HTML/XML tag removal while preserving relevant textual information. URLs and punctuation are retained because they may contribute to phishing detection.

Although these procedures improve input consistency, preprocessing alone does not guarantee protection against malicious inputs. A production system would require additional request validation, input size restrictions, safe output rendering and appropriate error handling to prevent misuse of application components.

Machine learning security and prediction reliability

The proposed classifier combines DeBERTa-v3-large contextual representations with twelve engineered phishing related features through a learned gated-fusion mechanism. The experimental evaluation reports an overall accuracy of 99.49% on the held out test dataset of 1,752 emails.

Despite this performance, the classifier remains susceptible to incorrect predictions and potentially unfamiliar phishing techniques. Attackers may modify message wording, structure or formatting to evade detection. Furthermore, a model trained on a fixed dataset may experience reduced effectiveness when exposed to new phishing campaigns or changing communication patterns.

False negatives are particularly significant because phishing emails incorrectly classified as safe may expose users to security threats. False positives may also affect usability by incorrectly identifying legitimate communications as suspicious. Therefore, the classifier should function as a decision-support mechanism rather than the sole security control.

Explainability and computational security

SHAP and LIME provide post-hoc explanations of individual predictions, helping users understand which textual characteristics influenced the classifier. However, explanation outputs do not independently establish whether an email is malicious and should not be interpreted as definitive security evidence.

Explainability also introduces computational overhead. The SHAP implementation separates the /predict and /explain endpoints so that ordinary classification can be performed without automatically generating a more computationally expensive explanation. This supports efficient system design, although production deployment would require additional safeguards such as request limits and resource monitoring.

Security risks and recommended controls

Table 12.1 summarizes the principal security risks associated with the implemented prototype and the corresponding controls recommended for future deployment.

Security area	Identified risk	Recommended control
API security	Unauthorized requests and insecure configurations	Authentication, restricted CORS and secure API configuration
Email processing	Malformed or malicious email content	Input validation, size restrictions and safe output rendering
Browser extension	Excessive permissions or unintended access to email information	Least privilege permissions and restricted host access
Model reliability	False classifications and evolving phishing techniques	Continuous evaluation, updated datasets and human oversight
Explainability	Misinterpretation of SHAP/LIME outputs	Clear explanations and appropriate confidence communication
System availability	Resource exhaustion from inference and explanation requests	Rate limiting, request limits and resource monitoring
Model and dependencies	Modified model files or vulnerable software packages	Integrity verification, dependency updates and vulnerability scanning

The security assessment demonstrates that the system incorporates a modular architecture that supports controlled integration of phishing detection and explainability. However, local deployment and successful functional testing do not establish production level security assurance. Further safeguards, including authentication, encrypted communication, stricter API configurations, dependency monitoring and security testing, would be necessary before deployment in an operational email environment.

Privacy and Data Handling

Privacy is an important consideration because email content may contain personal information, confidential correspondence and sensitive organisational details. The project was developed as a local prototype using FastAPI and Streamlit for the SHAP configuration and Flask for the LIME configuration. Local processing reduces reliance on external inference services, but does not eliminate risks associated with data transmission, storage or browser access.

Data Collection and Processing

The SHAP configuration accepts sender, subject and body information. The subject and body are combined for classification, while sender information is retained for contextual presentation. Email content is transferred between the browser extension, user interface and local backend through HTTP/JSON requests. The LIME configuration follows a similar local

processing approach. These workflows support selected-email analysis without requiring an externally hosted inference service during the demonstration.

The Australian Privacy Principles under the *Privacy Act 1988* (Cth) provide guidance on handling personal information, including its collection, use and security (OAIC 2025). A production implementation should minimise data collection, restrict browser permissions, require clear user action and protect communication between components using HTTPS/TLS and appropriate access controls.

Data Storage and Retention

The project uses a publicly available dataset containing 18,650 records, reduced to 17,519 after preprocessing, for model development and evaluation. Public availability does not guarantee that all records are free from sensitive information. Although the prototype processes emails locally, the available implementation documentation does not establish formal retention periods, automatic deletion or comprehensive logging controls. These should be addressed before deployment, including restricted access, secure deletion and avoiding unnecessary storage of complete email content.

12.2. Privacy Risks and Recommended Controls

Table 12.2 summarizes the principal privacy risks associated with the prototype and the safeguards recommended for future deployment.

Table 12.2: Privacy and data handling considerations

Privacy area	Potential risk	Recommended control
Email collection	Unnecessary collection of personal or confidential information	Collect only information required for classification
Data transmission	Exposure of email content during communication	Use encrypted transport and authenticated API access
Browser integration	Unintended access to unrelated email content	Restrict extension permissions and require user-initiated analysis
Data storage	Unnecessary retention of email content	Define retention periods and secure deletion procedures
Application logging	Disclosure of sensitive information through logs	Minimize logged content and restrict log access
Training datasets	Potential presence of personal information in publicly available records	Review dataset provenance, licensing and privacy implications

User transparency	Users may be unaware of how their email content is processed	Provide clear privacy notices and explain data handling practices
-------------------	--	---

12.3. Project Risk Register

Risk management was an important consideration throughout the development of the NLP-based phishing email detection system. The project involved multiple technical components, including dataset preprocessing, transformer-based model training, gated-hybrid architecture development, SHAP and LIME explainability, backend API integration and browser-based deployment. The interdependence of these components introduced technical, operational, security and project management risks that could affect system performance, development progress and the reliability of the final prototype.

A structured risk register was developed for this report to identify and assess the principal risks associated with the implemented system. The assessment follows the risk management principles outlined in ISO 31000:2018, which emphasize systematic risk identification, analysis, evaluation and treatment (International Organization for Standardization 2018).

Risks were assessed qualitatively according to their likelihood and potential impact. Likelihood was classified as Low, Medium or High, while impact was evaluated according to the potential consequences for system functionality, security, performance and project delivery. The resulting risk ratings indicate the relative priority of each identified issue.

Table 12.3: Project Risk Register

ID	Identified risk	Likelihood	Impact	Risk level	Proposed mitigation
R1	Incorrect phishing classification, including false positives and false negatives	Medium	High	High	Evaluate precision, recall, F1-score and confusion matrices, communicate prediction uncertainty
R2	Limited generalization to emerging phishing techniques and unfamiliar email formats	High	High	High	Expand dataset diversity, conduct external validation and periodically retrain the classifier
R3	High computational requirements of DeBERTa-v3-large and explainability methods	High	Medium	High	Separate prediction and explanation operations; optimize inference and monitor resource consumption

R4	Integration failures between the classifier, backend APIs and user-facing interfaces	Medium	High	High	Perform incremental integration, endpoint testing and end-to-end functional validation
R5	Exposure of sensitive email content during collection, processing or transmission	Medium	High	High	Minimise data collection, restrict access and introduce encrypted communication for production deployment
R6	API vulnerabilities, including insecure configurations and excessive requests	Medium	High	High	Restrict CORS, implement authentication, validate requests and introduce rate limiting
R7	Browser extension incompatibility with changing email platform interfaces	High	Medium	High	Improve DOM extraction robustness and test against different supported email environments
R8	Misinterpretation of SHAP or LIME explanations	Medium	Medium	Medium	Clearly distinguish explanations from classification decisions and provide understandable output descriptions
R9	Dataset imbalance, duplication or inconsistent preprocessing affecting evaluation reliability	Medium	High	High	Apply data cleaning, stratified splitting and consistent preprocessing across experiments
R10	Model checkpoint, dependency or configuration incompatibility	Medium	Medium	Medium	Maintain version controlled source code, document dependencies and verify model loading procedures

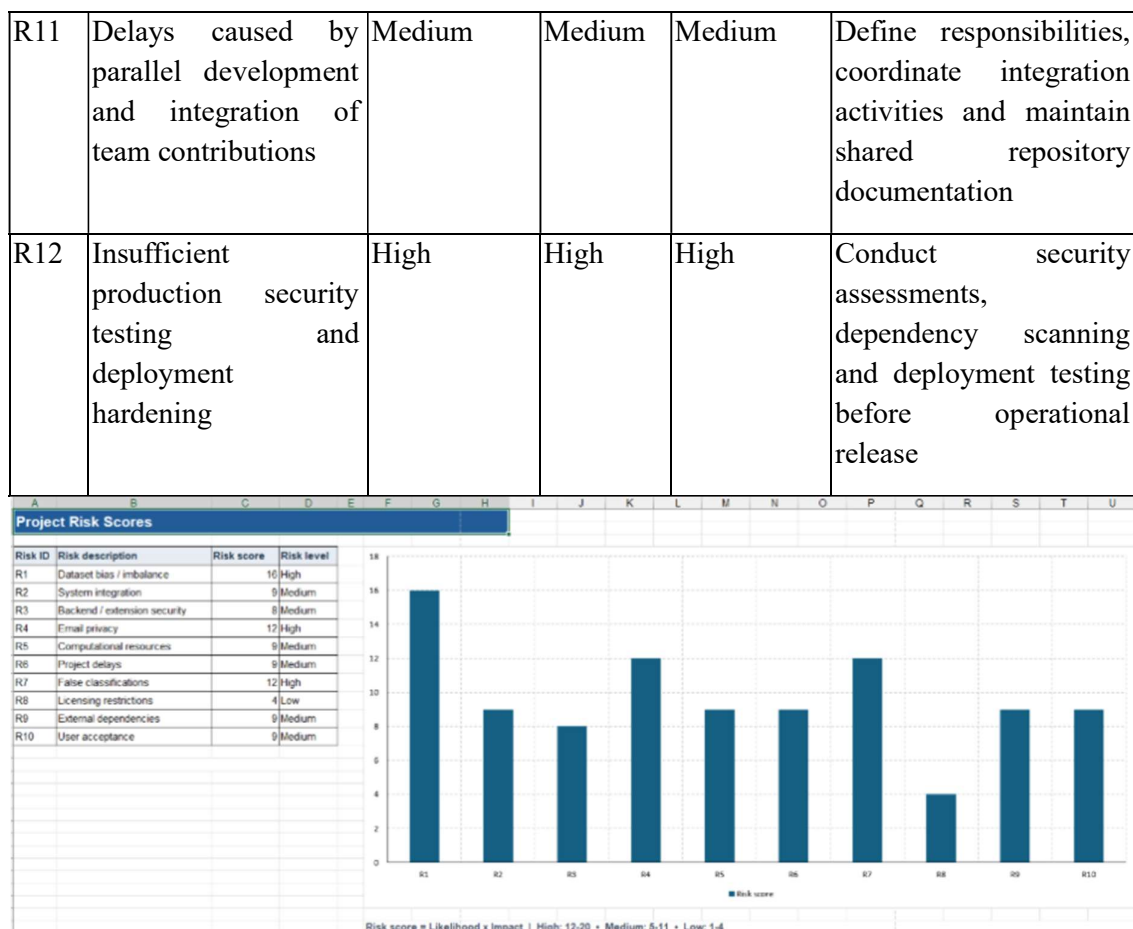


Figure 12.2.1: Project Risk Chart

12.4. Risk Mitigation

Risk mitigation focused on reducing technical, security, privacy and operational risks through preventive controls and testing. The approach follows the risk-treatment principles of ISO 31000:2018 (International Organization for Standardization 2018), while recognising that the system remains an academic prototype rather than a production-ready security service.

Model Reliability and Generalisation

Data quality was improved by removing missing values, empty records, duplicates and inconsistent labels before splitting the dataset into training (80%), validation (10%) and testing (10%) subsets. Classical, transformer-based and hybrid models were compared using accuracy, precision, recall, F1-score and confusion matrices. The proposed gated-hybrid DeBERTa-v3-large model achieved 99.49% accuracy on the reported evaluation. However, misclassification and reduced performance on unfamiliar emails remain risks. External dataset validation, adversarial testing and periodic model evaluation are recommended.

System Integration and Performance

A modular architecture separated the interfaces, backend services, classifier and explainability components. The SHAP configuration uses FastAPI with Streamlit and a Chrome extension, while the LIME configuration uses Flask. Controlled end-to-end tests checked email input, backend communication, classification and result presentation. Separating prediction from SHAP explanation generation also avoids unnecessary explanation processing. Model optimisation and resource monitoring would be needed for larger workloads.

Security and Privacy

Local inference reduced the need to transmit email content to external services during prototype testing. However, local processing does not eliminate privacy or API security risks. Production deployment would require appropriate authentication, restricted API access, HTTPS, request validation, rate limiting and data-retention controls. Browser extension permissions should also be limited to necessary functions. These recommendations align with the API security risks addressed by OWASP (OWASP Foundation 2023).

Explainability and User Interpretation

SHAP and LIME provide supporting explanations for individual predictions, but these explanations do not prove that an email is malicious. The classifier remains responsible for the final verdict. Clearer explanation labels, contextual warnings and user testing are recommended to reduce the risk of misinterpretation.

Project Coordination and Quality Assurance

A shared GitHub repository and modular development supported collaboration and independent component testing. Further improvements should include formal code reviews, documented integration requirements, dependency management and automated testing to reduce compatibility risks and improve maintainability.

Table 12.4.1: Risk Mitigation Strategies and Residual Risks

Risk category	Measures demonstrated in the project	Remaining mitigation requirements
Model reliability	Data cleaning, stratified splitting and comparative evaluation	External validation, adversarial testing and periodic retraining
System integration	Modular architecture and end-to-end functional testing	Automated integration testing and deployment monitoring
Computational performance	Separation of prediction and SHAP explanation	Model optimization, request limits and resource monitoring
API security	Controlled local backend deployment	Authentication, restricted CORS, HTTPS and rate limiting

Privacy	Local inference and limited classification inputs	Formal retention policies, access controls and privacy assessment
Browser integration	Controlled extension testing	Least privilege permissions and broader compatibility testing
Explainability	SHAP and LIME explanations accompanying predictions	User centered explanation design and usability evaluation
Project coordination	Shared repository and component-based development	Formal code review, dependency tracking and continuous integration

12.5. Ethical and Professional Considerations

The development of an artificial intelligence-based phishing detection system introduces ethical and professional responsibilities beyond achieving high classification accuracy. Although automated detection can assist users in identifying suspicious communications, incorrect predictions, inappropriate handling of personal information and misleading explanations may introduce additional risks. Consequently, the ethical acceptability of the proposed system must be evaluated in relation to its technical limitations, potential consequences for users and suitability for deployment in organizational environments.

The project incorporates a DeBERTa-v3-large gated-hybrid classifier, twelve engineered features and explainable artificial intelligence techniques, including SHAP and LIME. These components were developed to improve phishing identification and provide greater transparency regarding classification outcomes. Nevertheless, the inclusion of explainability mechanisms does not eliminate model uncertainty or guarantee that individual predictions are correct. Responsible implementation therefore requires consideration of fairness, transparency, privacy, accountability and the potential misuse of automated security decisions.

12.5.1 Ethical Issues in AI-Based Phishing Detection

One of the principal ethical concerns associated with phishing detection is the potential consequence of classification errors. A false negative occurs when a phishing email is classified as legitimate, potentially exposing users to credential theft, financial fraud or unauthorized disclosure of sensitive information. Conversely, a false positive occurs when legitimate communication is incorrectly identified as phishing, potentially disrupting business operations or causing users to disregard important messages.

These errors demonstrate why classification accuracy alone is insufficient for evaluating the practical reliability of a security system. Although the proposed gated-hybrid model achieved 99.49% accuracy on the held-out test dataset, this result does not establish equivalent performance across unfamiliar email environments or emerging phishing techniques. The ethical significance of prediction errors depends on their consequences, which may differ considerably between individual users and organizational contexts.

Another concern relates to the interpretation of model confidence. A high predicted probability represents the model's estimated confidence under its learned statistical relationships; it should not automatically be interpreted as an independently verified probability that an email is malicious. Research on neural network calibration has demonstrated that modern classification models may produce overconfident predictions, making appropriate confidence interpretation an important consideration (Guo et al. 2017).

The integration of SHAP and LIME was intended to improve transparency by identifying features that influence individual predictions. However, explanation methods provide approximations or attributions associated with model behavior rather than definitive evidence of malicious intent. Rudin (2019) argues that post-hoc explanations require caution in high stakes decision-making because they may not fully represent the underlying decision process.

Accordingly, the proposed system should function as a decision support mechanism rather than an autonomous authority for determining whether emails are safe. Users should retain the ability to review suspicious communications, consider additional contextual evidence and question classifications that appear inconsistent with the email content.

12.6. Societal and Organizational Implications

Phishing attacks present significant challenges for individuals and organizations because they exploit human judgement as well as technical vulnerabilities. Automated phishing detection systems can contribute to cybersecurity awareness by identifying potentially malicious communications before users interact with them. The proposed system seeks to support this objective by combining transformer-based classification with explanations that highlight influential textual characteristics.

At an organizational level, such systems may assist employees in identifying suspicious messages and reducing reliance on manual inspection. However, automated classification also creates the possibility of excessive dependence on algorithmic recommendations. Users may accept a legitimate verdict without further examination, particularly when the interface presents a high confidence score. This introduces a risk of automation bias, where machine generated outputs are trusted more readily than the available evidence justifies.

The societal implications extend to differences in user knowledge, language proficiency and familiarity with cybersecurity threats. A model trained predominantly on English language email data may perform less reliably when analyzing multilingual communications, culturally specific expressions or unfamiliar phishing tactics. Consequently, strong performance on a particular test dataset should not be interpreted as evidence of universal reliability.

The proposed system therefore offers potential benefits as an additional layer of phishing awareness, but it should complement rather than replace organizational security policies, employee education and established incident reporting procedures. Broader deployment would require evaluation across diverse users, communication environments and attack patterns.

12.6.1 Professional Responsibility and Accountability

Professional responsibility requires developers to evaluate both the capabilities and limitations of the systems they produce. The project's implementation of model evaluation, explainability mechanisms and controlled end-to-end testing provides evidence of attention to technical reliability and transparency. Nevertheless, professional accountability also requires recognition of limitations that remain unresolved.

The Association for Computing Machinery Code of Ethics emphasizes avoiding harm, respecting privacy, maintaining professional competence and conducting appropriate evaluations of computing systems (ACM 2018). These principles are relevant to the development of cybersecurity applications because inaccurate or misleading outputs may influence users' security decisions.

For the proposed phishing detection system, professional responsibility involves ensuring that experimental results are reported accurately, prediction explanations are not overstated, and deployment limitations are communicated clearly. Prototypes have acknowledged that the current prototype has not undergone the comprehensive operational validation necessary for unrestricted production use.

The responsibility for responding to incorrect classifications should not be transferred entirely to end users. Developers and deploying organisations must establish appropriate monitoring, feedback and review mechanisms, particularly where automated recommendations may affect important communications.

13. TEAMWORK FRAMEWORK

The project used a collaborative, modular approach to coordinate machine-learning research, software development and system integration. Work was organised around dataset preprocessing, baseline evaluation, hybrid model development, explainable AI and application deployment. This allowed individual components to be developed separately before integration into the complete phishing detection workflow.

13.1. Communication and Coordination

The team held two to three meetings per week through Microsoft Teams to review progress, discuss technical challenges, allocate tasks and agree on priorities. Microsoft Teams supported communication, GitHub facilitated code sharing and version control, and Microsoft OneDrive supported document sharing. Regular coordination helped maintain consistency between model development, backend services and user-facing applications.

13.2. Integration and Shared Responsibility

The project included two explainability configurations. The SHAP implementation used FastAPI, Streamlit and a Chrome browser extension, while the LIME implementation used a Flask backend and browser extension. Integration required compatible data formats and interfaces between the classifier, backend services and user-facing components. End-to-end demonstrations tested the workflow from email input and classification to explanation generation and result presentation.

13.3. Teamwork Outcomes

The collaborative approach supported the development of an integrated prototype incorporating the DeBERTa-v3-large gated-hybrid classifier, twelve handcrafted features, SHAP and LIME explanations, backend services and browser-based interfaces. The experience highlighted the importance of clear task allocation, shared documentation and coordinated testing. More formalised task tracking, code reviews and integration checkpoints could further improve collaboration and maintainability in future development.

13.4. Project Tools, Technologies and AI Usage

The development of the NLP-based phishing and social engineering email detection system required a combination of programming environments, machine learning frameworks, visual design applications, communication platforms and artificial intelligence assisted tools. These technologies supported different stages of the project, from initial research and dataset preprocessing to model experimentation, system integration, evaluation and technical documentation.

The selection of tools reflected the interdisciplinary nature of the project, which combined natural language processing, deep learning, cybersecurity, software engineering and collaborative project management. While programming frameworks enabled the implementation of the classification and explainability components, development environments supported experimentation and debugging. Collaboration and documentation tools facilitated the exchange of technical resources, communication between team members and preparation of project deliverables.

14. DEVELOPMENT ENVIRONMENTS AND PROGRAMMING TOOLS

Python was the primary programming language used throughout the project because of its extensive support for natural language processing, machine learning, deep learning and backend application development.

Google Colab was used for computational experimentation, including model training and evaluation, while Visual Studio Code (VS Code) supported local software development, debugging and integration of application components. These environments served complementary purposes: notebook-based experimentation enabled the team to investigate model behavior, whereas local development tools supported the construction and testing of the integrated prototype.

The project also incorporated specialized Python libraries and frameworks. Scikit-learn supported classical machine learning experiments and evaluation metrics, while PyTorch and Hugging Face Transformers provided the foundation for transformer-based model development. Pandas and NumPy supported dataset manipulation and numerical processing.

Table 14.1: Development Tools and Technologies Used in the Project

Tool or technology	Primary purpose	Application within the project
Python	Programming language	Dataset preprocessing, model development, evaluation and backend implementation
Google Colab	Cloud based notebook environment	Machine learning experimentation, model training and evaluation
Visual Studio Code	Integrated development environment	Local programming, debugging, backend development and interface integration
Scikit-learn	Machine learning library	TF-IDF-based classifiers, dataset splitting and evaluation metrics
PyTorch	Deep-learning framework	Transformer training, hybrid architecture implementation and inference
Hugging Face Transformers	Pretrained NLP model framework	Experimentation with DistilBERT, RoBERTa, DeBERTa and ModernBERT
Pandas and NumPy	Data processing libraries	Dataset cleaning, transformation and numerical feature processing

SHAP	Explainable AI framework	Feature attribution explanations for classification results
LIME	Explainable AI framework	Local explanations of individual email predictions
FastAPI	Python backend framework	Prediction and explanation endpoints for the SHAP implementation
Flask	Python web framework	Backend functionality for the LIME implementation
Streamlit	Web application framework	Interactive interface for email classification and SHAP explanations
HTML, CSS and JavaScript	Web development technologies	Chrome browser extension interfaces and email testing environments
Google Chrome	Browser and extension testing environment	Testing browser extension functionality and email scanning

14.1. Diagramming, Documentation and Visual Design Tools

Visual documentation was important for communicating the architecture and operation of the proposed system. Canva and draw.io were used to prepare diagrams and visual representations of the project's technical components.

These tools supported the presentation of model workflows, system architecture, application interactions and other technical processes. Visual representations were particularly useful for illustrating how email content passed through preprocessing, contextual and engineered feature extraction, gated fusion, classification and explainability components.

Canva supported the preparation and presentation of visual materials, while draw.io provided a diagramming environment for representing technical relationships and system workflows. The diagrams complemented written explanations by making complex interactions easier to follow.

15. ARTIFICIAL INTELLIGENCE TOOLS AND THEIR APPLICATION

Generative artificial intelligence tools were used to support selected research, programming and documentation activities. These included ChatGPT, Claude and Copilot.

AI assisted tools provided an additional source of technical guidance when investigating NLP algorithms, understanding transformer architectures, troubleshooting programming issues and improving the presentation of technical information.

However, AI-generated outputs were not treated as authoritative evidence of model performance or system functionality. The responsibility for implementing, evaluating and documenting the system remained with the project team.

Table 14.1: Artificial Intelligence Tools and Their Roles

AI tool	Purpose	Application and verification considerations
ChatGPT	Technical research and documentation assistance	Supporting understanding of NLP, transformer architectures and XAI concepts; assisting with technical explanations and report structure
Claude	Programming and technical assistance	Supporting code interpretation, debugging and exploration of implementation approaches
Copilot	AI-assisted programming	Providing code suggestions and development assistance within supported programming workflows

The use of these tools introduced important considerations regarding accuracy, academic integrity and professional responsibility. Generated code can contain implementation errors, while generated explanations may omit assumptions or present unsupported technical claims.

Consequently, AI-assisted work requires independent verification. For this project, appropriate verification includes executing and debugging suggested code, comparing explanations with technical documentation, checking research claims against credible sources and ensuring that reported evaluation results originate from actual experiments.

16. QUALITY ASSURANCE AND CHANGE MANAGEMENT

Quality assurance and change management were important to the development of the NLP-based phishing and social engineering email detection system because the project combined machine learning experimentation with multiple interconnected software components. Establishing a reliable prototype required more than achieving high classification accuracy. It also required evaluating model behavior, verifying communication between system components and managing technical changes as the project progressed.

The project adopted an iterative development approach, beginning with dataset preprocessing and baseline classification before progressing to transformer-based architecture, gated-hybrid modelling, explainable artificial intelligence (XAI) and application integration. Quality assurance activities were incorporated throughout these stages through comparative model evaluation, functional testing and end-to-end demonstrations.

Change management was particularly relevant because the project evolved from standalone classification experiments into an integrated system incorporating a DeBERTa-v3-large gated-hybrid classifier, SHAP and LIME explanations, backend services and browser-based interfaces. These developments introduced dependencies between components, requiring attention to compatibility, testing and technical documentation.

17. LIMITATIONS AND FUTURE WORK

The proposed DeBERTa-v3-large gated-hybrid model achieved 99.49% accuracy on the held-out test dataset. However, its performance on a single dataset does not guarantee reliable detection of unfamiliar or evolving phishing emails. The current system also has computational and deployment limitations, and its explanations require careful interpretation.

17.1. Current Limitations

The main limitations are summarised below.

Table 16.1: Current System Limitations

Limitation	Impact
Classification errors	Phishing emails may be missed, and legitimate emails may be flagged.
Computational cost	DeBERTa-v3-large and explanation methods require additional resources.
Explainability	SHAP and LIME provide approximate explanations, not proof of malicious intent.
Limited security signals	The system does not comprehensively verify sender identity, inspect attachments or check live URL reputation.
Prototype deployment	Real-world compatibility, security and operational reliability remain unverified.

17.2. Dataset and Generalisation Limitations

The project used the zefang-liu/phishing-email-dataset. After preprocessing, 17,519 emails remained: 10,978 legitimate and 6,541 phishing emails. The dataset was divided into training, validation and test subsets using an 80:10:10 stratified split.

Although this approach supports consistent model comparison, the dataset may not represent all phishing campaigns, organisations or writing styles. Duplicate removal was performed, but systematic separation of near-duplicate messages and related campaigns was not established. The model was also evaluated on a fixed dataset, so its performance against newer phishing techniques remains uncertain.

17.3. Future Work

Future development should prioritise testing independent datasets and newer phishing emails to assess generalisation. Additional improvements include monitoring changes in phishing behaviour, optimising inference and explanation speed, and incorporating sender authentication, URL reputation and attachment analysis. Further testing should also examine

AI-rewritten, obfuscated and multilingual emails. Finally, usability, security and compatibility testing is needed before integrating the prototype into operational email environments.

18. CONCLUSION

This project developed an explainable NLP-based phishing email detection system to identify phishing and legitimate emails through contextual language analysis and handcrafted phishing-related features. After comparing classical machine learning models, transformer architectures and hybrid approaches, the DeBERTa-v3-large gated-hybrid model was selected as the final classifier. It achieved 99.49% accuracy and an F1-score of 99.49% on the held-out test dataset, demonstrating strong classification performance under the evaluated conditions.

The project also integrated SHAP and LIME into two local prototypes to make individual predictions easier to interpret. The SHAP configuration combines a FastAPI backend, Streamlit interface and browser extension, while the LIME configuration uses a Flask backend and browser extension. Both demonstrate the integration of classification, prediction confidence and post-hoc explanations within a user-facing workflow.

The results show the potential of combining transformer-based language understanding with handcrafted features for phishing detection. However, high performance on a single dataset does not guarantee effectiveness against unfamiliar or evolving attacks. SHAP and LIME explanations also provide supporting information rather than proof of malicious intent. The current prototypes do not comprehensively verify sender authentication, inspect attachments or check live domain reputation, and they require further security, usability and deployment testing.

Overall, the project achieved its main objective by delivering a high-performing hybrid classifier and two explainability-based prototypes. Future work should prioritise external dataset validation, testing against newer and obfuscated phishing emails, improving computational efficiency, and integrating additional security signals. These improvements would help establish the system's reliability and practical value as a supplementary layer within existing email security controls.

REFERENCES

- ENISA 2024, *ENISA threat landscape 2024*, European Union Agency for Cybersecurity, viewed 8 September 2026, <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024>.
- Office of the Australian Information Commissioner 2025, *Chapter 11: APP 11 security of personal information*, viewed 8 September 2026, <https://www.oaic.gov.au/privacy/australian-privacy-principles/australian-privacy-principles-guidelines/chapter-11-app-11-security-of-personal-information>.
- Atawneh, S & Aljehani, H 2023, 'Phishing email detection model using deep learning', *Electronics*, vol. 12, no. 20, article 4261, <https://doi.org/10.3390/electronics12204261>.
- Kyaw, PH, Gutierrez, J & Ghobakhlou, A 2024, 'A systematic review of deep learning techniques for phishing email detection', *Electronics*, vol. 13, no. 19, article 3823, <https://doi.org/10.3390/electronics13193823>.
- Alhuzali, A, Alloqmani, A, Aljabri, M & Alharbi, F 2025, 'In-depth analysis of phishing email detection: evaluating the performance of machine learning and deep learning models across multiple datasets', *Applied Sciences*, vol. 15, no. 6, article 3396, <https://doi.org/10.3390/app15063396>.
- Meléndez, R, Ptaszynski, M & Masui, F 2024, 'Comparative investigation of traditional machine-learning models and transformer models for phishing email detection', *Electronics*, vol. 13, no. 24, article 4877.
- Alajaji, A 2026, 'Classical-first selective cascades for resource-constrained phishing email detection', *Electronics*, vol. 15, no. 14, article 3051.
- Chang, CM, Al-Andoli, MN & Zheng, C 2025, 'Hybrid phishing detection model: integrating BERT with TF-IDF for enhanced email security', *International Journal on Robotics, Automation and Sciences*, vol. 7, no. 3, pp. 43–48.
- He, P, Gao, J & Chen, W 2021, 'DeBERTaV3: improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing', *arXiv preprint arXiv:2111.09543*, <https://doi.org/10.48550/arXiv.2111.09543>.
- Kyaw, PH & Gutierrez, J 2026, 'Phishing email detection using subject and body text: a comparative study of transformers, Bi-LSTM, and traditional classifiers'.
- Lundberg, SM & Lee, SI 2017, 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, vol. 30, pp. 4765–4774.
- Ribeiro, MT, Singh, S & Guestrin, C 2016, '"Why should I trust you?": Explaining the predictions of any classifier', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, ACM, <https://doi.org/10.1145/2939672.2939778>.

Salih, AM, Raisi-Estabragh, Z, Galazzo, IB, Radeva, P, Petersen, SE, Lekadir, K & Menegaz, G 2025, 'A perspective on explainable artificial intelligence methods: SHAP and LIME', *Advanced Intelligent Systems*, vol. 7, no. 1, article 2400304, <https://doi.org/10.1002/aisy.202400304>.

Al-Subaiey, A, Al-Thani, M, Alam, NA, Antora, KF, Khandakar, A & Zaman, SMAU 2024, 'Novel interpretable and robust web-based AI platform for phishing email detection', *Computers & Electrical Engineering*, vol. 120, part A, article 109625, <https://doi.org/10.1016/j.compeleceng.2024.109625>.

Al-Fayoumi, M, Alhijawi, B, Abu Al-Haija, Q & Armoush, R 2024, 'XAI-PhD: Fortifying trust of phishing URL detection empowered by Shapley additive explanations', *International Journal of Online and Biomedical Engineering (iJOE)*, vol. 20, no. 11, pp. 80–101, viewed 10 September 2026, <https://online-journals.org/index.php/i-joe/article/view/49533>.

Al-Subaiey, A, Al-Thani, M, Alam, NA, Antora, KF, Khandakar, A & Zaman, SMAU 2024, 'Novel interpretable and robust web-based AI platform for phishing email detection', *Computers & Electrical Engineering*, vol. 120, part A, article 109625, doi:10.1016/j.compeleceng.2024.109625.

Atawneh, S & Aljehani, H 2023, 'Phishing email detection model using deep learning', *Electronics*, vol. 12, no. 20, article 4261, doi:10.3390/electronics12204261.

Australian Signals Directorate's Australian Cyber Security Centre (ASD's ACSC) 2026a, *Recognise and report scams*, Australian Government, viewed 5 September 2026, <https://www.cyber.gov.au/learn-basics/explore-basics/recognise-and-report-scams>.

Australian Signals Directorate's Australian Cyber Security Centre (ASD's ACSC) 2026b, *Scams*, Australian Government, viewed 5 September 2026, <https://www.cyber.gov.au/threats/types-threats/scams>.

Australian Signals Directorate's Australian Cyber Security Centre (ASD's ACSC) 2026c, *Spotting scams*, Australian Government, viewed 8 September 2026, <https://www.cyber.gov.au/protect-yourself/spotting-scams>.

Devlin, J, Chang, M-W, Lee, K & Toutanova, K 2019, 'BERT: Pre-training of deep bidirectional transformers for language understanding', *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, vol. 1, pp. 4171–4186, doi:10.18653/v1/N19-1423.

FastAPI 2026, *CORS (Cross-Origin Resource Sharing)*, FastAPI documentation, viewed 10 September 2026, <https://fastapi.tiangolo.com/tutorial/cors/>.

Fatima, Z, Tanveer, MH, Voicu, RC, Chakravarty, S, Ashfaq, M, Khan, M, Zaib, A & Desa, H 2025, 'Explainable AI for IOT devices and robotic communication phishing detection: A machine learning approach using LIME and SHAP', *Engineering, Technology & Applied Science Research*, vol. 15, no. 5, pp. 26478–26486, viewed 12 September 2026, <https://etasr.com/index.php/ETASR/article/view/11595>.

Ferrell, B, Raskin, SE & Zimmerman, EB 2023, 'Calibrating a transformer-based model's confidence on community-engaged research studies: decision support evaluation study', *JMIR Formative Research*, vol. 7, article e41516, doi:10.2196/41516.

Google 2023, *Manifest V3 migration checklist*, Chrome for Developers, viewed 7 October 2026, <https://developer.chrome.com/docs/extensions/develop/migrate/checklist>.

Google 2024, *Declare permissions*, Chrome for Developers, viewed 7 October 2026, https://developer.chrome.com/docs/extensions/mv3/declare_permissions.

Google 2026a, *Declare permissions*, Chrome for Developers, viewed 18 August 2026, <https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions>.

Google 2026b, *Extensions: Develop*, Chrome for Developers, viewed 18 August 2026, <https://developer.chrome.com/docs/extensions/develop>.

Google 2026c, *Manifest – content scripts*, Chrome for Developers, viewed 18 August 2026, <https://developer.chrome.com/docs/extensions/reference/manifest/content-scripts>.

Google 2026d, *Manifest file format*, Chrome for Developers, viewed 18 August 2026, <https://developer.chrome.com/docs/extensions/mv3/manifest>.

Google 2026e, *Message passing*, Chrome for Developers, viewed 18 August 2026, <https://developer.chrome.com/docs/extensions/develop/concepts/messaging>.

He, P, Gao, J & Chen, W 2023, 'DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing', *The Eleventh International Conference on Learning Representations*, viewed 7 October 2026, <https://openreview.net/forum?id=sE7-XhLxHA>.

He, P, Liu, X, Gao, J & Chen, W 2021, 'DeBERTa: Decoding-enhanced BERT with disentangled attention', *International Conference on Learning Representations (ICLR)*.

Ibrahim, M & Elhafiz, R 2026, 'Phishing email detection using BERT and RoBERTa', *Computation*, vol. 14, no. 2, article 46, viewed 9 September 2026, <https://www.mdpi.com/2079-3197/14/2/46>.

Kehkashan, T, Abdelhaq, M, Al-Shamayleh, AS, Huda, N, Yaseen, IA, Ahmed, AIAA & Akhunzada, A 2025, 'Explainable phishing website detection for secure and sustainable cyber infrastructure', *Scientific Reports*, vol. 15, article 41751, viewed 9 September 2026, <https://www.nature.com/articles/s41598-025-27984-w>.

Kyaw, PH, Gutierrez, J & Ghobakhlou, A 2024, 'A systematic review of deep learning techniques for phishing email detection', *Electronics*, vol. 13, no. 19, article 3823, doi:10.3390/electronics13193823.

Lundberg, SM & Lee, S-I 2017, 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, vol. 30, viewed 6 September 2026, <https://arxiv.org/abs/1705.07874>.

Mehdi Gholampour, P & Verma, RM 2023, 'Adversarial robustness of phishing email detection models', *Proceedings of the 9th ACM International Workshop on Security and Privacy Analytics*, pp. 67–76, doi:10.1145/3579987.3586567.

Office of the Australian Information Commissioner (OAIC) 2025, *Chapter 11: APP 11 – Security of personal information*, Australian Government, viewed 6 October 2026, <https://www.oaic.gov.au/privacy/australian-privacy-principles/australian-privacy-principles-guidelines/chapter-11-app-11-security-of-personal-information>.

OWASP Foundation 2023a, *Introduction – OWASP API Security Top 10*, OWASP Foundation, viewed 8 September 2026, <https://api-security.owasp.org/editions/2023/en/0x03-introduction/>.

OWASP Foundation 2023b, *OWASP API Security Top 10 – 2023*, OWASP Foundation, viewed 8 September 2026, <https://api-security.owasp.org/editions/2023/en/0x11-t10/>.

OWASP Foundation 2023c, *API4:2023 unrestricted resource consumption*, OWASP Foundation, viewed 7 October 2026, <https://api-security.owasp.org/editions/2023/en/0xa4-unrestricted-resource-consumption/>.

Pallets Projects 2025, *Deploying to production*, Flask documentation, viewed 7 October 2026, <https://flask.palletsprojects.com/en/stable/deploying/>.

Pascoe, C, Quinn, S & Scarfone, K 2024, *The NIST Cybersecurity Framework (CSF) 2.0*, NIST CSWP 29, National Institute of Standards and Technology, Gaithersburg, MD, doi:10.6028/NIST.CSWP.29.

Pedregosa, F, Varoquaux, G, Gramfort, A, Michel, V, Thirion, B, Grisel, O, Blondel, M, Prettenhofer, P, Weiss, R, Dubourg, V, Vanderplas, J, Passos, A, Cournapeau, D, Brucher, M, Perrot, M & Duchesnay, E 2011, 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830.

PyTorch 2026a, *CrossEntropyLoss*, PyTorch documentation, viewed 13 September 2026, <https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>.

PyTorch 2026b, *Softmax*, PyTorch documentation, viewed 13 September 2026, <https://docs.pytorch.org/docs/stable/generated/torch.nn.Softmax.html>.

Rigopoulos, K, Quinn, S, Pascoe, C, Marron, J, Mahn, A & Topper, D 2024, *NIST Cybersecurity Framework 2.0: Resource & Overview Guide*, NIST SP 1299, National Institute of Standards and Technology, Gaithersburg, MD, doi:10.6028/NIST.SP.1299.

Sajad, UP 2025, 'Explainable transformer-based email phishing classification with adversarial robustness', *arXiv*, arXiv:2511.12085, viewed 8 September 2026, <https://arxiv.org/abs/2511.12085>.

Salih, AM, Raisi-Estabragh, Z, Galazzo, IB, Radeva, P, Petersen, SE, Lekadir, K & Menegaz, G 2025, 'A perspective on explainable artificial intelligence methods: SHAP and LIME', *Advanced Intelligent Systems*, vol. 7, no. 1, article 2400304, doi:10.1002/aisy.202400304.

scikit-learn Developers 2026a, *f1_score*, scikit-learn documentation, viewed 15 September 2026, https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html.

scikit-learn Developers 2026b, *Metrics and scoring: quantifying the quality of predictions*, scikit-learn documentation, viewed 15 September 2026, https://scikit-learn.org/stable/modules/model_evaluation.html.

Streamlit 2026a, *Deployment concepts*, Streamlit documentation, viewed 10 September 2026, <https://docs.streamlit.io/deploy/concepts>.

Streamlit 2026b, *Deployment tutorials*, Streamlit documentation, viewed 10 September 2026, <https://docs.streamlit.io/deploy/tutorials>.

Streamlit 2026c, *Install Streamlit*, Streamlit documentation, viewed 10 September 2026, <https://docs.streamlit.io/get-started/installation>.

Streamlit 2026d, *Managing dependencies when deploying your app*, Streamlit documentation, viewed 10 September 2026, <https://docs.streamlit.io/deploy/concepts/dependencies>.

Tabassi, E 2023, *Artificial intelligence risk management framework (AI RMF 1.0)*, NIST AI 100-1, National Institute of Standards and Technology, Gaithersburg, MD, doi:10.6028/NIST.AI.100-1.

Uddin, KMM, Biswas, N, Rikta, ST, Nur-A-Alam, M & Mostafiz, R 2025, 'Explainable machine learning for phishing site detection: A high-efficiency approach using boosting models and SHAP', *The Journal of Engineering*, 2025, article 70110, viewed 11 September 2026, <https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/tje2.70110>.

Uddin, MA, Mahiuddin, M & Sarker, IH 2026, 'An explainable transformer-based model for phishing email detection: a large language model approach', *Computer Networks*, vol. 277, article 112061, doi:10.1016/j.comnet.2026.112061.

Vaswani, A, Shazeer, N, Parmar, N, Uszkoreit, J, Jones, L, Gomez, AN, Kaiser, L & Polosukhin, I 2017, 'Attention is all you need', *Advances in Neural Information Processing Systems*, vol. 30.

zefang-liu 2026, *Phishing email dataset*, Hugging Face, viewed 9 October 2026, <https://huggingface.co/datasets/zefang-liu/phishing-email-dataset>.

Dadgar, SMH, Araghi, MS & Farahani, MM 2016, 'A novel text mining approach based on TF-IDF and Support Vector Machine for news classification', *2016 IEEE International Conference on Engineering and Technology (ICETECH)*, Coimbatore, India, pp. 112–116, doi:10.1109/ICETECH.2016.7569223.

Sjarif, NNA, Azmi, NFM, Chuprat, S, Sarkan, HM, Yahya, Y & Sam, SM 2019, 'SMS spam message detection using term frequency-inverse document frequency and random forest algorithm', *Procedia Computer Science*, vol. 161, pp. 509–515.

Liu, Y, Ott, M, Goyal, N, Du, J, Joshi, M, Chen, D, Levy, O, Lewis, M, Zettlemoyer, L & Stoyanov, V 2019, 'RoBERTa: A robustly optimized BERT pretraining approach', *arXiv preprint arXiv:1907.11692*, viewed 9 October 2026, <https://arxiv.org/abs/1907.11692>.

Answer.AI 2024, *ModernBERT-large*, Hugging Face, viewed 9 October 2026, <https://huggingface.co/answerdotai/ModernBERT-large>.

Sanh, V, Debut, L, Chaumond, J & Wolf, T 2019, 'DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter', *arXiv preprint arXiv:1910.01108*, viewed 9 October 2026, <https://arxiv.org/abs/1910.01108>.

Jiao, X, Yin, Y, Shang, L, Jiang, X, Chen, X, Li, L, Zeng, X, Zhou, L & Liu, Q 2020, 'TinyBERT: Distilling BERT for natural language understanding', *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4163–4174.

He, P, Gao, J & Chen, W 2021, 'DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing', *arXiv preprint arXiv:2111.09543*, viewed 9 October 2026, <https://arxiv.org/abs/2111.09543>.

Sokolova, M & Lapalme, G 2009, 'A systematic analysis of performance measures for classification tasks', *Information Processing & Management*, vol. 45, no. 4, pp. 427–437.

Sajad U P 2026, 'A robust and explainable transformer-based framework for phishing email detection', *arXiv preprint arXiv:2511.12085*, viewed 9 October 2026, <https://doi.org/10.48550/arXiv.2511.12085>.

Tarapiah, S, Abbas, L, Mardawi, O, Atalla, S, Himeur, Y & Mansoor, W 2025, 'Evaluating the effectiveness of large language models (LLMs) versus machine learning (ML) in identifying and detecting phishing email attempts', *Algorithms*, vol. 18, no. 10, article 599.

Alajaji, A 2026, 'Classical-first selective cascades for resource-constrained phishing email detection', *Electronics*, vol. 15, no. 14, article 3051.